A semi-infinite constraint generation algorithm for two-stage robust optimization problems

Patxi Flambard*[1,2], Ayşe N. Arslan†[1,2], and Boris Detienne‡[1,2]

[1]Centre Inria de l'Universite de Bordeaux, F-33405 Talence, France
[2]Univ Bordeaux, CNRS, IMB, UMR 5251, F-33400 Talence, France

Abstract

In this paper, we consider two-stage linear robust optimization problems with continuous and fixed recourse. These problems have been the subject of exact solution approaches, notably constraint generation and constraint-and-column generation. Both approaches repose on an exponential-sized reformulation of the problem that uses a large number of constraints or constraints and variables. The decomposition algorithms then solve and reinforce a relaxation of the aforementioned reformulations through iterations that require the solution of bilinear separation problems. Here, we present an alternative approach that is based on a novel reformulation of the problem with an exponential number of semi-infinite constraints. We present a nested decomposition algorithm to deal with the exponential and semi-infinite natures of our formulation separately. We argue that our algorithm will result in a reduced number of bilinear separation problems being solved while providing high-quality relaxations. We additionally study possible MILP reformulations of the bilinear subproblem solved by these algorithms at each iteration, reviewing and consolidating the existing literature. Finally, we perform a detailed numerical study that showcases the superior performance of our proposed approach compared to the state of the art and evaluates the contribution of different algorithmic components. We do so on two distinct applications, a facility location transportation problem and a network design problem.

Keywords: Adjustable robust optimization, decomposition algorithms, constraint-and-column generation, linearization.

1 Introduction

Many real-life applications encountered in the context of optimization involve uncertain parameters due to measurement errors, variability and the time duration of the processes under study, or a simple lack of access to reliable data. There are different optimization under uncertainty paradigms that can take such uncertainty in the problem parameters into account. For instance, in stochastic optimization, uncertain parameters are characterized through a probability distribution, and a solution that performs best with respect to a statistical measure defined over this distribution is sought. On the other hand, distributionally robust optimization works with a family of distributions, called the ambiguity set, and seeks to find a

*patxi.flambard@inria.fr
†ayse-nur.arslan@inria.fr
‡boris.detienne@math.u-bordeaux.fr

solution that performs best with respect to a statistical measure under the worst-case distribution from this set. Finally, robust optimization is a distribution-free approach in which the uncertain parameters are assumed to belong to a typically compact set, called the uncertainty set, and a solution that performs best with respect to the worst realization in this set is sought. Each of these approaches can be appropriate, depending on the available data for characterizing the uncertain parameters and the level of risk that the decision-makers are willing to take.

In this paper, we focus on the robust optimization paradigm. Since its resurgence at the beginning of the century (Ben-Tal and Nemirovski, 1998; Bertsimas and Sim, 2004), robust optimization has been an active field of research. One of the main reasons for the popularity of this paradigm is that static robust optimization models with polyhedral or convex uncertainty sets lead to deterministic equivalent formulations that are often in the same complexity class as their deterministic counterparts. Advances in static robust optimization are presented in (Bertsimas et al., 2011; Gabrel et al., 2014b), and practical application of the static robust optimization paradigm is discussed in (Gorissen et al., 2015). On the other hand, static robust optimization is known to be "overly conservative" for certain applications. To remedy this problem, one can consider adjusting/adapting certain decisions (adjustability/adaptability) to the realization of uncertainty, leading to the adjustable robust optimization paradigm. In the following, we consider adjustable (two-stage) robust optimization problems where the recourse decisions are continuous. The applications that can be studied using this paradigm are numerous and diverse. Some examples from the recent literature include facility location (Cheng et al., 2021), resilient network design (Matthews et al., 2019), health care and inventory management (See and Sim, 2010; Tsang et al., 2023), and unit commitment and market clearing problems (Bertsimas et al., 2012; Vieira et al., 2023). We refer the reader to Yanıkoğlu et al. (2019) for further details and examples.

Adjustable robust optimization problems were first introduced by Ben-Tal et al. (2004) and are known to be NP-hard even when the first- and second-stage feasible regions, as well as the uncertainty set, are polyhedral (Guslitser, 2002). The authors, therefore, propose an approximate solution approach in which the second-stage variables are viewed as functions of uncertainty and their form is restricted to simpler functions to obtain easier-to-solve problems. This approach is known as a *decision rule approximation*. Ben-Tal et al. (2004) proposed, in particular, to restrict the recourse variables to be affine functions of uncertain parameters in the continuous recourse case. Under this restriction, an approximate adjustable solution can be obtained by solving a deterministic equivalent formulation, which introduces only a polynomial number of additional variables and constraints with respect to the deterministic counterpart of the problem. Although affine decision rule approximations can be shown to be optimal in certain special cases (Bertsimas et al., 2010), they are, in general, known to be suboptimal (Bertsimas and Goyal, 2012). Many studies in the literature have proposed measures of this suboptimality expressed as a gap between the optimal value of the affinely adjustable model and the (fully) adjustable model. These can be classified as a priori (Bertsimas and Goyal, 2012; Bertsimas and Bidkhori, 2015) and a posteriori bounds (Kuhn et al., 2011; Hadjiyiannis et al., 2011; Bertsimas and de Ruiter, 2016). In particular, it was established in (Bertsimas and Goyal, 2012) that the approximation ratio between the affinely and fully adjustable models scales with the square root of the number of second-stage constraints for certain problem classes. As such, many other decision rule approximations have been introduced to improve upon the affine decision rule approximation. These approximations often search for a trade-off between the complexity of the functional form imposed on the adjustable variables and the numerical difficulty of solving the resulting approximation. Some examples include, but are not limited to, extended and lifted affine (Chen and

Zhang, 2009; Georghiou et al., 2015) and piecewise affine (Bertsimas and Georghiou, 2015; Ben-Tal et al., 2020) decision rules.

One particular approach for creating piecewise affine and piecewise constant decision rules is known as uncertainty set partitioning (Postek and den Hertog, 2016; Bertsimas and Dunning, 2016). In this approach, the uncertainty set is iteratively partitioned into polyhedral subsets for each of which an affine or constant function is determined. This approach is similar in spirit to the finite/K-adaptability approach (Bertsimas and Caramanis, 2010; Subramanyam et al., 2020) where K second-stage functions are chosen in anticipation of uncertainty with a view to implementing the best of them once the uncertainty realization is observed. While uncertainty set partitioning approaches explicitly create partitions, the K-Adaptability approach does so implicitly. We additionally remark that both approaches can handle the random recourse case in which the recourse matrix depends on uncertain parameters.

Adjustable robust optimization problems with continuous and fixed recourse (*i.e.*, a constant recourse matrix) can also be solved exactly. The Benders-like constraint generation algorithm, introduced in (Bertsimas et al., 2012), is based on a monolithic reformulation of the problem with an exponential number of constraints, each representing a supporting hyperplane of the second-stage value function. On the other hand, Zeng and Zhao (2013) introduce a monolithic reformulation with an exponential number of variables and constraints and propose the constraint-and-column generation algorithm. In both algorithms, relaxations are solved and improved iteratively, where the feasibility of a relaxation solution is tested by solving a separation problem that is typically a bilinear or mixed-integer linear program. These approaches, initially introduced under the relatively complete recourse assumption, were later extended to handle infeasible first-stage solutions in Ayoub and Poss (2016), who additionally studied reformulation of separation problems as mixed-integer linear programs. Since the solution of separation problems in constraint and constraint-and-column generation algorithms can be numerically challenging, Goerigk and Kurtz (2022) proposed a data-driven heuristic to determine an initial set of relevant uncertainty realizations. Further, Dumouchelle et al. (2023) proposed to use machine learning to build an approximation of the recourse problem, which can be used both for separation problems and constraint-and-column generation relaxations. On the other hand, Tsang et al. (2023) proposed an inexact constraint-and-column generation algorithm, in which relaxations are solved to a given optimality tolerance that is reduced throughout the algorithm's iterations. They show that their approach can improve the numerical performance of the algorithm in applications where solving constraint-and-column generation relaxations poses a numerical challenge. They additionally establish theoretical guarantees for their approach to finitely converge to an optimal solution. From a different perspective, Zhen et al. (2018) detail a Fourier-Motzkin elimination approach to iteratively project out second-stage variables. This approach enables the derivation of a monolithic static robust formulation with an exponential number of constraints, which can then be solved directly. However, its use is limited to instances with few recourse variables due to the large number of constraints generated through projection. The authors therefore propose using their approach as a preprocessing step to improve other exact and approximate approaches, as it enables reducing the number of adjustable variables in the problem. Finally, Georghiou et al. (2020) propose primal and dual bounding problems for the adjustable robust optimization problem that are based on a constraint-and-column generation relaxation coupled with affine decision rules applied both to a primal and a dual formulation of the problem. They propose an algorithm in which extreme points of the uncertainty set are iteratively removed from the decision rule approximation and added to the constraint-and-column generation relaxation, improving both the primal and the dual bounds progressively, and converging to

an optimal solution. Their numerical results show that the proposed algorithm is outperformed by the constraint-and-column generation algorithm despite improving the detection of infeasible problems.

In this paper, we study the exact solution of linear adjustable (two-stage) robust optimization problems with fixed and continuous recourse. Our contributions can be summarized as follows:

- We present a novel exact decomposition algorithm for solving linear adjustable robust optimization problems with fixed and continuous recourse. This algorithm is based on reformulating the two-stage problem as a static robust optimization problem with an exponential number of semi-infinite constraints.
- We review the literature concerning the reformulation of bilinear subproblems as mixed-integer linear programming problems with a generalized approach to their feasibility and based on the structure of the uncertainty set. To the best of our knowledge, this is the first generalized presentation of the treatment of subproblems under various hypotheses.
- We conduct a detailed numerical study that highlights the efficiency of the proposed algorithm and analyzes the numerical contributions of different algorithmic components. We do so on two applications that are frequently studied in the literature: facility location-transportation and network design problems.

The remainder of this paper is organized as follows. Section 2 presents the theory underlying the proposed decomposition algorithm, initially focusing on the details of our algorithmic development before studying the reformulation of separation problems under different assumptions. The numerical experiments are presented in Section 3. We conclude the paper in Section 4 with some future research directions.

Notation: Throughout the paper, vectors are written in lowercase, matrices in uppercase, and sets in calligraphic letters. The vectors $\mathbf{0}$, $\mathbf{1}$, and e_i represent vectors of appropriate dimensions with all zeros and ones as entries, and the i^{th} canonical vector, respectively. The cardinality of a given set $\mathcal{X}$ is denoted by $|\mathcal{X}|$ while $u \circ v$ denotes the Hadamard product of vectors u and v of same dimension. For each mathematical model, the first label corresponds to the name of the model. Inequalities between vectors are considered to be component-wise. The objective value of any minimization problem without a feasible solution is set to $+\infty$. Finally, for a given positive integer n, the notation $[n] := [1, n] \cap \mathbb{N}$ denotes the set of integers between 1 and n. For a concise presentation, all proofs are relegated to Appendix A.

2 Methodological development

Let $\bar{x} \in \bar{\mathcal{X}} \subseteq \mathbb{Z}^{n^i_{\bar{x}}} \times \mathbb{R}^{n^c_{\bar{x}}}$, with $n_{\bar{x}} = n^i_{\bar{x}} + n^c_{\bar{x}}$, denote the first-stage decision variables, $\xi \in \Xi \subseteq \mathbb{R}^{n_\xi}$ denote the uncertain parameters and $y \in \mathbb{R}^{n_y}_+$ denote the second-stage decision variables, respectively. Let, further, $\bar{c} \in \mathbb{R}^{n_{\bar{x}}}$, $f \in \mathbb{R}^{n_y}$, $\bar{W} \in \mathbb{R}^{(m_y-1)\times n_y}$, $\bar{h}(\cdot) : \Xi \to \mathbb{R}^{m_y-1}$ and $\bar{T}(\cdot) : \Xi \to \mathbb{R}^{(m_y-1)\times n_{\bar{x}}}$ denote the associated objective and constraint coefficients. In this paper, we study linear adjustable (two-stage) robust optimization problems with fixed and continuous recourse formalized as:

$$\inf_{(\theta,\bar{x})\in\mathbb{R}\times\bar{\mathcal{X}}} \theta + \bar{c}^\top \bar{x} + \sup_{\xi\in\Xi} \inf_{y\in\mathbb{R}^{n_y}_+} \quad 0 \tag{$\bar{P}$}$$

$$\text{s.t.} \quad -f^\top y \geq -\theta \tag{1}$$

$$\bar{W}y \geq \bar{h}(\xi) - \bar{T}(\xi)\bar{x}, \tag{2}$$

where the second-stage problem is written as a feasibility problem following a similar convention to the literature (Ayoub and Poss, 2016; Georghiou et al., 2020). In this formulation, the auxiliary variable $\theta \in \mathbb{R}$ models the worst-case optimal value of the recourse problem with the help of the epigraph constraint (1). Constraints (2) describe the *linking constraints* between the first-stage variables x and the recourse variables y, where we assume that any deterministic (linear) constraints on y are included in (2).

Throughout, we assume that the following assumptions hold:

Assumption 1. *a. $\bar{\mathcal{X}}$ is compact; b. Ξ is a polytope; c. $\bar{T}(\cdot)$ and $\bar{h}(\cdot)$ are affine functions of ξ.*

Combining the variables θ and $\bar{x}$ and the constraints (1) and (2) with the following notations,

$$x = \begin{pmatrix} \theta \\ \bar{x} \end{pmatrix}, \quad c = \begin{pmatrix} 1 \\ \bar{c} \end{pmatrix}, \quad W = \begin{pmatrix} -f^\top \\ \bar{W} \end{pmatrix}, \quad h(\xi) = \begin{pmatrix} 0 \\ \bar{h}(\xi) \end{pmatrix}, \quad T(\xi) = \begin{pmatrix} 1 & \mathbf{0}^\top \\ \mathbf{0} & \bar{T}(\xi) \end{pmatrix},$$

and letting $\mathcal{X} = \mathbb{R} \times \bar{\mathcal{X}}$, problem $(\bar{P})$ can be reformulated as:

$$\min_{x \in \mathcal{X}} \; c^\top x + \max_{\xi \in \Xi} \; \min_{y \in \mathbb{R}^{n_y}_+} \; 0 \tag{P}$$

$$\text{s.t.} \;\; Wy \geq h(\xi) - T(\xi)x. \tag{3}$$

We remark that, in formulation (P), the infimum and supremum operators are replaced with minimum and maximum operators since, under Assumption 1, the respective infimum and supremum values are attained. Further, if for any $x \in \mathcal{X}$ and $\xi \in \Xi$, the set $\left\{y \in \mathbb{R}^{n_y}_+ \mid Wy \geq h(\xi) - T(\xi)x\right\}$ is empty, then the inner minimization problem evaluates to $+\infty$ by convention (as does the infimum operator). In such a case, both the maximum and the supremum operators evaluate to $+\infty$.

We study the exact solution of problem (P) throughout the paper. In the remainder of this section, we first recall the two classical decomposition algorithms, constraint generation (CG) (Bertsimas et al., 2012; Thiele et al., 2009) and constraint-and-column generation (C&CG) (Zeng and Zhao, 2013), that can be used to solve this problem exactly. We then present a novel reformulation of problem (P) and develop a decomposition algorithm for its solution. The relations between the developed algorithm and the CG and C&CG algorithms, as well as correctness and convergence properties, are studied. Finally, we analyze different reformulations of the associated separation problems with a generalized approach to their feasibility and based on the structure of the uncertainty set.

2.1 Reformulations and decomposition algorithms

In this subsection, we first present two reformulations of problem (P) and review the CG and C&CG algorithms, which are based on these reformulations. Then, we introduce a reformulation of problem (P) with an exponential number of semi-infinite constraints and develop the proposed algorithm. We additionally analyze how the mountain climbing algorithm (Konno, 1976) can be incorporated within the proposed algorithm and assess its numerical impact in Appendix B.3. All algorithms presented in this section are adapted to adjustable robust optimization problems with an optimality problem at the second stage in Appendix D to obtain primal bounds throughout their execution.

2.1.1 CG and C&CG algorithms

In the following, we derive a sequence of reformulations of problem (P) to transform the max-min term in the objective function into a finite but exponential number of linear constraints on first-stage variables x. We then provide the CG and C&CG algorithms that solve the obtained reformulations by generating only a subset of the aforementioned constraints.

We start by integrating the max-min term in the objective function into the constraints of the first-stage variables x so as to obtain an equivalent nonlinear monolithic reformulation of problem (P):

$$\min_{x\in\mathcal{X}} c^\top x \tag{$P1$}$$

$$\text{s.t.} \max_{\xi\in\Xi} \min_{\substack{y\in\mathbb{R}^{n_y}_+ \\ Wy\geq h(\xi)-T(\xi)x}} 0 \leq 0. \tag{4}$$

This formulation is equivalent to problem (P) since, for any given $x \in \mathcal{X}$, the left-hand side of (4) is equal to $+\infty$ if and only if there exists $\xi \in \Xi$ such that the inner minimization problem in formulation (P) is infeasible.

We next apply linear programming duality to the inner minimization problem of constraint (4), obtaining a monolithic maximization problem on the left-hand side of the constraint. Since the objective function of the primal problem is zero, the corresponding dual feasible set, denoted by $\Pi := \{\pi \in \mathbb{R}^{m_y}_+ \mid W^\top \pi \leq \mathbf{0}\}$, is a pointed polyhedral cone. Moreover, the dual solution $\pi = \mathbf{0}$ is feasible for any $W \in \mathbb{R}^{m_y \times n_y}$. We therefore conclude that the primal problem is infeasible if and only if the dual problem is unbounded. Further, since Π is a pointed cone, the dual problem is unbounded if and only if there exists a dual solution with a strictly positive objective value. We, therefore, equivalently obtain:

$$\min_{x\in\mathcal{X}} c^\top x \tag{$P2$}$$

$$\text{s.t.} \max_{\xi\in\Xi} \max_{\pi\in\Pi} \pi^\top (h(\xi) - T(\xi)x) \leq 0. \tag{5}$$

We finally transform the maximization over $\xi \in \Xi$ into a finite but exponential number of constraints. Using Assumptions 1 (b) and (c), for $\hat{x} \in \mathcal{X}$, the function $\xi \mapsto \max_{\pi\in\Pi} \pi^\top (h(\xi) - T(\xi)\hat{x})$ is the point-wise maximum of affine functions over a polytope. Thus, it is a convex function in ξ and, as such, only the extreme points of Ξ, denoted by $\text{ext}(\Xi)$, are needed to ensure a valid formulation:

$$\min_{x\in\mathcal{X}} c^\top x \tag{$P3$}$$

$$\text{s.t.} \max_{\pi\in\Pi} \pi^\top (h(\xi) - T(\xi)x) \leq 0 \qquad \forall \xi \in \text{ext}(\Xi). \tag{6}$$

In $(P3)$, we can further restrict Π to its extreme directions, since the left-hand side of each constraint is a linear programming problem defined over a pointed cone and the right-hand side is 0. We denote as $\text{dir}(\Pi)$ a minimal (with respect to inclusion) and finite set such that for each $\pi \in \text{dir}(\Pi)$, $||\pi|| = 1$ and $\Pi = \left\{\sum_{i\in[|\text{dir}(\Pi)|]} \lambda_i \pi_i \;\middle|\; \lambda \in \mathbb{R}^{|\text{dir}(\Pi)|}_+, \pi_i \in \text{dir}(\Pi)\ \forall i \in [|\text{dir}(\Pi)|]\right\}$. We then obtain the following reformulation of (P) on which the CG algorithm is based:

$$\min_{x\in\mathcal{X}} c^\top x \tag{P-CG}$$

$$\text{s.t.}\ \pi^\top (h(\xi) - T(\xi)x) \leq 0 \qquad \forall (\xi, \pi) \in \text{ext}(\Xi) \times \text{dir}(\Pi). \tag{7}$$

The reformulation of problem (P) on which the C&CG algorithm is based can be derived from either $(P1)$ or $(P3)$. From $(P1)$, using the fact that replacing Ξ with $\text{ext}(\Xi)$ suffices to obtain a valid formulation, one can write a linear deterministic equivalent formulation by introducing second-stage variables y^ξ for each realization $\xi \in \text{ext}(\Xi)$ and creating $|\text{ext}(\Xi)|$ copies of the recourse problem. Alternatively, one can view $(P3)$ as a static robust optimization problem with $|\text{ext}(\Xi)|$ semi-infinite constraints where π plays the role of the uncertain parameters and derive a deterministic equivalent formulation based on linear programming duality, denoting the dual variables y^ξ. Both approaches lead to the formulation:

$$\min_{x \in \mathcal{X}, y \in \mathbb{R}_+^{|\text{ext}(\Xi)| \times n_y}} c^\top x \qquad (P\text{-}C\&CG)$$

$$\text{s.t.} \qquad W y^\xi \geq h(\xi) - T(\xi)x \qquad \forall \xi \in \text{ext}(\Xi). \tag{8}$$

We remark that, although monolithic and linear, both formulations $(P\text{-}CG)$ and $(P\text{-}C\&CG)$ contain an exponential number of constraints, in general, not to mention the difficulty of enumerating the extreme points $\text{ext}(\Xi)$ and extreme directions $\text{dir}(\Pi)$. As such, both CG and C&CG algorithms will iteratively solve and reinforce a so-called master program, which is a relaxation of $(P\text{-}CG)$ or $(P\text{-}C\&CG)$, until the obtained relaxation is proven to be infeasible or its optimal solution $\hat{x} \in \mathcal{X}$ is proven to be feasible for problem (P). In the first case, we conclude that problem (P) is infeasible, while in the second case, $\hat{x}$ is proven to be optimal for problem (P) since the objective function of the master program and problem (P) are the same. To prove the feasibility of a given first-stage relaxation solution $\hat{x} \in \mathcal{X}$, using formulation $(P2)$, we verify if the optimal value of the bilinear problem:

$$\max_{\xi \in \Xi} \max_{\pi \in \Pi} \pi^\top \left(h(\xi) - T(\xi)\hat{x} \right), \qquad (SP\text{-}P\text{-}MaxMax)$$

is less than or equal to zero. However, this problem is, in most cases, unbounded. To address this issue, following (Ayoub and Poss, 2016), we bound the ℓ_1-norm of the dual variables to obtain a bounded bilinear problem:

$$z_{\text{SP}}(\hat{x}) := \max_{\xi \in \Xi} \max_{\pi \in \Pi_{\text{norm}}} \pi^\top \left(h(\xi) - T(\xi)\hat{x} \right). \qquad (SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$$

where $\Pi_{\text{norm}} := \{\pi \in \Pi | \mathbf{1}^\top \pi \leq 1\}$. Since $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$ has a strictly positive objective value if and only if $(SP\text{-}P\text{-}MaxMax)$ does, it can be used instead of the latter. We further remark that each extreme point of Π_{norm} except for point $\mathbf{0}$ corresponds to an extreme direction of Π. If $z_{\text{SP}}(\hat{x}) = 0$, then $\hat{x}$ is feasible for $(P\text{-}CG)$ and $(P\text{-}C\&CG)$, and optimal for problem (P). Otherwise, the master problem should be reinforced with additional constraints (and variables) in order to cut off the current infeasible solution $\hat{x}$. Mixed-integer linear programming formulations of $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$ are reviewed in Section 2.2.

Algorithm 3 (provided in Appendix B.1) summarizes the constraint generation algorithm proposed by Bertsimas et al. (2012). This algorithm is based on a relaxation of $(P\text{-}CG)$ defined by restricting the set of constraints to $\tilde{\Psi} \subseteq \text{ext}(\Xi) \times \text{dir}(\Pi)$:

$$z_{\text{CG}}(\tilde{\Psi}) := \min_{x \in \mathcal{X}} c^\top x \qquad (MP\text{-}CG(\tilde{\Psi}))$$

$$\text{s.t.} \;\; \pi^\top \left(h(\xi) - T(\xi)x \right) \leq 0 \qquad \forall (\xi, \pi) \in \tilde{\Psi}. \tag{9}$$

Algorithm 4 (provided in Appendix B.1) summarizes the constraint-and-column generation algorithm

proposed by Zeng and Zhao (2013) in which new variables and constraints are added to the relaxed master problem at each iteration of the algorithm. This algorithm is based on a relaxation of (P-$C\&CG$) defined by restricting the set of constraints to $\tilde{\Xi} \subseteq \text{ext}(\Xi)$:

$$z_{\text{C\&CG}}(\tilde{\Xi}) := \min_{x \in \mathcal{X}, y \in \mathbb{R}_+^{|\tilde{\Xi}| \times n_y}} c^\top x \qquad (MP\text{-}C\&CG(\tilde{\Xi}))$$

$$\text{s.t.} \quad Wy^\xi \geq h(\xi) - T(\xi)x \qquad \forall \xi \in \tilde{\Xi}. \qquad (10)$$

Assuming that the separation subproblem is solved to optimality at each iteration of both algorithms and that it returns extreme points of Ξ and Π_{norm}, respectively, the C&CG algorithm converges in at most $|\text{ext}(\Xi)|+1$ iterations. In contrast, the CG algorithm can take up to $|\text{ext}(\Xi)| \cdot |\text{dir}(\Pi)|+1$ iterations (since $\text{dir}(\Pi) = \text{ext}(\Pi_{\text{norm}}) \backslash \{\mathbf{0}\}$). For both algorithms, the number of bilinear separation subproblems solved is equal to the number of iterations. In practice, these numbers are rarely reached, and the C&CG algorithm often converges in fewer iterations than CG (Zeng and Zhao, 2013; Simchi-Levi et al., 2019). This behavior can be explained by the quality of the dual bound obtained by solving the C&CG master problem compared to that of CG. Indeed, by construction, for any $\tilde{\Psi} \subseteq \Xi \times \Pi$ and $\tilde{\Xi} = \{\xi \mid \exists \pi \text{ s.t } (\xi, \pi) \in \tilde{\Psi}\}$, we have that $z_{\text{CG}}(\tilde{\Psi}) \leq z_{\text{C\&CG}}(\tilde{\Xi})$, *i.e.*, the dual bound obtained from the C&CG master is at least as large as that of the CG master provided that the two contain the same uncertainty realizations $\tilde{\Xi}$. Since, in both algorithms, bilinear separation subproblems are solved at each iteration, the expected reduction in the number of iterations afforded by the C&CG algorithm can have a significant positive impact on the solution time. On the other hand, adding an entire second-stage system to the master problem at each iteration within the C&CG algorithm can also have a negative impact since this can hinder the solution of the master problem, especially if many scenarios are needed for convergence or the second-stage system has a complex structure (Tsang et al., 2023).

2.1.2 Semi-infinite constraint generation algorithm

The proposed algorithm aims to combine the advantages of CG and C&CG algorithms, that is, providing good-quality dual bounds without the addition of new variables and second-stage constraints. We start by deriving a reformulation of problem (P) on which the algorithm will be based. To do so, starting from ($P3$), we change the maximum operator over $\pi \in \Pi$ in (6) to an enumeration operator, to obtain equivalently:

$$\min_{x \in \mathcal{X}} c^\top x \qquad (P\text{-}SICG)$$

$$\text{s.t. } \pi^\top (h(\xi) - T(\xi)x) \leq 0 \qquad \forall \xi \in \text{ext}(\Xi), \forall \pi \in \Pi. \qquad (11)$$

Since Π is a polyhedral cone, model (P-$SICG$) is composed of an exponential number of semi-infinite constraints. To address the exponential nature of these constraints, similar to CG and C&CG algorithms, we employ a decomposition algorithm based on a relaxation of (P-$SICG$) defined by restricting the set of constraints to $\tilde{\Xi} \subseteq \text{ext}(\Xi)$:

$$z_{\text{SICG}}(\tilde{\Xi}) := \min_{x \in \mathcal{X}} c^\top x \qquad (MP\text{-}SICG(\tilde{\Xi}))$$

$$\text{s.t. } \pi^\top (h(\xi) - T(\xi)x) \leq 0 \qquad \forall \xi \in \tilde{\Xi}, \forall \pi \in \Pi. \qquad (12)$$

Then, to address the semi-infinite nature of the constraints, we interpret them as classical static robust constraints where π plays the role of the uncertain parameters. In doing so, we can utilize the two methods discussed in (Bertsimas et al., 2016; Fischetti and Monaci, 2012). The first approach involves applying LP duality to each semi-infinite constraint, resulting in a reformulation that is identical to the C&CG master problem, $(MP\text{-}C\&CG(\tilde{\Xi}))$. The second approach involves generating the necessary constraints dynamically through a cut generation algorithm. When applied to $(MP\text{-}SICG(\tilde{\Xi}))$, this approach restricts the set of constraints relative to each realization ξ in $\tilde{\Xi}$ to $\tilde{\Pi}^{\xi} \subseteq \text{dir}(\Pi)$. Let us further denote by $\tilde{\Pi} := (\tilde{\Pi}^{\xi})_{\xi \in \tilde{\Xi}}$ the family of sets of generated dual solutions associated with the generated realizations $\tilde{\Xi}$. The master problem of the constraint generation algorithm solving $(MP\text{-}SICG(\tilde{\Xi}))$ is then given as:

$$z_{\text{SICG}}(\tilde{\Xi}, \tilde{\Pi}) := \min_{x \in \mathcal{X}} c^\top x \qquad (MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$$

$$\text{s.t. } \pi^\top (h(\xi) - T(\xi)x) \leq 0 \qquad \forall \xi \in \tilde{\Xi}, \forall \pi \in \tilde{\Pi}^{\xi}. \qquad (13)$$

Any solution $\hat{x} \in \mathcal{X}$ of $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ is feasible for $(MP\text{-}SICG(\tilde{\Xi}))$ if for all $\hat{\xi} \in \tilde{\Xi}$ we have that $\max_{\pi \in \Pi_{\text{norm}}} \pi^\top (h(\hat{\xi}) - T(\hat{\xi})\hat{x}) \leq 0$. We refer to this linear problem as $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$. Otherwise, for some $\hat{\xi} \in \tilde{\Xi}$ there exists a $\pi^* \in \Pi_{\text{norm}}$ which satisfies ${\pi^*}^\top (h(\hat{\xi}) - T(\hat{\xi})\hat{x}) > 0$ that should be added to $\tilde{\Pi}^{\hat{\xi}}$. When no further violated cuts are found, the optimal value of $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ coincides with that of $(MP\text{-}SICG(\tilde{\Xi}))$ and therefore $(MP\text{-}C\&CG(\tilde{\Xi}))$. The proposed algorithm builds upon this perspective in order to further strengthen the master problem $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ without resorting to solving bilinear separation problems. Indeed, when a violated constraint ${\pi^*}^\top (h(\hat{\xi}) - T(\hat{\xi})\hat{x}) \leq 0$ is identified, one can perform a cut selection operation in order to determine if a more deeply violated cut can be found by solving the linear program $\max_{\xi \in \Xi} {\pi^*}^\top (h(\xi) - T(\xi)\hat{x})$, where the optimal value is necessarily strictly positive. We refer to this linear problem as $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$. If the optimal solution of this problem, ξ^*, is different from $\hat{\xi}$, then we will obtain a cut that is potentially deeper at $\hat{x}$. Further, ξ^* can now be incorporated into $\tilde{\Xi}$ (if it is not already present) in order to expand the set of realizations (called pool expansion in the following) from which violated cuts can be generated. In the following, we refer to this improved version of the constraint generation algorithm as the *augmented constraint generation algorithm*.

The overall SICG algorithm is thus composed of two decomposition algorithms, Algorithm 1 that solves problem (P) using $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$ as its separation subproblem and $(MP\text{-}SICG(\cdot))$ as its relaxed master problem, and the augmented constraint generation algorithm, Algorithm 2, that solves the latter using linear separation subproblems $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ and $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$ to cut infeasible solutions and $(MP\text{-}SICG(\cdot, \cdot))$ as its relaxed master problem.

In Algorithm 1, the relaxed master problem, $(MP\text{-}SICG(\tilde{\Xi}))$, initially defined with $\tilde{\Xi} = \emptyset$, is iteratively solved at Line 3 and reinforced until its solution $\hat{x} \in \mathcal{X}$ is proven feasible, and therefore optimal for problem (P), or (P) is proven infeasible. The feasibility of a given relaxation solution $\hat{x} \in \mathcal{X}$ is tested through the solution of bilinear separation problems at Line 5. If (ξ^*, π^*) such that ${\pi^*}^\top (h(\xi^*) - T(\xi^*)\hat{x}) > 0$ is identified then the semi-infinite constraint $\pi^\top (h(\xi^*) - T(\xi^*)x) \leq 0 \ \ \forall \pi \in \Pi$ is added to the master problem by augmenting the set $\tilde{\Xi}$ with the realization ξ^*. At this point, the set of corresponding dual solutions, $\tilde{\Pi}^{\xi^*}$, is also initialized with the dual solution π^*.

In Algorithm 2, the feasibility of the current optimal solution of $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ with respect to the already generated semi-infinite constraints corresponding to $\tilde{\Xi}$ is tested at Line 8. When a violated constraint is found, the cut selection phase is performed at Line 10, potentially exhibiting a new realization

ξ^*. In the case where ξ^* is a new realization, it is added to $\tilde{\Xi}_{new}$. At this point, the set of corresponding dual solutions $\tilde{\Pi}^{\xi^*}$ is initialized with the dual solution π^* at Line 15. By merging $\tilde{\Xi}_{new}$ with $\tilde{\Xi}$ at Line 19, the pool of semi-infinite constraints is (potentially) augmented in the pool expansion phase. We remark that the master problem ($MP\text{-}SICG(\cdot,\cdot)$) is re-solved only after the complete execution of the *for*-loop, *i.e.*, it is not re-solved each time a violated constraint is found. Further, any uncertainty realizations discovered through the pool expansion phase are not used to find additional violated constraints until a new first-stage solution $\hat{x}$ is computed.

Algorithm 1: Semi-Infinite Constraint Generation (SICG) algorithm

1 $\tilde{\Xi} \leftarrow \emptyset$, $\tilde{\Pi} := (\tilde{\Pi}^{\xi})_{\xi \in \tilde{\Xi}}$
2 **while** true **do**
3 Let $(\tilde{\Xi}, \tilde{\Pi}, \hat{x})$ be the output of Algorithm 2: Augmented constraint generation algorithm $(\tilde{\Xi}, \tilde{\Pi})$
4 **if** $(MP\text{-}SICG(\tilde{\Xi}))$ *is feasible* ($\hat{x} \neq$ null) **then**
5 Let (ξ^*, π^*) be an optimal solution of $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$
6 **if** ${\pi^*}^\top (h(\xi^*) - T(\xi^*)\hat{x}) > 0$ **then**
7 $\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \{\xi^*\}$
8 $\tilde{\Pi}^{\xi^*} \leftarrow \{\pi^*\}$
9 **else**
10 **return** *An optimal solution of* (P) *:* $\hat{x}$
11 **end**
12 **else**
13 **return** (P) *is infeasible*
14 **end**
15 **end**

Remark 1. *We remark that if the* for *loop over* $\hat{\xi} \in \tilde{\Xi}$ *(Lines 6-15) is removed from Algorithm 2 then Algorithm 1 becomes equivalent to the constraint generation algorithm described in Algorithm 3. We remark, however, that, in this case, the solution* $\hat{x}$ *returned by Algorithm 2 is no longer an optimal solution of* $(MP\text{-}SICG(\tilde{\Xi}))$.

Remark 2. *A dual variant of the semi-infinite constraint generation algorithm described in Algorithm 1 can be developed by applying the same algorithmic understanding starting from the reformulation:*

$$\min_{x \in \mathcal{X}} c^\top x \tag{14}$$

$$s.t.\ \ \pi^\top (h(\xi) - T(\xi)x) \leq 0 \qquad \forall \pi \in \text{dir}(\Pi), \forall \xi \in \Xi. \tag{15}$$

While the presented version of the algorithm enumerates over the extreme points of the uncertainty set Ξ *and adds semi-infinite constraints over the dual cone* Π*, this dual variant would enumerate over the extreme rays of the dual cone* Π *and add semi-infinite constraints over the uncertainty set* Ξ*. The nature and formulation of the bilinear separation problems that need to be solved are the same for both versions.*

We next show that Algorithm 1 terminates in a finite number of iterations and returns an optimal solution of problem (P) (or concludes that problem (P) is infeasible).

Assumption 2. *For* $\hat{x} \in \mathcal{X}$*,* $\hat{\xi} \in \Xi$*, and* $\pi^* \in \Pi_{\text{norm}}$*, subproblems* $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$*,* $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ *and* $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$ *are solved to optimality and return extreme points of* Ξ *and* Π_{norm}*, respectively.*

Algorithm 2: Augmented constraint generation algorithm $(\tilde{\Xi}, \tilde{\Pi})$

while true **do**
  $ctr\text{-}added \leftarrow$ false, $\tilde{\Xi}_{new} \leftarrow \emptyset$
  Solve $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ `// master problem phase`
  if $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ *is feasible* **then**
    Let $\hat{x}$ be an optimal solution of $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$
    for $\hat{\xi} \in \tilde{\Xi}$ **do**
      Let π^* be an optimal solution of $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ `// cut generation phase`
      if $\pi^{*\top}\left(h(\hat{\xi}) - T(\hat{\xi})\hat{x}\right) > 0$ **then**
        $ctr\text{-}added \leftarrow$ true
        Let ξ^* be an optimal solution of $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$ `// cut selection phase`
        if $\xi^* \in \tilde{\Xi} \cup \tilde{\Xi}_{new}$ **then**
          $\tilde{\Pi}^{\xi^*} \leftarrow \tilde{\Pi}^{\xi^*} \cup \{\pi^*\}$
        else
          $\tilde{\Xi}_{new} \leftarrow \tilde{\Xi}_{new} \cup \{\xi^*\}$
          $\tilde{\Pi}^{\xi^*} \leftarrow \{\pi^*\}$
        end
      end
    end
    $\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \tilde{\Xi}_{new}$ `// pool expansion phase`
    if $\neg ctr\text{-}added$ **then**
      return $(\tilde{\Xi}, \tilde{\Pi}, \hat{x})$ *with* $\tilde{\Xi}$ *and* $\tilde{\Pi}$ *updated,* $\hat{x}$ *an optimal solution of* $(MP\text{-}SICG(\tilde{\Xi}))$
    end
  else
    return $(\tilde{\Xi}, \tilde{\Pi}, \text{null})$ *with* $\tilde{\Xi}$ *and* $\tilde{\Pi}$ *updated,* $(MP\text{-}SICG(\tilde{\Xi}))$ *infeasible*
  end
end

We state the following results assuming that Assumption 2 holds.

Lemma 1. *Algorithm 2 called with* $(\tilde{\Xi}, \tilde{\Pi})$ *as input either returns an optimal solution of problem* $(MP\text{-}SICG(\tilde{\Xi}'))$ *or concludes that problem* $(MP\text{-}SICG(\tilde{\Xi}'))$ *is infeasible, with* $\tilde{\Xi} \subseteq \tilde{\Xi}'$.

Proposition 1. *Algorithm 1 either returns an optimal solution of problem* (P) *or concludes that the problem is infeasible.*

Proposition 2. *The number of iterations of Algorithm 1 is at most* $|\text{ext}(\Xi)|+1$*, and the cumulated number of iterations of the* while*-loop of Algorithm 2 is at most* $|\text{ext}(\Xi)|\cdot|\text{dir}(\Pi)|+1$.

Theorem 1. *Algorithm 1 requires the solution of at most* $|\text{ext}(\Xi)|+1$ *bilinear separation subproblems* $(SP\text{-}P\text{-}MaxMax)$ *and* $\mathcal{O}(|\text{dir}(\Pi)|\cdot|\text{ext}(\Xi)|^2)$ *linear separation subproblems* $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ *and* $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$.

As a result of Theorem 1, Algorithm 1 solves, in the worst case, the same number of bilinear separation subproblems as the C&CG algorithm, in addition to a number of linear separation subproblems solved through the execution of Algorithm 2. Nevertheless, we claim that Algorithm 1 may have a better numerical performance in practice. Indeed, for given $\tilde{\Xi} \subseteq \Xi$, both $(MP\text{-}SICG(\tilde{\Xi}))$ and the C&CG master problem, $(MP\text{-}C\&CG(\tilde{\Xi}))$, yield the same dual bound. However, while C&CG uses an entire second-stage system for each realization in $\tilde{\Xi}$, the proposed algorithm employs only cutting planes. Further, in

Algorithm 2 we may add elements to the set $\tilde{\Xi}$ through the pool expansion phase which means that the algorithm returns an optimal solution to $MP\text{-}SICG(\tilde{\Xi}')$ with $\tilde{\Xi}' \supseteq \tilde{\Xi}$. This is formalized in the following proposition which shows that for given $\tilde{\Xi}$, the execution of Algorithm 2 at Line 3 of Algorithm 1 leads to a dual bound that is at least as strong as the one obtained with the solution of $(MP\text{-}C\&CG(\tilde{\Xi}))$ at Line 3 of Algorithm 4.

Proposition 3. *Let $(\tilde{\Xi}', \tilde{\Pi}', \hat{x})$ be the output of Algorithm 2 with input $(\tilde{\Xi}, \tilde{\Pi})$, then $z_{\text{C\&CG}}(\tilde{\Xi}) \leq z_{\text{SICG}}(\tilde{\Xi}')$.*

As a result of Proposition 3 the number of times the bilinear separation subproblem is solved within the SICG algorithm can be significantly smaller than that of the C&CG algorithm.

We next study the special case of right-hand-side uncertainty, which may hold in many real-world applications.

Assumption 3. *The function $T(\xi)$ is constant, that is, $T(\xi) = T$ for all $\xi \in \Xi$.*

Lemma 2 shows that the cut selection phase yields dominating constraints if Assumption 3 is satisfied. As a consequence, it suffices to consider at most one uncertainty realization for each dual solution in $\text{dir}(\Pi)$ to obtain a reformulation of problem (P).

Lemma 2. *Let us assume that Assumption 3 holds. Then:*

a) For all $\hat{\pi} \in \Pi$, the set of optimal solutions of $(SP\text{-}P(x, \Xi, \{\hat{\pi}\}))$ is independent of $x \in \mathcal{X}$.

b) For all $\hat{\pi} \in \Pi$, $x \in \mathcal{X}$ and $\xi \in \Xi$, we have that $\hat{\pi}^\top (h(\xi) - Tx) \leq \hat{\pi}^\top (h(\xi^*_{\hat{\pi}}) - Tx)$, where $\xi^*_{\hat{\pi}} \in \Xi$ is any optimal solution of $(SP\text{-}P(x, \Xi, \{\hat{\pi}\}))$.

c) For given $\hat{\pi} \in \text{dir}(\Pi)$, let $\xi^*_{\hat{\pi}}$ be any optimal of $(SP\text{-}P(x, \Xi, \{\hat{\pi}\}))$ for any $x \in \mathcal{X}$. Let, further, $\tilde{\Xi} := \{\xi^*_{\hat{\pi}} \mid \hat{\pi} \in \text{dir}(\Pi)\}$, $\tilde{\Pi}^{\xi^*_{\hat{\pi}}} := \{\hat{\pi}\}$ for $\hat{\pi} \in \text{dir}(\Pi)$, and $\tilde{\Pi} := (\tilde{\Pi}^{\xi})_{\xi \in \tilde{\Xi}}$. We then have that $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ is equivalent to problems $(P\text{-}SICG)$ and (P).

As a result of Lemma 2, when Assumptions 2 and 3 are satisfied, the worst-case complexity of Algorithms 1 and 2 is improved. This result is formalized in Theorem 2.

Theorem 2. *If Assumptions 2 and 3 are satisfied, then the number of iterations of Algorithm 1 is bounded by* $\min(|\text{ext}(\Xi)|, |\text{dir}(\Pi)|) + 1$. *Moreover, the cumulated number of iterations of the* while*-loop of Algorithm 2 is bounded by $|\text{dir}(\Pi)|+1$.In this case, Algorithm 1 requires the solution of at most* $\min(|\text{ext}(\Xi)|, |\text{dir}(\Pi)|) + 1$ *bilinear separation subproblems $(SP\text{-}P\text{-}MaxMax)$, and $\mathcal{O}(|\text{ext}(\Xi)| \cdot |\text{dir}(\Pi)|)$ linear separation subproblems, $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ and $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$.*

We remark that a similar result to Theorem 2 was proven in Zeng and Wang (2022) concerning the C&CG algorithm for adjustable robust optimization problems with decision-dependent uncertainty sets.

2.2 MILP reformulations for the separation subproblem

In Section 2.1 and Appendix D, we described algorithms that solve linear adjustable (two-stage) robust optimization problems with fixed and continuous recourse. Within these algorithms, one needs to repeatedly verify whether a given first-stage solution $\hat{x} \in \mathcal{X}$ is feasible or whether its worst-case value is correctly evaluated, and to reinforce the associated relaxation if not. From an algorithmic point of view, this can be achieved using an oracle that either certifies the feasibility/optimality of the given solution or returns a

certificate of its infeasibility/true worst-case value that can then be used to strengthen the relaxation. In Section 2.1, we propose to solve $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$ for this purpose. However, this is a bilinear problem and most optimization solvers cannot provide its global optimal solution within a reasonable solution time. As such, in this section, we review MILP formulations for this problem. The main focus of our analysis is

$$\max_{\xi \in \Xi} \min_{y \in \mathbb{R}^{n_y}_+} 0 \qquad (SP\text{-}P\text{-}MaxMin(\hat{x}))$$
$$\text{s.t. } Wy \geq h(\xi) - T(\xi)\hat{x},$$

which led to the derivation of $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$ in Section 2.1 in the context of decomposition algorithms for problem (P). The treatment of subproblems in the context of decomposition algorithms for adjustable robust optimization problems with an optimality problem at the second stage is very similar. Corresponding mathematical models are provided in Appendix D for completeness.

The analysis is divided into two sections, considering the duality paradigm used and the structure of the uncertainty set. These properties determine the linearization technique to be used as well as whether uncertain parameters can be assumed to be binary. Our review is based on (Zeng and Zhao, 2013; Ayoub and Poss, 2016) and generalizes the formulations presented therein through the use of artificial variables. We further establish the relationship between the use of artificial variables and normalization constraints imposed on dual variables in the literature (Ayoub and Poss, 2016).

In order to cast $(SP\text{-}P\text{-}MaxMin(\hat{x}))$ as an MILP, two steps are required: first, the max-min problem is reformulated as a monolithic bilinear maximization problem. The resulting bilinear problem is then linearized using various reformulation techniques. For the first step, it is common to either use linear programming (LP) duality or the Karush-Kuhn-Tucker (KKT) optimality conditions on the inner minimization problem, with the latter requiring the relatively complete recourse assumption to be satisfied. For the second step, depending on the approach used for reformulating the inner problem, one can either rely on the McCormick linearization to linearize bilinear terms or on big-M constraints to linearize complementary slackness conditions, which are viewed as disjunctions. Both approaches require the terms involved in bilinear products or disjunctions to be bounded so that finite big-M coefficients are guaranteed to exist.

At this point, we remark that, in the context of decomposition algorithms for problem (P), the inner minimization problem in $(SP\text{-}P\text{-}MaxMin(\hat{x}))$ has an optimal solution for all $\xi \in \Xi$ only when the first-stage solution $\hat{x} \in \mathcal{X}$ is feasible, *i.e.*, at convergence of the proposed algorithms. We, therefore, consider the following problem instead, which estimates how far the second-stage problem is from being feasible in the worst case for a given first-stage solution $\hat{x} \in \mathcal{X}$:

$$\max_{\xi \in \Xi} \min_{\alpha \in \mathbb{R}^{n_\alpha}_+, y \in \mathbb{R}^{n_y}_+} \mathbf{1}^\top \alpha \qquad (SP\text{-}P\text{-}ArtMaxMin(\hat{x}))$$
$$\text{s.t.} \quad Wy + N\alpha \geq h(\xi) - T(\xi)\hat{x}. \qquad (16)$$

With the artificial variables $\alpha \in \mathbb{R}^{n_\alpha}_+$ and any matrix $N \in \mathbb{R}^{m_y \times n_\alpha}_{>0}$, the inner minimization problem in $(SP\text{-}P\text{-}ArtMaxMin(\hat{x}))$ is feasible for any first-stage solution $\hat{x} \in \mathcal{X}$ and any $\xi \in \Xi$. We remark that we do not need the inner minimization problem to be feasible for applying LP duality. However, creating an artificially feasible problem has the advantage of unifying our discussion on the use of both LP duality and KKT optimality conditions, as well as creating bounded dual problems, as we show in the following.

As a result of its feasibility for any $\hat{x} \in \mathcal{X}$ and $\xi \in \Xi$, both LP duality and KKT optimality conditions can be used to reformulate the inner minimization problem of $(SP\text{-}P\text{-}ArtMaxMin(\hat{x}))$. Further, $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$ can be replaced with $(SP\text{-}P\text{-}ArtMaxMin(\hat{x}))$ in the algorithms described in Sections 2.1.1 and 2.1.2 without further modification since each model has a strictly positive objective value if and only if the other does. Indeed $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$ is obtained by applying LP duality to a special case of $(SP\text{-}P\text{-}ArtMaxMin(\hat{x}))$ where $n_\alpha = 1$ and $N = \mathbf{1}$, *i.e.*, the same artificial variable is used for all second-stage constraints with a coefficient of one.

2.2.1 Subproblem reformulation based on LP duality

Using LP duality on the inner minimization problem of $(SP\text{-}P\text{-}ArtMaxMin(\hat{x}))$, we obtain the dual set $\Pi_N := \{\pi \in \Pi | N^\top \pi \leq \mathbf{1}\}$. This set is non-empty and bounded for any $N \in \mathbb{R}_{>0}^{m_y \times n_\alpha}$ thanks to the constraints $N^\top \pi \leq \mathbf{1}$ upper bounding the dual variables π (we recall that $\pi \in \mathbb{R}_+^{m_y}$). We next turn our attention to the linearization of the bilinear formulation:

$$\max_{\xi \in \Xi} \max_{\pi \in \Pi_N} \pi^\top (h(\xi) - T(\xi)\hat{x}) \,. \tag{17}$$

To obtain an exact reformulation of a bilinear product using the McCormick linearization, in addition to the boundedness of the variables involved, we also need at least one of the two variables to be binary. Let, as proposed in Ayoub and Poss (2016), $\Omega \subseteq \mathbb{R}^{n_\omega}$ be a polyhedron with binary extreme points such that the uncertainty set Ξ can be obtained as an affine transformation of Ω, *i.e.*, there exists a matrix $A \in \mathbb{R}^{n_\xi \times n_\omega}$ such that $\Xi := \{A\omega \mid \omega \in \Omega\}$. Using this new uncertainty set, we obtain the equivalent model:

$$\max_{\omega \in \Omega} \max_{\pi \in \Pi_N} \pi^\top (h(A\omega) - T(A\omega)\hat{x}) \qquad (SP\text{-}P\text{-}Dual(\hat{x}))$$

Since we only need the extreme points of the uncertainty set to ensure a valid reformulation and the extreme points of Ω are assumed to be binary, we may then restrict the uncertain parameters ω to be binary. As a result, the bilinear terms in $(SP\text{-}P\text{-}Dual(\hat{x}))$ involve dual variables π which are bounded thanks to the constraints $N^\top \pi \leq \mathbf{1}$ and the uncertain parameters ω which are binary. We can then use the McCormick envelope to obtain a linear reformulation of $(SP\text{-}P\text{-}Dual(\hat{x}))$. To do so, we create new continuous variables, G_{ij} for $(i,j) \in [m_y] \times [n_\omega]$, representing the bilinear product, $\pi_i \omega_j$. For $i \in [m_y]$, we let $u_i^\pi \in \mathbb{R}_+$ be an upper bound of the variable π_i which can be derived from the constraint $N^\top \pi \leq \mathbf{1}$. We then impose the following constraints on G_{ij} for $(i,j) \in [m_y] \times [n_\omega]$: $G_{ij} \leq \pi_i$, $G_{ij} \leq u_i^\pi \omega_j$, $G_{ij} \geq \pi_i - u_i^\pi(1 - \omega_j)$, $G_{ij} \geq 0$. We remark that, depending on the sign of the objective coefficients, lower or upper bound constraints may suffice to ensure a valid reformulation. Furthermore, since each uncertain parameter typically impacts only a few constraints of the second-stage problem, most G_{ij} variables have a zero coefficient in the objective function and can equivalently be omitted in the model.

Finally, an affine transformation as required for $(SP\text{-}P\text{-}Dual(\hat{x}))$ always exists and can be obtained by using the Minkowski-Weyl formulation of $\Xi \subseteq \mathbb{R}^{n_\xi}$ (with Ω the standard simplex) since we assume that the uncertainty set Ξ is a polytope (Assumption 1 (b)). However, such a transformation comes at the expense of introducing an exponential number of dual variables ω in $(SP\text{-}P\text{-}Dual(\hat{x}))$, which is not desirable given the crucial impact of the parameter n_ω on our capacity to numerically solve $(SP\text{-}P\text{-}Dual(\hat{x}))$ after linearization.

Assumption 4. *n_ω is a polynomial function of n_ξ.*

If Assumption 4 is verified, then the use of $(SP\text{-}P\text{-}Dual(\hat{x}))$ can be numerically advantageous. Several classical uncertainty sets satisfy Assumption 4, including the well-known budgeted uncertainty set (Bertsimas and Sim, 2003) with both integer and fractional budgets (Ayoub and Poss, 2016). On the other hand, when this assumption is not satisfied, the practical interest of $(SP\text{-}P\text{-}Dual(\hat{x}))$ is limited since it contains an exponential number of variables and constraints.

2.2.2 Subproblem reformulation based on KKT optimality conditions

The inner minimization problem in $(SP\text{-}P\text{-}ArtMaxMin(\hat{x}))$ is a feasible linear program. It follows that the KKT conditions are necessary and sufficient optimality conditions that characterize optimal (α, y) solutions. Using KKT optimality conditions on the inner minimization problem of $(SP\text{-}P\text{-}ArtMaxMin(\hat{x}))$, we obtain:

$$\max_{\xi\in\Xi} \max_{\substack{\alpha\in\mathbb{R}^{n_\alpha}_+, y\in\mathbb{R}^{n_y}_+ \\ \pi\in\Pi_N}} \mathbf{1}^\top\alpha \qquad (SP\text{-}P\text{-}KKT(\hat{x}))$$

$$\text{s.t.} \quad Wy + N\alpha \geq h(\xi) - T(\xi)\hat{x} \tag{18}$$

$$(Wy + N\alpha - h(\xi) + T(\xi)\hat{x}) \circ \pi = \mathbf{0} \tag{19}$$

$$\left(-W^\top\pi\right) \circ y = \mathbf{0} \tag{20}$$

$$\left(\mathbf{1} - N^\top\pi\right) \circ \alpha = \mathbf{0}, \tag{21}$$

where constraints (18), along with $\alpha \in \mathbb{R}^{n_\alpha}_+, y \in \mathbb{R}^{n_y}_+$, are primal feasibility, constraints $\pi \in \Pi_N$ are dual feasibility and constraints (19)-(21) are complementary slackness constraints. To obtain a mixed integer problem from $(SP\text{-}P\text{-}KKT(\hat{x}))$, one can linearize each complementary slackness constraint. Each of these constraints can be viewed as a disjunction in which at least one of the elements involved in the multiplication should be equal to zero. They can, therefore, be linearized using the *big-M* method (Zeng and Zhao, 2013). For example, to linearize the first set of complementary slackness constraints, we create new binary variables, $\sigma \in \{0,1\}^{m_y}$, use the corresponding upper bounds $u^p \in \mathbb{R}^{m_y}_+$ and $u^d \in \mathbb{R}^{m_y}_+$ for the primal and dual terms respectively, and replace the constraint $(Wy + N\alpha - h(\xi) + T(\xi)\hat{x}) \circ \pi = \mathbf{0}$ with the following equivalent system:

$$\begin{aligned} Wy + N\alpha - h(\xi) + T(\xi)\hat{x} &\leq u^p \circ \sigma \\ \pi &\leq u^d \circ (\mathbf{1} - \sigma). \end{aligned}$$

We remark that unlike $(SP\text{-}P\text{-}Dual(\hat{x}))$, $(SP\text{-}P\text{-}KKT(\hat{x}))$ does not require an assumption on the structure of the uncertainty set for the linearization of disjunctive complementary slackness constraints.

We provide in Appendix C an analysis of different subproblem reformulations and a numerical comparison on the two applications presented in Section 3.

3 Numerical results

In this section, we numerically test the algorithms presented in Section 2.1 and Appendix D on two applications: facility location-transportation (FLT) and network design (ND), which have both been

studied in the literature and possess different characteristics. Notably, the FLT problem (Zeng and Zhao, 2013; Gabrel et al., 2014a; Cheng et al., 2021) is naturally formulated with an optimality problem in the second stage, whereas the second stage of the ND problem (Ayoub and Poss, 2016; Matthews et al., 2019) is a feasibility problem. Additionally, the FLT problem contains integer first-stage variables, whereas the ND problem only contains continuous variables. Further, the ND problem has no constraints in the first stage and many constraints in the second stage, whereas the FLT problem has a fairly even distribution of constraints between the first and the second stages.

In both applications, we consider different uncertain parameters affecting the technology matrix and the right-hand-side vector independently. To model these parameters, we use budgeted uncertainty sets (Bertsimas and Sim, 2004) with different budget parameters. For a given budget $b \in \mathbb{R}_+$ and a set $S \subset \mathbb{N}$, we write these budgeted uncertainty sets as $B_S^b := \{\xi \in [0,1]^{|S|} | \sum_{s\in S} \xi_s \leq b\}$. Using these sets provides easily modifiable parameters that affect the characteristics and difficulty of both problems. In particular, if the budget parameter of the uncertainty set impacting the technology matrix is zero, then Assumption 3 is satisfied (the function $T(\xi)$ is constant). Additionally, uncertainty sets with fractional budgets have more extreme points than those with integer budgets, which often leads to an increased number of iterations required for each algorithm to converge (Ayoub and Poss, 2016).

In the remainder of this section, we first introduce the FLT and ND problems and then present a comprehensive analysis of the numerical behavior of different algorithms. We focus our study on comparing the performance of the proposed algorithm to that of CG and C&CG algorithms as well as assessing the contribution of each algorithmic component to the numerical performance.

3.1 Problem description

3.1.1 Facility location-transportation (FLT) problem

In the facility location-transportation problem, a subset of locations among a set $I \subset \mathbb{N}$ of potential locations should be opened with the aim of serving the uncertain demand of a set $J \subset \mathbb{N}$ of customers. The facilities can be opened at a fixed cost of s_i for $i \in I$. Each open facility $i \in I$ can have up to c_i units of capacity, and installing a unit of capacity costs a_i. The uncertain customer demand is modeled as $\bar{d}_j + \hat{d}_j \delta_j$, with $\bar{d}_j, \hat{d}_j \geq 0$, where uncertain parameters δ_j for $j \in J$ are governed by the uncertainty set B_J^{Δ}. We further consider that the installed capacity of an open facility $i \in I$ can be reduced by an uncertain factor γ_i, with γ_i for $i \in I$ governed by the uncertainty set B_I^{Γ}. The available capacity at each facility $i \in I$ can then be used to satisfy the demand of a customer $j \in J$ at a unit cost of $f_{ij} \geq 0$. We allow for unsatisfied demand with a cost of $\hat{f} = \max_{(i,j)\in I\times J} f_{ij}$ per unit, which ensures that the relatively complete recourse property is satisfied. We remark that previous studies (Zeng and Zhao, 2013; Gabrel et al., 2014a) that considered the case of demand uncertainty only, enforce this property by imposing that the installed capacity should be sufficient to satisfy the worst-case demand. In Gabrel et al. (2014a), the installed capacity is assumed to be sufficient to satisfy the worst-case demand calculated with a box uncertainty set (which is more strict than a budgeted uncertainty set). This assumption is then exploited to calculate smaller big-M coefficients for their linearized subproblems. We additionally remark that (Zeng and Zhao, 2013) use the big-M coefficients proposed by Gabrel et al. (2014a) in their numerical study.

We assume that facility opening and capacity installation decisions should be made before the realization of uncertainty, while the demand satisfaction decisions should be made at the second stage once the uncertain demand and remaining capacities are known. Let, to this end, x_i and z_i, for $i \in I$ denote the first-stage decisions where $x_i \in \{0,1\}$ denotes whether facility $i \in I$ is opened and z_i denotes the capacity

installed at facility $i \in I$, respectively. Let further, y_{ij} and h_j be second-stage decisions representing the quantity of product transported from facility $i \in I$ to client $j \in J$ and the quantity of unsatisfied demand of customer $j \in J$, respectively. Then, the FLT problem can be formulated as:

$$\min_{\substack{x\in\{0,1\}^{|I|},\, z\in\mathbb{R}_+^{|I|} \\ z_i \le c_i x_i \ \forall i\in I}} \sum_{i\in I} s_i x_i + a_i z_i + \max_{\substack{\gamma\in B_I^\Gamma \\ \delta\in B_J^\Delta}} \min_{\substack{y\in\mathbb{R}_+^{|I|\times|J|} \\ h\in\mathbb{R}_+^{|J|}}} \sum_{(i,j)\in I\times J} f_{ij} y_{ij} + \sum_{j\in J} \hat{f} h_j \tag{22}$$

$$\text{s.t.} \quad \sum_{j\in J} y_{ij} \le z_i(1-\gamma_i) \qquad \forall i \in I \tag{23}$$

$$\sum_{i\in I} y_{ij} + h_j \ge \bar{d}_j + \hat{d}_j \delta_j \qquad \forall j \in J \tag{24}$$

In the following, we use this natural formulation of the problem and do not integrate the objective function in an epigraph constraint as was done for formulation (P). We, additionally, remark that, since the relatively complete recourse assumption is satisfied by our formulation, we do not need feasibility cuts for this application. Hence, for the numerical results presented in Section 3.2, we implement simplified versions of Algorithms 8, 9, and 10 (see Appendix D) for this problem, which provide primal and dual bounds on the optimal value.

For our numerical experiments, we randomly generate 80 instances with n facilities and n customers as described in (Zeng and Zhao, 2013; Gabrel et al., 2014a). We vary n in $\{20, 30\}$ to observe the impact of instance size on the algorithms, and denote these instances as FLT-20 and FLT-30, respectively. We vary Δ and Γ in $\{\lfloor \frac{n}{3} \rfloor, \lfloor \frac{n}{3} \rfloor + 0.5\}$ and $\{0, \lfloor \frac{n}{5} \rfloor, \lfloor \frac{n}{5} \rfloor + 0.5\}$, respectively, to obtain a total of 480 tests per instance size (n). These values were selected as they correspond to the instances with the longest solution times for each algorithm in our preliminary tests. Further, different values for the budget parameters allow us to test the effect of integer versus fractional budgets, as well as the impact of Assumption 3.

3.1.2 Network design (ND) problem

In the network design problem, given a directed graph with $V \subset \mathbb{N}$ the set of vertices and $A \subseteq V \times V$ the set of arcs, the capacity of each arc should be determined so that a set of commodities $K \subset \mathbb{N}$, each with an uncertain demand, can be transported from their source nodes, $s_k \in V$, to their destination nodes, $t_k \in V$. Installing a unit of capacity costs c_a for $a \in A$ and the uncertain commodity demands are modeled as $\bar{d}_k + \hat{d}_k \delta_k$, where $\bar{d}_k, \hat{d}_j \ge 0$, and the uncertain parameters δ_k are governed by the uncertainty set B_K^Δ. We assume that the installed capacities can be reduced by an uncertain factor γ_a governed by the uncertainty set B_A^Γ.

We assume that the capacities must be installed before the realization of uncertainty, while the flow decisions are made once the uncertain demand of each commodity, as well as remaining arc capacities, are known. Let x_a be a first-stage variable denoting the installed capacity of arc $a \in A$ and y_a^k be a second-stage variable representing the flow on arc $a \in A$ to satisfy the demand of commodity $k \in K$. We further denote by σ_v^- and σ_v^+ the sets of incoming and outgoing arcs of a vertex $v \in V$, respectively. The ND problem can then be formulated as:

$$\min_{x\in\mathbb{R}_+^{|A|}} \sum_{a\in A} c_a x_a + \max_{\substack{\gamma\in B_A^\Gamma \\ \delta\in B_K^\Delta}} \min_{y\in\mathbb{R}_+^{|K|\times|A|}} 0 \tag{25}$$

$$\text{s.t.} \quad \sum_{a \in \sigma_v^-} y_a^k - \sum_{a \in \sigma_v^+} y_a^k = \begin{cases} -\bar{d}_k - \hat{d}_k \delta_k & \text{if } v = s_k \\ +\bar{d}_k + \hat{d}_k \delta_k & \text{if } v = t_k \\ 0 & \text{otherwise} \end{cases} \quad \forall (v,k) \in V \times K \tag{26}$$

$$\sum_{k \in K} y_a^k \leq x_a (1 - \gamma_a) \quad \forall a \in A \tag{27}$$

For this problem, we perform a similar numerical study to what was presented in (Ayoub and Poss, 2016). We, however, remark that the results presented in this paper suffer from a bug that has been corrected since (Poss, 2023). We study four instances from SNDlib (Orlowski et al., 2010) that have directed links and demands: giul-39, janos-us, janos-us-ca, and sun. For each instance, the unit capacity cost of each arc (*i.e.*, $c_a \ \forall a \in A$) is computed by dividing the cost of the arc's first module (specified in the instance) by its capacity. We consider the $|K| \in \{10, 20, 30, 40, 50\}$ commodities with the largest nominal demands, and set the deviation $\hat{d}$ to $0.4\bar{d}$. We also vary Δ and Γ in $\{1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5\}$ and $\{0, 1, 1.5\}$, respectively. We thus obtain the same number of instances for the ND problem as for each instance size (n) of the FLT problem. Our choice of Γ values is guided by two observations: $\Gamma = 0$ corresponds to the instances tested in (Ayoub and Poss, 2016), whereas for $\Gamma \geq 2$ most of the instances become infeasible. Additionally, similar to the FLT problem, considering integer and fractional budgets allows us to evaluate the impact of this parameter on the performance of different algorithms.

3.2 Numerical results

Our numerical study is divided into three parts. In the first part, we compare the proposed SICG algorithm (Algorithm 1) to the classical CG and C&CG algorithms. We also test different variants of the SICG algorithm that omit certain phases to better understand their impact. In the second part, we study the quality of primal and dual bounds provided by each method on FLT instances that were not solved to optimality by any method. Finally, in Appendix B.3, we describe and compare the performance of the SICG algorithm (Algorithm 1) with its variants that incorporate the mountain-climbing algorithm.

All algorithms are implemented in Julia-1.9.1 using the mathematical optimization package JuMP (URL: `https://github.com/orgs/jump-dev/`). Experiments are run on a single thread of a Cascade Lake Intel Xeon Skylake Gold 6240 @ 2.6 GHz CPU machine with a 3-hour time limit. The source code of our implementation as well as all generated instances are available on a git repository (Flambard et al., 2025). For each application, the subproblem reformulation used is the same for all methods and is described in Appendix C. We use Gurobi (URL: `https://www.gurobi.com`) to solve each model with relative and absolute gap tolerance parameters set to 0 and 10^{-4}, respectively. Details on how to handle numerical instabilities are provided in Appendix C.4.

The results for both applications are provided in Figures 1-3, where in each row of each figure we present the results obtained from instances sharing a parameter value organized into two performance plots and a detailed results table. The first of these plots reports the percentage of instances solved as a function of time, while the second is the performance profile (both plots consider all instances in each category). The table reports the detailed results as an average over the instances solved to optimality by all methods presented in the table (the number of such instances is provided in each table's legend). In each table, Time refers to the solution time in seconds, whereas %SP and #SP refer to the percentage of time spent solving the bilinear separation problems and the number of times they are solved, respectively.

Finally, #Ctr and $|\tilde{\Xi}|$ refer to the number of constraints and uncertainty realizations added to the master problem. We remark that the value of $|\tilde{\Xi}|$ is not reported for CG since, unlike C&CG and SICG algorithms, this algorithm does not generate systems of constraints or semi-infinite constraints associated with an uncertainty realization.

3.2.1 Comparing the SICG variants to CG and C&CG algorithms

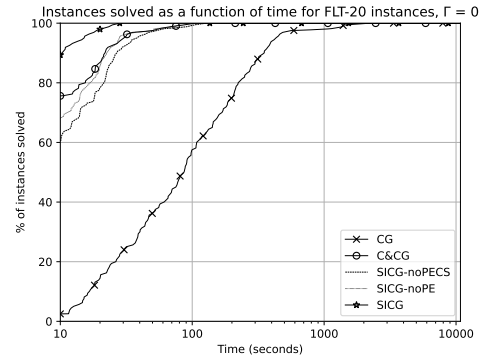
In this subsection, we first present a comparison of the classical CG and C&CG algorithms to the SICG algorithm (Algorithm 1). To assess the contribution of each algorithmic element, we additionally report the performance of two variants of the SICG algorithm.

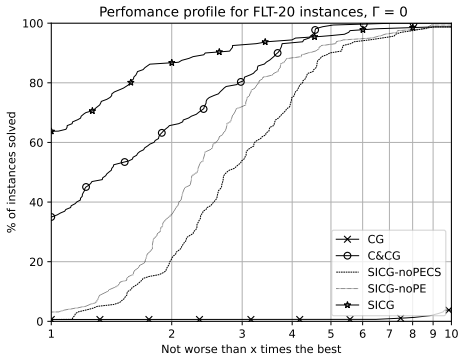
The first variant called SICG-noPE concerns the pool expansion phase performed at Line 19 of Algorithm 2. Indeed, in this phase, we potentially augment the set $\tilde{\Xi}$ with new realizations ($\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \tilde{\Xi}_{new}$) with the effect of generating new semi-infinite constraints $\pi^\top (h(\xi) - T(\xi)x) \leq 0 \quad \forall \xi \in \tilde{\Xi}_{new}, \forall \pi \in \Pi$. The SICG-noPE variant, does not perform this phase, and, instead, generates the standalone constraint $\pi^{*\top} (h(\xi^*) - T(\xi^*)x) \leq 0$ whenever the cut selection phase, performed at Line 10 of Algorithm 2, yields an uncertainty realization $\xi^* \notin \tilde{\Xi}$. This also implies that in subsequent iterations of Algorithm 2 the considered ξ^* will not be in $\tilde{\Xi}$ and therefore will not be used to generate new violated constraints. A detailed description of this variant is provided in Algorithm 5 in Appendix B.2.

The second variant is called SICG-noPECS and concerns the cut selection and pool expansion phases, performed, respectively, at Lines 10 and 19 of Algorithm 2. This variant foregoes both of these phases and is equivalent to an algorithm in which SICG master problems, $MP\text{-}SICG(\tilde{\Xi})$, are solved with classical cut generation. Its detailed description can be obtained from Algorithm 2 by replacing Line 10 of this algorithm by $\xi^* \leftarrow \hat{\xi}$. However, for the sake of completeness, we provide a dedicated description of this variant is provided in Algorithm 6 in Appendix B.2.

The results obtained by these five algorithms on both applications are reported in Figures 1-3. In particular, Figures 1 and 2 present the results obtained for the facility location-transportation (FLT) problem with 20 and 30 customers, respectively, while Figure 3 presents the results for the network design (ND) problem. In each figure, the rows present results obtained with different values of Γ. Performance plots with different values of Δ are provided in the Appendix E, along with tables of detailed results excluding the CG algorithm.

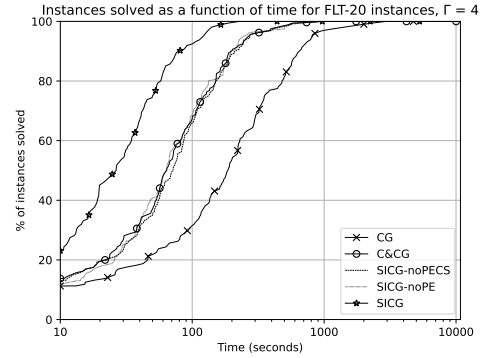
According to these results, the C&CG algorithm performs significantly better than CG, on both applications, as previously observed in the literature (Zeng and Zhao, 2013; Simchi-Levi et al., 2019). Indeed, the CG algorithm needs to solve the bilinear subproblem many times, which significantly hinders its performance. For instance, the CG algorithm was not able to solve some of the FLT-20 instances, whereas all other algorithms solved all instances in this category. On the other hand, we can observe that as the value of Γ increases, the discrepancy between the performance profiles of CG and C&CG algorithms becomes less significant. This evolution is also observed in the tables as the average ratio of the CG solution time to the C&CG solution time on FLT-30 instances goes from 26.0 when $\Gamma = 0$ to 1.1 when $\Gamma = 6.5$.

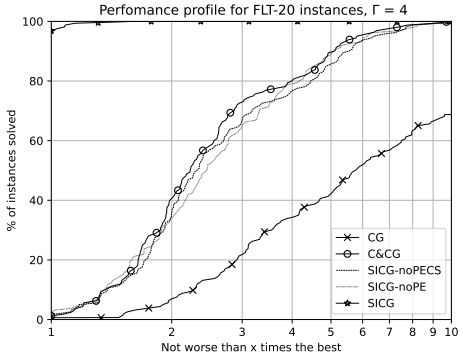




Detailed results for FLT-20 instances, $\Gamma = 0$: 160/160 instances

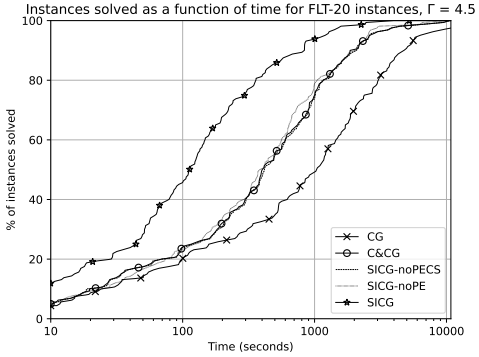
Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
CG	160.1	81.5	157.1	156.1	-
C&CG	9.3	75.9	6.1	204.8	5.1
SICG-noPECS	13.3	52.3	6.1	130.6	5.1
SICG-noPE	10.7	57.0	5.7	88.7	4.7
SICG	4.9	36.4	2.2	228.9	21.8

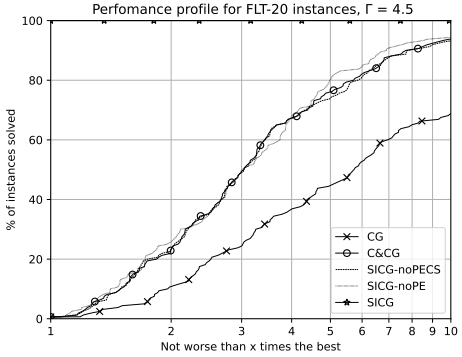




Detailed results for FLT-20 instances, $\Gamma = 4$: 160/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
CG	295.1	98.2	51.5	50.5	-
C&CG	97.5	94.1	17.2	647.0	16.2
SICG-noPECS	102.0	89.9	17.2	238.8	16.2
SICG-noPE	98.0	95.0	14.7	115.9	13.7
SICG	37.1	83.3	4.7	358.2	120.6

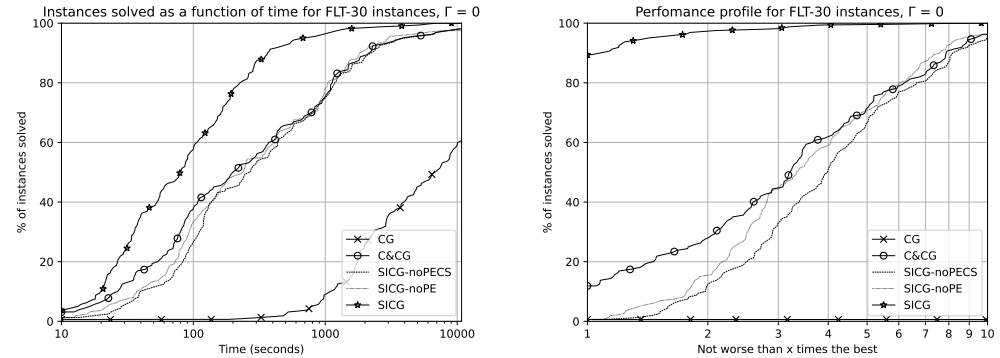




Detailed results for FLT-20 instances, $\Gamma = 4.5$: 156/160 instances

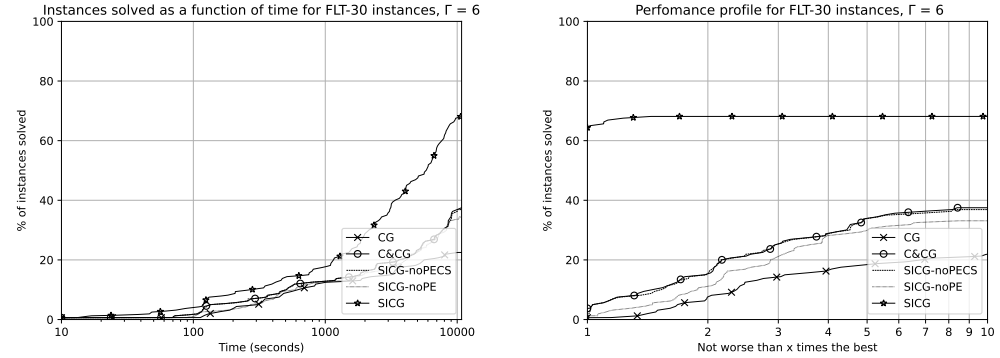
Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
CG	1618.3	99.3	47.7	46.7	-
C&CG	681.3	97.8	19.7	747.2	18.7
SICG-noPECS	688.1	97.7	19.7	210.8	18.7
SICG-noPE	631.5	98.7	15.8	90.4	14.8
SICG	229.6	92.5	4.7	411.4	175.0

Figure 1: Comparison of the CG, C&CG, SICG-noPECS, SICG-noPE and SICG algorithms on FLT-20 instances.



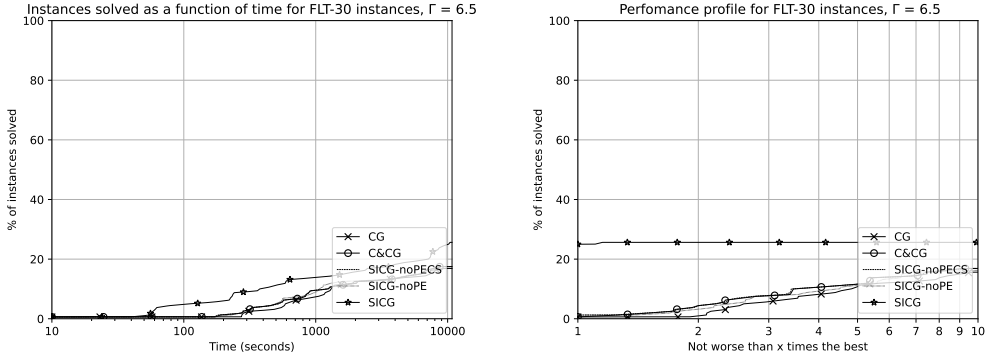
Detailed results for FLT-30 instances, $\Gamma = 0$: 97/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
CG	3642.0	92.5	392.1	391.1	-
C&CG	139.6	97.4	7.8	405.2	6.8
SICG-noPECS	173.7	74.0	7.8	288.5	6.8
SICG-noPE	155.2	80.0	7.3	194.7	6.3
SICG	59.8	53.8	2.4	888.9	54.0



Detailed results for FLT-30 instances, $\Gamma = 6$: 35/160 instances

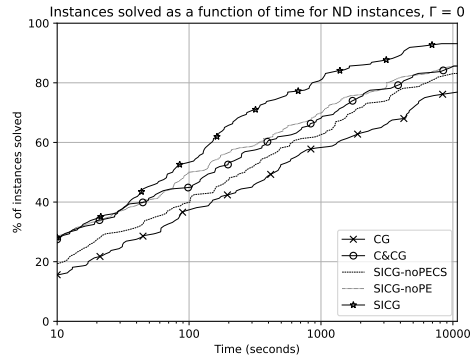
Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
CG	2152.0	99.9	23.0	22.0	-
C&CG	1359.3	99.0	16.5	929.1	15.5
SICG-noPECS	1368.0	99.0	16.5	115.4	15.5
SICG-noPE	1635.1	99.7	14.6	50.2	13.6
SICG	834.7	98.7	5.9	107.6	54.4

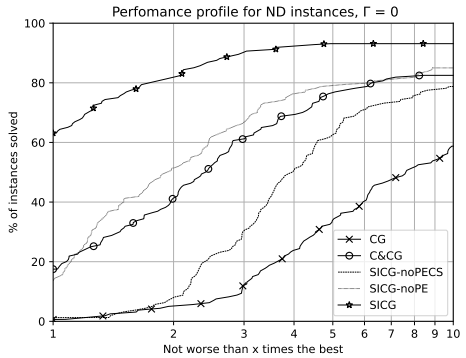


Detailed results for FLT-30 instances, $\Gamma = 6.5$: 26/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
CG	2115.6	99.9	25.6	24.6	-
C&CG	1867.6	99.3	20.8	1188.5	19.8
SICG-noPECS	1864.1	99.5	20.8	107.9	19.8
SICG-noPE	1803.8	99.8	15.2	45.0	14.2
SICG	577.7	98.7	4.9	142.9	81.5

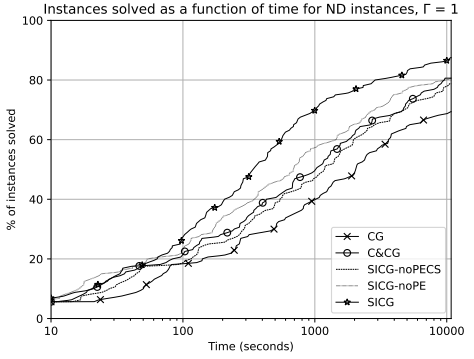
Figure 2: Comparison of the CG, C&CG, SICG-noPECS, SICG-noPE and SICG algorithms on FLT-30 instances.

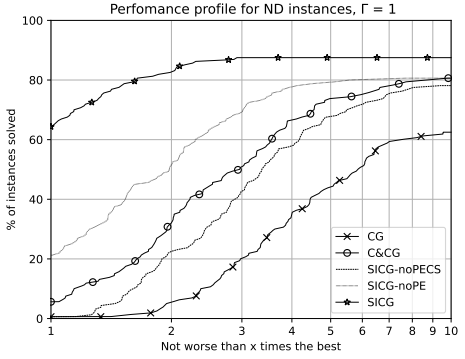




Detailed results for ND instances, $\Gamma = 0$: 121/160 instances

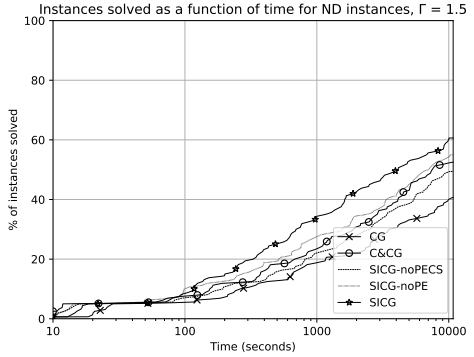
Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
CG	1085.1	93.2	102.7	101.7	-
C&CG	361.4	74.8	20.7	21514.8	19.7
SICG-noPECS	497.9	54.4	21.2	469.8	20.2
SICG-noPE	282.9	60.6	10.9	157.6	9.9
SICG	115.1	30.7	3.1	650.8	75.0

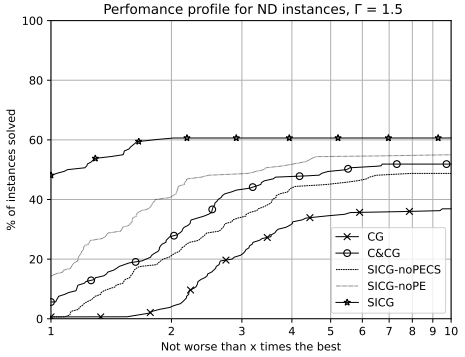




Detailed results for ND instances, $\Gamma = 1$: 105/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
CG	1587.8	98.2	128.6	127.6	-
C&CG	914.3	84.4	48.1	45365.6	47.1
SICG-noPECS	1005.7	72.8	48.3	791.9	47.3
SICG-noPE	539.4	77.8	29.2	348.9	28.2
SICG	283.2	37.6	8.4	1962.4	312.1





Detailed results for ND instances, $\Gamma = 1.5$: 54/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
CG	2317.7	99.2	172.5	171.5	-
C&CG	1454.2	93.6	76.9	55200.0	75.9
SICG-noPECS	1562.4	87.6	77.4	830.2	76.4
SICG-noPE	1069.0	89.4	50.4	386.4	49.4
SICG	632.7	59.5	22.2	2468.4	596.3

Figure 3: Comparison of the CG, C&CG, SICG-noPECS, SICG-noPE and SICG algorithms on ND instances.

Our results further reveal that algorithms C&CG and SICG-noPECS behave very similarly. This result may be expected since their master problems, ($MP\text{-}C\&CG(\tilde{\Xi})$) and ($MP\text{-}SICG(\tilde{\Xi})$), respectively, are equivalent for given $\tilde{\Xi} \subseteq \Xi$. The small differences between these two algorithms can be explained by the way their master problems are solved: while the C&CG algorithm reformulates all semi-infinite constraints through duality (thus obtaining one recourse block for each uncertainty realization), the SICG-noPECS variant ensures their feasibility using a cutting-plane approach. In our experiments, this second approach required significantly fewer constraints to be added to the master problem, reducing the average number of added constraints by 98% for the ND application when $\Gamma = 1.5$. Despite this difference, the C&CG algorithm spent less time solving the master problem and obtained slightly better solution times overall. The structure of the second-stage problem in both applications can explain this phenomenon. Both problems correspond to transportation/minimum-cost flow problems that are typically handled well by commercial solvers.

Adding the cut selection phase to SICG-noPECS, leading to the SICG-noPE variant, proved its efficiency by reducing the number of constraints and uncertainty realizations required for convergence. For the ND problem, SICG-noPE converged, on average, with 2-3 times fewer constraints and 1.5-2 times fewer realizations compared to the SICG-noPECS variant. However, the solution time of this variant is comparable to that of the C&CG algorithm for the FLT problem. In contrast, for the ND problem, the cut selection phase suffices to reduce the average solution time among the instances solved to optimality by both methods. Indeed, the SICG-noPE algorithm converges approximately 1.5 times faster, on average, than the C&CG algorithm. Moreover, for both problems when the uncertainty budget Γ is equal to zero (Assumption 3 is satisfied) the SICG-noPE algorithm adds dominating constraints in its cut generation phase (Lemma 2). However, this property does not positively impact the solution time in our experiments.

The bilinear subproblem is clearly the bottleneck of all algorithms, as it constitutes a significant percentage of their total execution time. It follows that decreasing the number of times it is solved can have a considerable impact on the overall performance, which is reflected by the superior performance of the complete SICG algorithm. Indeed, the pool expansion phase augments $\tilde{\Xi}$ without resorting to the bilinear separation problem, dividing the average number of times it must be solved by 3.7, 3.7, and 5.0 for the FLT-20, FLT-30, and ND instances, respectively (compared to C&CG based on Figure 5). This reduction has a significant effect on the total solution time. Notably SICG is, on average, 3.0, 2.9, and 3.2 times faster than C&CG on those instances, respectively (based on Figure 5).

3.2.2 Comparing the quality of primal and dual bounds

In this subsection, we present a comparison of the quality of the primal and dual bounds obtained by each algorithm for the FLT-30 instances that were not solved to optimality. We recall that, since the FLT problem satisfies the relatively complete recourse property, all algorithms stop with a feasible solution when the time limit is reached, and provide primal and dual bounds.

We report, in Table 1, five different optimality gaps that inform about different characteristics of the considered algorithms. The column $\frac{\text{P}-\text{D}}{\text{P}}$ provides the optimality gap returned by each method, *i.e.*, using information available from a direct implementation of Algorithms 8, 9, and 10 (see Appendix D). The value P (resp. D) denotes the value of the incumbent solution (resp. value of the last master problem solved to optimality). We remark that the CG algorithm produced larger gaps on average than the C&CG algorithm. Further, the latter obtained the same gaps as SICG-noPECS, as expected, since their master problems are equivalent and their solution times are very similar. However, SICG-noPE, which has larger

Method	$\frac{\text{P}-\text{D}}{\text{P}}$	$\frac{\text{P}-\text{maxD}}{\text{P}}$	$\frac{\text{minP}-\text{D}}{\text{minP}}$	$\frac{\text{P}^+-\text{D}}{\text{P}^+}$	$\frac{\text{postP}-\text{D}}{\text{postP}}$	%SP	#SP	|#sp=1|	#post-time-limit
CG	23.7	8.7	20.4	22.3	21.7	100.0	16.5	0	141
C&CG	10.9	7.2	7.7	10.3	8.7	100.0	11.5	0	135
SICG-noPECS	10.9	7.2	7.7	10.2	8.8	99.8	11.4	0	132
SICG-noPE	10.6	7.3	7.3	9.1	8.2	100.0	7.1	0	124
SICG	44.1	44.0	3.8	5.9	5.7	99.7	2.3	84	76

Table 1: Optimality gaps obtained on FLT-30 instances solved by none of the algorithms (160/480 instances).

solution times than the C&CG algorithm on instances solved to optimality by both methods, obtained better gaps on average. On the other hand, the SICG algorithm obtained the worst gaps by a factor of 4.0 compared to C&CG.

The next two columns assess the quality of the upper and lower bounds provided by each method by comparing them to the best lower and upper bounds among all methods, respectively. The value maxD (resp. minP) denotes the maximum dual (resp. minimum primal) bound obtained among all methods. Columns $\frac{\text{P}-\text{maxD}}{\text{P}}$ and $\frac{\text{minP}-\text{D}}{\text{minP}}$ show that the SICG algorithm actually returned good-quality dual bounds. Indeed, the difference between the average gap returned by the SICG algorithm and that calculated using the maxD value is nearly negligible. On the other hand, the average gap calculated using the minP value is an order of magnitude smaller than that returned by the SICG algorithm. It is clear that the latter gap is high due to the poor quality of the primal bounds obtained by this method, which are only updated each time the bilinear subproblem is solved to optimality. We can further observe that none of the other algorithms consistently produced the best primal or dual bounds.

The value P reported by each algorithm is updated only when the bilinear subproblem, $(SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$, is solved to optimality. When the time limit is reached during the solution of this subproblem with first-stage solution $\hat{x}$, an overestimation of the actual worst-case cost of $\hat{x}$, denoted θ^+ in the following, is readily available through the best dual bound on the value of the subproblem provided by the MIP solver. In Table 1, P^+ denotes the value $\min\{\text{P}, c^\top \hat{x} + \theta^+\}$ if the algorithm has stopped during the solution of a bilinear subproblem. Otherwise, it is equal to P. As such, the value P^+ corresponds to the best primal bound available using each method without any additional computational effort. Finally, we also estimate the worst-case cost of solution $\hat{x}$ using a post-time-limit procedure attempting to solve $(SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$ with an additional 3-hour time limit. In Table 1, the value PostP corresponds to the best primal bound available using additionally the information from the post-time-limit procedure.

We finally examine the gaps that make use of the primal bounds P^+ and postP to better understand the quality of the last first-stage solutions obtained by each algorithm. When these primal bounds are used, the SICG algorithm produces better relative optimality gaps than the other algorithms on average. We conclude that the SICG algorithm produces good-quality dual bounds and primal solutions. However, it suffers from infrequent primal bound updates in its reported optimality gaps. In order to circumvent this issue, all algorithms can be modified to use the primal bound P^+ to potentially update their primal bound whenever the time limit is reached while a bilinear subproblem is being solved. We further emphasize how costly a single solution of the bilinear subproblem can be. Indeed, with a 3-hour time limit, only 608 of the 800 post-time-limit evaluation subproblems were solved to optimality.

Table 1 additionally reports the average percentage of time spent solving the bilinear subproblems (%SP) and the average number of bilinear subproblems solved to optimality within the time limit by each algorithm (#SP), the number of instances where only one bilinear subproblem was solved to optimality |#SP=1|, and the number of instances where the post-time-limit bilinear subproblem was solved to

optimality (#post-time-limit).

4 Conclusion

In this paper, we present a semi-infinite constraint generation algorithm to solve adjustable robust optimization problems with continuous and fixed recourse. This algorithm relies on a reformulation of the problem as a deterministic equivalent problem in which there are an exponential number of semi-infinite constraints. The exponential nature of these constraints is handled through a decomposition algorithm in which semi-infinite constraints are added to a relaxation as needed. The semi-infinite constraints are then handled through a cut-generation algorithm that additionally allows for generating new ones as needed.

We develop two versions of the algorithm that can be used depending on whether the problem is formulated with a feasibility or an optimality problem at the second stage. We additionally study MILP reformulations for the bilinear subproblems that need to be solved in all presented algorithms, unifying the existing literature in terms of how subproblem infeasibility is handled and based on the structure of the uncertainty set. We present detailed numerical results on two different applications comparing the proposed algorithm to the classical constraint and constraint-and-column generation algorithms, as well as its different variants. Our results demonstrate the superior numerical performance of the proposed algorithm with respect to the state of the art both in terms of solution time and quality of primal and dual bounds. We additionally identify the most critical parts of the algorithm for better numerical performance.

We believe that the methodological and numerical results developed in this paper reveal further research questions. Although our algorithm outperforms the existing approaches in the literature it still cannot overcome the numerical difficulty of the bilinear separation problems that need to be repeatedly solved in all algorithms. This reveals the need for further research into the solution of such problems as well as alternative ways of certifying feasibility/optimality in the context of adjustable robust optimization. One possible avenue in this sense could be to use the realizations produced throughout the algorithm to dynamically reduce the uncertainty set. We additionally remark that our algorithm opens the door to an implementation where the cut generation procedure can be embedded within a branch-and-cut algorithm for problems where the first stage includes integer variables. It would then be interesting to numerically compare such an implementation to the recently proposed inexact C&CG algorithm on applications where the master programs pose a numerical challenge.

Acknowledgements

This work was supported by Agence Nationale de la Recherche, the French national research agency, under the grant ANR-22-CE48-0018. For the purpose of Open Access, a CC-BY public copyright license has been applied by the authors to the present document and will be applied to all subsequent versions up to the Author Accepted Manuscript arising from this submission, in accordance with the grant's open access conditions.

References

Ayoub, J. and Poss, M. (2016). Decomposition for adjustable robust linear optimization subject to uncertainty polytope. *Computational Management Science*, 13:219–239.

Ben-Tal, A., El Housni, O., and Goyal, V. (2020). A tractable approach for designing piecewise affine policies in two-stage adjustable robust optimization. *Mathematical Programming*, 182:57–102.

Ben-Tal, A., Goryashko, A., Guslitzer, E., and Nemirovski, A. (2004). Adjustable robust solutions of uncertain linear programs. *Mathematical programming*, 99(2):351–376.

Ben-Tal, A. and Nemirovski, A. (1998). Robust convex optimization. *Mathematics of operations research*, 23(4):769–805.

Bertsimas, D. and Bidkhori, H. (2015). On the performance of affine policies for two-stage adaptive optimization: a geometric perspective. *Mathematical Programming*, 153(2):577–594.

Bertsimas, D., Brown, D. B., and Caramanis, C. (2011). Theory and applications of robust optimization. *SIAM review*, 53(3):464–501.

Bertsimas, D. and Caramanis, C. (2010). Finite adaptability in multistage linear optimization. *IEEE Transactions on Automatic Control*, 55(12):2751–2766.

Bertsimas, D. and de Ruiter, F. J. (2016). Duality in two-stage adaptive linear optimization: Faster computation and stronger bounds. *INFORMS Journal on Computing*, 28(3):500–511.

Bertsimas, D. and Dunning, I. (2016). Multistage robust mixed-integer optimization with adaptive partitions. *Operations Research*, 64(4):980–998.

Bertsimas, D., Dunning, I., and Lubin, M. (2016). Reformulation versus cutting-planes for robust optimization: A computational study. *Computational Management Science*, 13:195–217.

Bertsimas, D. and Georghiou, A. (2015). Design of Near Optimal Decision Rules in Multistage Adaptive Mixed-Integer Optimization. *Operations Research*, 63(3):610–627.

Bertsimas, D. and Goyal, V. (2012). On the power and limitations of affine policies in two-stage adaptive optimization. *Mathematical programming*, 134(2):491–531.

Bertsimas, D., Iancu, D. A., and Parrilo, P. A. (2010). Optimality of Affine Policies in Multi-stage Robust Optimization. *Mathematics of Operations Research*, 35(2):363–394.

Bertsimas, D., Litvinov, E., Sun, X. A., Zhao, J., and Zheng, T. (2012). Adaptive robust optimization for the security constrained unit commitment problem. *IEEE transactions on power systems*, 28(1):52–63.

Bertsimas, D. and Shtern, S. (2018). A scalable algorithm for two-stage adaptive linear optimization. *arXiv preprint arXiv:1807.02812*.

Bertsimas, D. and Sim, M. (2003). Robust discrete optimization and network flows. *Mathematical Programming*, 98(1-3):49–71.

Bertsimas, D. and Sim, M. (2004). The price of robustness. *Operations research*, 52(1):35–53.

Chen, X. and Zhang, Y. (2009). Uncertain linear programs: Extended affinely adjustable robust counterparts. *Operations Research*, 57(6):1469–1482.

Cheng, C., Adulyasak, Y., and Rousseau, L.-M. (2021). Robust facility location under demand uncertainty and facility disruptions. *Omega*, 103:102429.

Dumouchelle, J., Julien, E., Kurtz, J., and Khalil, E. B. (2023). Neur2ro: Neural two-stage robust optimization. In *The Twelfth International Conference on Learning Representations*.

Fischetti, M. and Monaci, M. (2012). Cutting plane versus compact formulations for uncertain (integer) linear programs. *Mathematical Programming Computation*, 4:239–273.

Fischetti, M., Salvagnin, D., and Zanette, A. (2010). A note on the selection of benders' cuts. *Mathematical Programming*, 124:175–182.

Flambard, P., Arslan, A. N., and Detienne, B. (2025). A semi-infinite constraint generation algorithm for two-stage robust optimization problems. Available for download at https://github.com/INFORMSJoC/2025.1491.

Gabrel, V., Lacroix, M., Murat, C., and Remli, N. (2014a). Robust location transportation problems under uncertain demands. *Discrete Applied Mathematics*, 164:100–111.

Gabrel, V., Murat, C., and Thiele, A. (2014b). Recent advances in robust optimization: An overview. *European journal of operational research*, 235(3):471–483.

Georghiou, A., Tsoukalas, A., and Wiesemann, W. (2020). A primal–dual lifting scheme for two-stage robust optimization. *Operations Research*, 68(2):572–590.

Georghiou, A., Wiesemann, W., and Kuhn, D. (2015). Generalized decision rule approximations for stochastic programming via liftings. *Mathematical Programming*, 152(1-2):301–338.

Goerigk, M. and Kurtz, J. (2022). Data-driven prediction of relevant scenarios for robust optimization. *CoRR*, abs/2203.16642.

Gorissen, B. L., Yanıkoğlu, İ., and den Hertog, D. (2015). A practical guide to robust optimization. *Omega*, 53:124–137.

Guslitser, E. (2002). *Uncertainty-immunized solutions in linear programming*. PhD thesis, Citeseer.

Hadjiyiannis, M. J., Goulart, P. J., and Kuhn, D. (2011). An efficient method to estimate the suboptimality of affine controllers. *IEEE Transactions on Automatic Control*, 56(12):2841–2853.

Hosseini, M. and Turner, J. (2024). Deepest Cuts for Benders Decomposition. *Operations Research*, page opre.2021.0503.

Konno, H. (1976). A cutting plane algorithm for solving bilinear programs. *Mathematical Programming*, 11(1):14–27.

Kuhn, D., Wiesemann, W., and Georghiou, A. (2011). Primal and dual linear decision rules in stochastic and robust optimization. *Mathematical Programming*, 130:177–209.

Lorca, A. and Sun, X. A. (2017). The adaptive robust multi-period alternating current optimal power flow problem. *IEEE Transactions on Power Systems*, 33(2):1993–2003.

Matthews, L. R., Gounaris, C. E., and Kevrekidis, I. G. (2019). Designing networks with resiliency to edge failures using two-stage robust optimization. *European Journal of Operational Research*, 279(3):704–720.

Orlowski, S., Pióro, M., Tomaszewski, A., and Wessäly, R. (2010). SNDlib 1.0–Survivable Network Design Library. *Networks*, 55(3):276–286.

Poss, M. (2023). Private communication.

Postek, K. and den Hertog, D. (2016). Multistage adjustable robust mixed-integer optimization via iterative splitting of the uncertainty set. *INFORMS Journal on Computing*, 28(3):553–574.

See, C.-T. and Sim, M. (2010). Robust approximation to multiperiod inventory management. *Operations research*, 58(3):583–594.

Simchi-Levi, D., Wang, H., and Wei, Y. (2019). Constraint generation for two-stage robust network flow problems. *INFORMS Journal on Optimization*, 1(1):49–70.

Subramanyam, A., Gounaris, C. E., and Wiesemann, W. (2020). K-adaptability in two-stage mixed-integer robust optimization. *Mathematical Programming Computation*, 12(2):193–224.

Thiele, A., Terry, T., and Epelman, M. (2009). Robust linear optimization with recourse. *Rapport technique*, pages 4–37.

Tsang, M. Y., Shehadeh, K. S., and Curtis, F. E. (2023). An inexact column-and-constraint generation method to solve two-stage robust optimization problems. *Operations Research Letters*, 51(1):92–98.

Vieira, M. P., Martins, A. C. P., Soler, E. M., Balbo, A. R., and Nepomuceno, L. (2023). Two-stage robust market clearing procedure model for day-ahead energy and reserve auctions of wind–thermal systems. *Renewable Energy*, 218:119276.

Yanıkoğlu, İ., Gorissen, B. L., and den Hertog, D. (2019). A survey of adjustable robust optimization. *European Journal of Operational Research*, 277(3):799–813.

Zeng, B. and Wang, W. (2022). Two-stage robust optimization with decision dependent uncertainty.

Zeng, B. and Zhao, L. (2013). Solving two-stage robust optimization problems using a column-and-constraint generation method. *Operations Research Letters*, 41(5):457–461.

Zhen, J., Den Hertog, D., and Sim, M. (2018). Adjustable robust optimization via fourier–motzkin elimination. *Operations Research*, 66(4):1086–1100.

Appendices

A Proofs of theorems

Lemma 1. Algorithm 2 called with $(\tilde{\Xi}, \tilde{\Pi})$ as input either returns an optimal solution of problem $(MP\text{-}SICG(\tilde{\Xi}'))$ or concludes that problem $(MP\text{-}SICG(\tilde{\Xi}'))$ is infeasible, with $\tilde{\Xi} \subseteq \tilde{\Xi}'$.

Proof. We first remark that the sets $\tilde{\Xi}$ and $\tilde{\Pi}$ are potentially enlarged throughout Algorithm 2. As such, for notational clarity, we denote the initial sets by $\tilde{\Xi}$ and $\tilde{\Pi}$ and the sets obtained at convergence of Algorithm 2 by $\tilde{\Xi}' \supseteq \tilde{\Xi}$ and $\tilde{\Pi}' \supseteq \tilde{\Pi}$.

We then note that, for all $\tilde{\Pi}'$, $(MP\text{-}SICG(\tilde{\Xi}', \tilde{\Pi}'))$ is a relaxation of $(MP\text{-}SICG(\tilde{\Xi}'))$. Therefore, if $(MP\text{-}SICG(\tilde{\Xi}', \tilde{\Pi}'))$ is infeasible at Line 4 then $(MP\text{-}SICG(\tilde{\Xi}'))$ is also infeasible and the algorithm returns with this conclusion.

Further, when Algorithm 2 returns with a solution $\hat{x} \neq$ null, the test performed at Line 8 guarantees that, for all $\hat{\xi} \in \tilde{\Xi}'$, the constraint $\pi^\top(h(\hat{\xi}) - T(\hat{\xi})\hat{x}) \leq 0$ is satisfied for all $\pi \in \Pi_{\text{norm}}$. This implies that the solution $\hat{x}$ is feasible for $(MP\text{-}SICG(\tilde{\Xi}'))$ and is optimal for $(MP\text{-}SICG(\tilde{\Xi}', \tilde{\Pi}'))$. Since $(MP\text{-}SICG(\tilde{\Xi}', \tilde{\Pi}'))$ is a relaxation of $(MP\text{-}SICG(\tilde{\Xi}'))$ and $\hat{x}$ is an optimal solution of $(MP\text{-}SICG(\tilde{\Xi}', \tilde{\Pi}'))$ then $\hat{x}$ is also optimal for $(MP\text{-}SICG(\tilde{\Xi}'))$. This concludes the proof.

■

Proposition 1. Algorithm 1 either returns an optimal solution of problem (P) or concludes that the problem is infeasible.

Proof. We first remark that $(MP\text{-}SICG(\tilde{\Xi}))$ is a relaxation of (P). Therefore, if $(MP\text{-}SICG(\tilde{\Xi}))$ is infeasible at Line 4, then problem (P) is infeasible, and the algorithm returns with this conclusion.

Further, the test performed at Line 6 ensures that $\hat{x}$ is a feasible solution of $(P\text{-}SICG)$. Moreover, by Lemma 1, $\hat{x}$ is an optimal solution of $(MP\text{-}SICG(\tilde{\Xi}))$ for some $\tilde{\Xi} \subseteq \Xi$. Since $(MP\text{-}SICG(\tilde{\Xi}))$ is a relaxation of $(P\text{-}SICG)$ and has the same objective function, $\hat{x}$ is optimal for $(P\text{-}SICG)$ (and therefore for problem (P)).

■

Proposition 2 The number of iterations of Algorithm 1 is at most $|\text{ext}(\Xi)|+1$, and the cumulated number of iterations of the *while*-loop of Algorithm 2 is at most $|\text{ext}(\Xi)|\cdot|\text{dir}(\Pi)|+1$.

Proof. We first prove that Algorithm 1 terminates in at most $|\text{ext}(\Xi)|+1$ iterations. Indeed, at Line 7, $\xi^* \notin \tilde{\Xi}$ by definition of $(MP\text{-}SICG(\tilde{\Xi}))$ and Lemma 1. We, therefore, obtain $\tilde{\Xi} = \text{ext}(\Xi)$ after $|\text{ext}(\Xi)|$ iterations. A final iteration may be necessary to conclude that $\hat{x}$ is feasible for $(MP\text{-}SICG(\Xi))$ (and therefore for problem (P)).

We now prove the result for Algorithm 2. For given $\tilde{\Xi}$ and $\tilde{\Pi} := (\tilde{\Pi}^\xi)_{\xi \in \tilde{\Xi}}$, let $\tilde{\Psi}(\tilde{\Xi}, \tilde{\Pi}) := \bigcup_{\xi \in \tilde{\Xi}, \pi \in \tilde{\Pi}^\xi} \{(\xi, \pi)\}$. At each execution of the *while*-loop in Algorithm 2, two cases can occur based on whether a violated cut

is found or not. In the first case, Line 9 is executed at least once and at least one element (ξ^*, π^*) is added to $\tilde{\Psi}(\tilde{\Xi}, \tilde{\Pi})$. In the second case, Line 9 is not executed, and the *while*-loop terminates and returns either $\hat{x} = \text{null}$ or a relaxation solution $\hat{x}$ to Algorithm 1. In the first case, Algorithm 1 terminates with the conclusion that problem (P) is infeasible. In the second case, depending on whether the solution $\hat{x}$ is feasible or not, either one element is added to $\tilde{\Psi}(\tilde{\Xi}, \tilde{\Pi})$, and Algorithm 2 is called again or Algorithm 1 terminates with the conclusion that $\hat{x}$ is an optimal solution. It follows that between two consecutive executions of Line 3 of Algorithm 2, either Algorithm 1 terminates or $\tilde{\Psi}(\tilde{\Xi}, \tilde{\Pi})$ has been augmented by at least one new element since by definition $\pi^\top (h(\xi) - T(\xi)\hat{x}) \leq 0$ holds for all $(\xi, \pi) \in \tilde{\Psi}(\tilde{\Xi}, \tilde{\Pi})$. We conclude that the number of times the *while*-loop of Algorithm 2 is executed is at most $|\text{ext}(\Xi)| \cdot |\text{dir}(\Pi)| + 1$ (since $\text{dir}(\Pi) = \text{ext}(\Pi_{\text{norm}}) \backslash \{\mathbf{0}\}$).

■

Theorem 1. Algorithm 1 requires the solution of at most $|\text{ext}(\Xi)|+1$ bilinear separation subproblems $(SP\text{-}P\text{-}MaxMax)$ and $\mathcal{O}(|\text{dir}(\Pi)| \cdot |\text{ext}(\Xi)|^2)$ linear separation subproblems $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ and $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$.

Proof. Throughout the execution of Algorithm 1, a bilinear subproblem of form $(SP\text{-}P\text{-}MaxMax)$ is only solved at Line 5 which is executed at most $|\text{ext}(\Xi)|+1$ times by Proposition 2. On the other hand, linear separation problems $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ and $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$ are solved for each $\hat{\xi} \in \tilde{\Xi}$ each time the while loop of Algorithm 2 is executed. The number of such problems solved is, therefore, $\mathcal{O}(|\text{dir}(\Pi)| \cdot |\text{ext}(\Xi)|^2)$.

Proposition 3. Let $(\tilde{\Xi}', \tilde{\Pi}', \hat{x})$ be the output of Algorithm 2 with input $(\tilde{\Xi}, \tilde{\Pi})$, then $z_{\text{C\&CG}}(\tilde{\Xi}) \leq z_{\text{SICG}}(\tilde{\Xi}')$.

Proof. At convergence, Algorithm 2 ensures that $z_{\text{SICG}}(\tilde{\Xi}', \tilde{\Pi}') = z_{\text{SICG}}(\tilde{\Xi}') = z_{\text{C\&CG}}(\tilde{\Xi}')$ by Lemma 1. Further, $\tilde{\Xi}$ is possibly augmented during the algorithm implying that $\tilde{\Xi} \subseteq \tilde{\Xi}'$. We thus have that $z_{\text{C\&CG}}(\tilde{\Xi}) \leq z_{\text{C\&CG}}(\tilde{\Xi}') = z_{\text{SICG}}(\tilde{\Xi}')$.

■

Lemma 2 Let us assume that Assumption 3 holds. Then:

a) For all $\hat{\pi} \in \Pi$, the set of optimal solutions of $(SP\text{-}P(x, \Xi, \{\hat{\pi}\}))$ is independent of $x \in \mathcal{X}$.

b) For all $\hat{\pi} \in \Pi$, $x \in \mathcal{X}$ and $\xi \in \Xi$, we have that $\hat{\pi}^\top (h(\xi) - Tx) \leq \hat{\pi}^\top (h(\xi^*_{\hat{\pi}}) - Tx)$, where $\xi^*_{\hat{\pi}} \in \Xi$ is any optimal solution of $(SP\text{-}P(x, \Xi, \{\hat{\pi}\}))$.

c) For given $\hat{\pi} \in \text{dir}(\Pi)$, let $\xi^*_{\hat{\pi}}$ be any optimal of $(SP\text{-}P(x, \Xi, \{\hat{\pi}\}))$ for any $x \in \mathcal{X}$. Let, further, $\tilde{\Xi} := \{\xi^*_{\hat{\pi}} \mid \hat{\pi} \in \text{dir}(\Pi)\}$, $\tilde{\Pi}^{\xi^*_{\hat{\pi}}} := \{\hat{\pi}\}$ for $\hat{\pi} \in \text{dir}(\Pi)$, and $\tilde{\Pi} := (\tilde{\Pi}^{\xi})_{\xi \in \tilde{\Xi}}$. We then have that $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ is equivalent to problems $(P\text{-}SICG)$ and (P).

Proof. For given $\hat{\pi} \in \Pi$, $x \in \mathcal{X}$ and $\hat{\xi} \in \Xi$, we have:

$$\hat{\pi}^\top \left(h(\hat{\xi}) - Tx\right) \leq \max_{\xi \in \Xi} \hat{\pi}^\top (h(\xi) - Tx) = -\hat{\pi}^\top Tx + \max_{\xi \in \Xi} \hat{\pi}^\top h(\xi).$$

Let us remark that the maximization problem in the middle term is (SP-$P(x, \Xi, \{\hat{\pi}\})$). Then, the rightmost equality shows that the choice of an optimal ξ in (SP-$P(x, \Xi, \{\hat{\pi}\})$) does not depend on x, which proves claim a). Claim b) directly follows from the leftmost inequality, by selecting an arbitrary optimal solution $\xi^*_{\hat{\pi}}$ of (SP-$P(x, \Xi, \{\hat{\pi}\})$). Finally, let us consider an optimal solution $\hat{x}$ of (MP-$SICG(\tilde{\Xi}, \tilde{\Pi})$), with $\tilde{\Xi}$ and $\tilde{\Pi}$ defined as in claim c). By constraints (13), we have that $\pi^\top(h(\xi^*_\pi) - T\hat{x}) \leq 0$ for all $\pi \in \text{dir}(\Pi)$. Further, by optimality of ξ^*_π for (SP-$P(x, \Xi, \{\pi\})$) for any $x \in \mathcal{X}$ and the fact that the optimal solution is independent of x, we have that $\pi^\top(h(\xi) - T\hat{x}) \leq 0$ for all $\xi \in \Xi$ and $\pi \in \text{dir}(\Pi)$. Finally, by convexity of the left-hand side of this latter inequality in π, it is also satisfied for all $\pi \in \Pi$, which proves that $\hat{x}$ is feasible for (P-$SICG$). Since (MP-$SICG(\tilde{\Xi}, \tilde{\Pi})$) is a relaxation of ($P$-$SICG$) and has the same objective value as (P-$SICG$), $\hat{x}$ is an optimal solution of (P-$SICG$) and thus of problem (P).

■

Theorem 2. If Assumptions 2 and 3 are satisfied, then the number of iterations of Algorithm 1 is bounded by $\min(|\text{ext}(\Xi)|, |\text{dir}(\Pi)|) + 1$. Moreover, the cumulated number of iterations of the *while*-loop of Algorithm 2 is bounded by $|\text{dir}(\Pi)|+1$. In this case, Algorithm 1 requires the solution of at most $\min(|\text{ext}(\Xi)|, |\text{dir}(\Pi)|) + 1$ bilinear separation subproblems (SP-P-$MaxMax$), and $\mathcal{O}(|\text{ext}(\Xi)|\cdot|\text{dir}(\Pi)|)$ linear separation subproblems, (SP-$P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}})$) and ($SP$-$P(\hat{x}, \Xi, \{\pi^*\})$).

Proof. We first observe that, throughout Algorithms 1 and 2, when an element π^* is added to $\bigcup_{\xi \in \Xi} \tilde{\Pi}^\xi$, it is added to $\tilde{\Pi}^{\xi^*}$ for some ξ^* optimal for (SP-$P(x, \Xi, \{\pi^*\})$) with x an arbitrary element of $\mathcal{X}$. Indeed, such ξ^* is obtained at Line 5 of Algorithm 1 by optimality of (ξ^*, π^*), or at Line 10 of Algorithm 2 by optimality of ξ^* in the cut selection phase. So, by claims a) and b) of Lemma 2, the conditions at Line 6 of Algorithm 1 and at Line 8 of Algorithm 2 are satisfied only if $\pi^* \notin \bigcup_{\xi \in \tilde{\Xi}} \tilde{\Pi}^\xi$. Since $\pi^* \in \text{dir}(\Pi)$ by Assumption 2, we have $\bigcup_{\xi \in \tilde{\Xi}} \tilde{\Pi}^\xi = \text{dir}(\Pi)$ after $|\text{dir}(\Pi)|$ executions of Line 8 of Algorithm 1 or Line 10 of Algorithm 2. When this latter condition is satisfied, claim c) of Lemma 2 then ensures that the master problem (MP-$SICG(\tilde{\Xi}, \tilde{\Pi})$) with sets $\tilde{\Xi}$ and $\tilde{\Pi}$ constructed through Algorithms 1 and 2, is equivalent to problem (P). This implies that the conditions at Line 6 of Algorithm 1 and at Line 8 of Algorithm 2 will not be satisfied at their next execution and both algorithms will terminate. This proves the result regarding Algorithm 2, and the result regarding Algorithm 1 follows from Proposition 2 and claim c) of Lemma 2.

The number of bilinear and linear subproblems that need to be solved through the execution of Algorithm 1 then follows directly from the above results.

■

B Detailed algorithms

In this section, we present the details of the algorithms under study. We first detail the CG, C&CG, SICG-noPE, and SICG-noPECS algorithms tested in Section 3. We then analyze how the mountain climbing

algorithm (Konno, 1976) can be incorporated within the SICG algorithm and assess its numerical impact.

B.1 CG and C&CG algorithms

Algorithm 3: Constraint Generation (CG) algorithm (Bertsimas et al., 2012)

$\tilde{\Psi} \leftarrow \emptyset$
while true **do**
  Solve $(MP\text{-}CG(\tilde{\Psi}))$
  if $(MP\text{-}CG(\tilde{\Psi}))$ *is feasible* **then**
    Let $\hat{x}$ be an optimal solution of $(MP\text{-}CG(\tilde{\Psi}))$
    Let (ξ^*, π^*) be an optimal solution of $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$
    if ${\pi^*}^\top (h(\xi^*) - T(\xi^*)\hat{x}) > 0$ **then**
      $\tilde{\Psi} \leftarrow \tilde{\Psi} \cup \{(\xi^*, \pi^*)\}$
    else
      return *An optimal solution of* (P) *:* $\hat{x}$
    end
  else
    return (P) *is infeasible*
  end
end

Algorithm 4: Constraint-and-Column Generation (C&CG) algorithm (Zeng and Zhao, 2013)

$\tilde{\Xi} \leftarrow \emptyset$
while true **do**
  Solve $(MP\text{-}C\&CG(\tilde{\Xi}))$
  if $(MP\text{-}C\&CG(\tilde{\Xi}))$ *is feasible* **then**
    Let $\hat{x}$ be an optimal solution of $(MP\text{-}C\&CG(\tilde{\Xi}))$
    Let (ξ^*, π^*) be an optimal solution of $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$
    if ${\pi^*}^\top (h(\xi^*) - T(\xi^*)\hat{x}) > 0$ **then**
      $\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \{\xi^*\}$
    else
      return *An optimal solution of* (P) *:* $\hat{x}$
    end
  else
    return (P) *is infeasible*
  end
end

B.2 SICG-noPE and SICG-noPECS algorithms

In this section, we provide further details on the SICG-noPE and SICG-noPECS variants of the SICG algorithm. These variants aim to isolate the numerical contributions of each algorithmic element within the algorithm. The numerical results comparing each algoritmic variant to the SICG algorithm are presented in Section 3.2.1.

Algorithm 5: The SICG-noPE algorithm

$\tilde{\Xi} \leftarrow \emptyset$, $\tilde{\Pi} := (\tilde{\Pi}^{\xi})_{\xi \in \tilde{\Xi}}$, $\tilde{\Psi} \leftarrow \emptyset$
while true **do**
  ctr-added $\leftarrow$ true
  while *ctr-added* **do**
    ctr-added $\leftarrow$ false
    Solve $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}, \tilde{\Psi}))$ `// master problem phase`
    if $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}, \tilde{\Psi}))$ *is feasible* **then**
      Let $\hat{x}$ be an optimal solution of $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}, \tilde{\Psi}))$
      for $\hat{\xi} \in \tilde{\Xi}$ **do**
        Let π^* be an optimal solution of $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ `// cut generation phase`
        if $\pi^{*\top}\left(h(\hat{\xi}) - T(\hat{\xi})\hat{x}\right) > 0$ **then**
          ctr-added $\leftarrow$ true
          Let ξ^* be an optimal solution of $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$ `// cut selection phase`
          if $\xi^* \in \tilde{\Xi}$ **then**
            $\tilde{\Pi}^{\xi^*} \leftarrow \tilde{\Pi}^{\xi^*} \cup \{\pi^*\}$
          else
            $\tilde{\Psi} \leftarrow \tilde{\Psi} \cup \{(\xi^*, \pi^*)\}$
          end
        end
      end
    end
  end
  if $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}, \tilde{\Psi}))$ *is feasible* **then**
    Let (ξ^*, π^*) be an optimal solution of $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$
    if $\pi^{*\top}(h(\xi^*) - T(\xi^*)\hat{x}) > 0$ **then**
      $\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \{\xi^*\}$
      $\tilde{\Pi}^{\xi^*} \leftarrow \{\pi^*\}$
    else
      return *An optimal solution of* (P) *:* $\hat{x}$
    end
  else
    return (P) *is infeasible*
  end
end

The first variant called SICG-noPE concerns the pool expansion phase performed at Line 19 of Algorithm 2. In order to describe this variant, we first introduce a relaxation of $(P\text{-}SICG)$ that considers CG constraints (29) in addition to the semi-infinite constraints (28):

$$z_{\text{SICG}}(\tilde{\Xi}, \tilde{\Pi}, \tilde{\Psi}) := \min_{x \in \mathcal{X}} c^\top x \qquad (MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}, \tilde{\Psi}))$$

$$\text{s.t. } \pi^\top (h(\xi) - T(\xi)x) \leq 0 \qquad \forall \xi \in \tilde{\Xi}, \forall \pi \in \tilde{\Pi}^{\xi} \tag{28}$$

$$\pi^\top (h(\xi) - T(\xi)x) \leq 0 \qquad \forall (\xi, \pi) \in \tilde{\Psi}. \tag{29}$$

The SICG-noPE algorithm is described in Algorithm 5. This algorithm, contrary to the SICG algo-

Algorithm 6: The SICG-noPECS algorithm

1 $\tilde{\Xi} \leftarrow \emptyset$, $\tilde{\Pi} := (\tilde{\Pi}^{\xi})_{\xi \in \tilde{\Xi}}$
2 **while** true **do**
3 *ctr-added* $\leftarrow$ true
4 **while** *ctr-added* **do**
5 *ctr-added* $\leftarrow$ false
6 Solve $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ `// master problem phase`
7 **if** $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ *is feasible* **then**
8 Let $\hat{x}$ be an optimal solution of $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$
9 **for** $\hat{\xi} \in \tilde{\Xi}$ **do**
10 Let π^* be an optimal solution of $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ `// cut generation phase`
11 **if** $\pi^{*\top}\left(h(\hat{\xi}) - T(\hat{\xi})\hat{x}\right) > 0$ **then**
12 *ctr-added* $\leftarrow$ true
13 $\xi^* \leftarrow \hat{\xi}$
14 $\tilde{\Pi}^{\xi^*} \leftarrow \tilde{\Pi}^{\xi^*} \cup \{\pi^*\}$
15 **end**
16 **end**
17 **end**
18 **end**
19 **if** $(MP\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}))$ *is feasible* **then**
20 Let (ξ^*, π^*) be an optimal solution of $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$
21 **if** $\pi^{*\top}(h(\xi^*) - T(\xi^*)\hat{x}) > 0$ **then**
22 $\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \{\xi^*\}$
23 $\tilde{\Pi}^{\xi^*} \leftarrow \{\pi^*\}$
24 **else**
25 **return** *An optimal solution of* (P) *:* $\hat{x}$
26 **end**
27 **else**
28 **return** (P) *is infeasible*
29 **end**
30 **end**

rithm provided in Algorithm 1, does not perform a pool expansion phase. Indeed, this phase potentially augments the set $\tilde{\Xi}$ with new realizations ($\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \tilde{\Xi}_{new}$) with the effect of generating new semi-infinite constraints $\pi^\top (h(\xi) - T(\xi)x) \leq 0 \;\; \forall \xi \in \tilde{\Xi}_{new}, \forall \pi \in \Pi$. The SICG-noPE variant, instead, generates the standalone constraint $\pi^{*\top}(h(\xi^*) - T(\xi^*)x) \leq 0$ whenever the cut selection phase, performed at Line 10 of Algorithm 2, yields an uncertainty realization $\xi^* \notin \tilde{\Xi}$. This also implies that in subsequent iterations of Algorithm 2 the considered ξ^* will not be in $\tilde{\Xi}$ and therefore will not be used to generate new violated constraints.

The second variant is called SICG-noPECS and concerns the cut selection and pool expansion phases, performed, respectively, at Lines 10 and 19 of Algorithm 2. This variant foregoes both of these phases and is equivalent to an algorithm in which SICG master problems, $MP\text{-}SICG(\tilde{\Xi})$, are solved with classical cut generation. We provide a detailed description of this variant in Algorithm 6.

B.3 Mountain climbing algorithm and numerical comparison of SICG mountain climbing variants

First introduced in (Konno, 1976), the mountain climbing algorithm is a heuristic algorithm developed for disjoint bilinear programming problems. During the algorithm, starting from an initial solution, the problem is solved by alternately fixing one of the two sets of variables until a convergence criterion is met. The algorithm, therefore, can improve a given solution by solving a number of linear problems.

The mountain climbing algorithm has been utilized in the adjustable robust optimization literature to help decomposition algorithms, such as CG and C&CG, in addressing the numerical difficulty of the bilinear subproblem. Indeed, for a given first-stage solution $\hat{x} \in \mathcal{X}$, $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$ is the maximization of a bilinear function over the two independent sets Ξ and Π. The algorithm has been mostly used in heuristics to obtain dual bounds (Vieira et al., 2023; Lorca and Sun, 2017), or as a preprocessing step to warm-start the solution of the subproblem (Bertsimas and Shtern, 2018).

B.3.1 Adapting the Mountain climbing algorithm to SICG

In the context of the SICG algorithm, a mountain climbing algorithm can be incorporated into Algorithm 2. In particular, the two linear problems $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$ and $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$, used, respectively, to identify and improve violated cuts, are the same as those alternately used within the mountain climbing algorithm. It follows that we can continue this process to try to find a more deeply violated cut.

Algorithm 7: Mountain climbing phase $(\hat{x}, \hat{\xi}, \pi^*, \xi^*)$

while ${\pi^*}^\top \left(h(\xi^*) - T(\xi^*)\hat{x}\right) > {\pi^*}^\top \left(h(\hat{\xi}) - T(\hat{\xi})\hat{x}\right)$ **do**
  $\hat{\xi} \leftarrow \xi^*$
  Let π^* be an optimal solution of $(SP\text{-}P(\hat{x}, \{\hat{\xi}\}, \Pi_{\text{norm}}))$
  Let ξ^* be an optimal solution of $(SP\text{-}P(\hat{x}, \Xi, \{\pi^*\}))$
end
return (π^*, ξ^*)

The mountain climbing phase described in Algorithm 7 can be incorporated into Algorithm 2, in between the execution of Lines 10 and 11. Each iteration of this procedure successively updates optimal solutions π^* and ξ^* until the constraint violation obtained for π^* at two consecutive realizations $\hat{\xi}$ and ξ^* is equal, as proposed in (Vieira et al., 2023; Lorca and Sun, 2017). Each iteration ends with a maximization over Ξ to fully exploit Lemma 2 and potentially generate a new semi-infinite constraint afterwards at Line 19 of Algorithm 2.

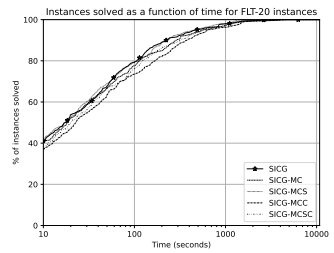
In addition to obtaining a deeper cut, the intermediate information gathered during the mountain climbing algorithm can also be incorporated into the SICG algorithm. Indeed, since many different uncertainty realizations are encountered over the course of Algorithm 7, one can generate a semi-infinite constraint corresponding to each of those intermediate realizations (by adding them to $\tilde{\Xi}$). One can also generate each violated constraint encountered during the execution of Algorithm 7 instead of just the most violated ones obtained at convergence, without generating a semi-infinite constraint from intermediate realizations. In so doing, the cut obtained as a result of the cut generation phase of Algorithm 2 would also be generated. Finally, these two ideas can also be used simultaneously, adding both the individual violated constraints and their semi-infinite versions (through the addition of intermediate realizations to $\tilde{\Xi}$) to the master problem. In Section B.3.2, we numerically test each of these variants.

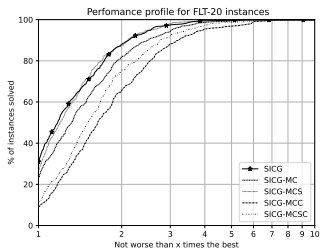
B.3.2 Comparing the mountain climbing variants to the SICG algorithm

In this subsection, we present in Figure 4 the comparison between the SICG algorithm and its variants incorporating different implementations of the mountain climbing algorithm. We denote the version of the algorithm that includes the mountain climbing procedure described in Algorithm 7 by SICG-MC and consider three variants that each incorporate additional information from the intermediate steps of the mountain climbing algorithm to the relaxed master problem. These include a version that adds every non-generated uncertainty realization encountered during the mountain climbing algorithm to $\tilde{\Xi}$ (so that a semi-infinite constraint is generated), one that adds every violated constraint encountered to the master program (without adding the associated uncertainty realizations to $\tilde{\Xi}$), and one that adds both (uncertainty realizations are added to $\tilde{\Xi}$ and a constraint is added for each violated uncertainty realization-dual point pair). These variants are denoted SICG-MCS, SICG-MCC, and SICG-MCSC, respectively, in the following. Characteristics of each algorithmic variant are summarized in Table 2.

Variants	Cut Selection	Pool Expansion	Mountain Climbing	Add Realizations	Add Constraints
SICG-noPECS					
SICG-noPE	✓				
SICG	✓	✓			
SICG-MC	✓	✓	✓		
SICG-MCS	✓	✓	✓	✓	
SICG-MCC	✓	✓	✓		✓
SICG-MCSC	✓	✓	✓	✓	✓

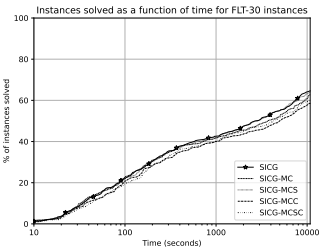
Table 2: Summary of SICG variants.

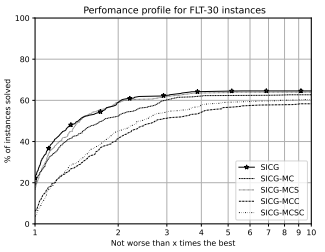




Detailed results for FLT-20 instances: 480/480 instances

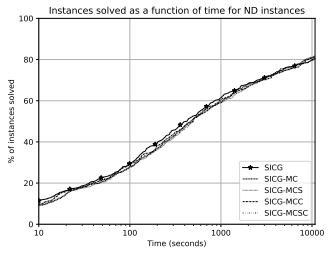
Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
SICG	107.7	70.8	3.9	336.4	108.0
SICG-MC	129.2	72.8	4.4	165.7	58.5
SICG-MCS	94.1	68.9	3.9	269.4	167.2
SICG-MCC	154.9	74.2	5.4	360.2	38.0
SICG-MCSC	122.2	71.1	4.7	632.5	91.7

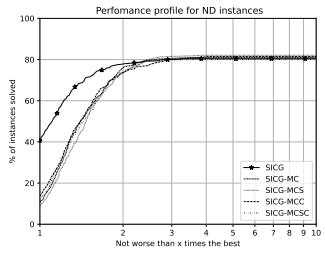




Detailed results for FLT-30 instances: 274/480 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
SICG	971.2	77.9	3.8	1212.2	219.6
SICG-MC	1227.9	80.9	4.5	425.3	111.6
SICG-MCS	995.4	78.4	4.0	762.9	350.1
SICG-MCC	1523.9	78.6	5.4	1660.9	74.3
SICG-MCSC	1360.5	76.4	5.0	2515.5	158.7





Detailed results for ND instances: 355/480 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
SICG	736.4	45.4	9.2	1803.4	304.0
SICG-MC	840.5	41.8	9.4	1907.6	269.1
SICG-MCS	852.7	38.5	8.9	2286.0	332.0
SICG-MCC	956.1	47.7	11.0	3608.8	211.5
SICG-MCSC	870.4	44.3	10.5	4420.3	265.2

Figure 4: Comparison of the SICG algorithm and its variants incorporating different implementations of the mountain climbing algorithm on FLT-20, FLT-30, and ND instances.

The results presented in Figure 4 show that all variants behave very similarly, on average, with SICG performing the best.

We first note that the addition of the mountain climbing phase to the SICG algorithm allows the SICG-MC variant to identify deeper cuts at each iteration and thus to converge with fewer constraints added to the master program compared to the SICG algorithm. However, the SICG-MC variant is also characterized by a reduction of $|\tilde{\Xi}|$ and an augmentation of #SP compared to SICG. This difference can be explained by different mountain climbing phases converging to the same violated uncertainty realization. As a result of these differences, the SICG-MC variant has an increased solution time compared to that of the SICG algorithm.

We next consider the SICG-MCS variant which adds every non-generated uncertainty realization encountered during the mountain climbing phase to $\tilde{\Xi}$. We observe that this has the intended effect as it increases $|\tilde{\Xi}|$, which in turn reduces #SP while increasing #Ctr compared to SICG-MC. These changes reduce the solution time for the FLT instances, while slightly increasing the solution time for the ND instances. On the other hand, compared to the SICG algorithm, the number of constraints generated is reduced because the SICG-MCS variant finds deeper cuts thanks to the mountain climbing phase. Further, more uncertainty realizations are generated with $|\tilde{\Xi}|$ increasing compared to SICG because SICG-MCS has a better chance of encountering new uncertainty realizations within the mountain climbing phase and adding them to $\tilde{\Xi}$. However, the SICG-MCS variant does not have a consistent positive impact on #SP and the solution time in our experiments.

Finally, we focus on the SICG-MCC and SICG-MCSC variants, which add every violated constraint encountered during the mountain climbing phase to the master program. This increases the solution time compared to the SICG-MC and SICG-MCS variants. Indeed, adding every identified cut makes Algorithm 2 converge with fewer iterations. Consequently, fewer new uncertainty realizations are generated, and the bilinear subproblem is solved more often, which negatively impacts the overall solution time.

C Subproblem reformulations and their numerical comparison

In this section, we present some numerical choices that affect the solution of the subproblems. We then detail the subproblem reformulations for each application considered in Section 3 and present a numerical comparison of these formulations.

C.1 Numerical choices affecting the solution of subproblems

We summarize the characteristics of subproblem formulations $(SP\text{-}P\text{-}Dual(\hat{x}))$ and $(SP\text{-}P\text{-}KKT(\hat{x}))$ in Table 3. We remark that the choice between the two models is sometimes dictated by the structure of the problem, *e.g.*, the uncertainty set Ξ. On the other hand, when both models can be used, the one with a better numerical performance would be preferable. Unfortunately, an a priori comparison between $(SP\text{-}P\text{-}Dual(\hat{x}))$ and $(SP\text{-}P\text{-}KKT(\hat{x}))$ cannot be established since they contain different types of variables and constraints.

Another choice that can affect the numerical solution of subproblems is the choice of matrix N. From a theoretical perspective, as long as the feasibility of the inner problem for any first-stage solution and uncertainty realization, and therefore the boundedness of its dual, can be guaranteed, any value of N can be used. However, from a practical perspective, this choice has an impact on the solution time of the subproblem as well as the convergence of decomposition algorithms since different subproblem

Type	$(SP\text{-}P\text{-}KKT(\hat{x}))$	$(SP\text{-}P\text{-}Dual(\hat{x}))$
Binary variables	$m_y + n_y + n_\alpha$	n_ω
Continuous variables	$n_y + n_\alpha + m_y$	$m_y + \nu$
Constraints	$m_\xi + 3(m_y + n_y + n_\alpha)$	$m_\omega + n_y + n_\alpha + 2\nu$
Product linearized	$m_y + n_y + n_\alpha$	ν

Table 3: Size of $(SP\text{-}P\text{-}KKT(\hat{x}))$ and $(SP\text{-}P\text{-}Dual(\hat{x}))$ models. We denote the number of linear constraints describing Ω and Ξ by m_ω and m_ξ, respectively, and do not consider the non-negativity constraints. For $(SP\text{-}P\text{-}Dual(\hat{x}))$, we also denote by $\nu \leq m_y n_\omega$ the number of variables in G with a non-zero objective coefficient.

solutions lead to different constraints added to the relaxed master problems. In the Benders' decomposition literature, the numerical effects of the matrix N and the normalization constraint $N^\top \pi \leq \mathbf{1}$ on the convergence of the algorithm are referred to as *cut selection strategies* (Fischetti et al., 2010). Indeed, certain choices of N can be associated with a heuristic search for a minimal infeasible system that cuts off an infeasible solution. To this end, Fischetti et al. (2010) propose the choice $N = \mathbf{1}$, with $n_\alpha = 1$, that is, the same artificial variable is used for all second-stage constraints with a coefficient of one. This choice was adopted in Ayoub and Poss (2016) implicitly through the normalization constraint $||\pi||_1 \leq 1$. One can further refine this choice by using artificial variables only when needed, as suggested by Fischetti et al. (2010). To do so, let $I \subseteq [m_y]$ such that for $i \in I$ we have that $T_i(\xi) = 0$ for all $\xi \in \Xi$. One can then set $N = \mathbf{1} - \sum_{i \in I} e_i$ whenever problem (P) is known to be feasible. Indeed, for $i \in I$, $W_i y \geq h_i(\xi) - T_i(\xi)\hat{x}$ is equivalent to $W_i y \geq h_i(\xi)$, *i.e.*, the satisfaction of this constraint is independent of the choice of first-stage variables $\hat{x}$. As such, the existence of $\xi \in \Xi$ and $i \in I$ such that $W_i y < h_i(\xi)$ implies that problem (P) is infeasible. In our numerical results, we adopt the choice $N = \mathbf{1} - \sum_{i \in I} e_i$ and refer the reader to (Fischetti et al., 2010; Hosseini and Turner, 2024) for a more detailed discussion of cut selection strategies as it relates to the Benders' decomposition algorithm.

A comparison similar to the one made in Table 3 can also be made concerning the choice between different MILP formulations of $(SP\text{-}Feas(\hat{x}, \Xi, \bar{\Pi}_{\text{feas}}))$ and $(SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$. These problems need to be solved in the context of decomposition algorithms for adjustable robust optimization problem with an optimality problem at the second stage (see Appendix D).

To conclude, Table 4 summarizes the different subproblem formulations presented along with the necessary assumptions for their use. Although we have focused on generic formulations here, problem-specific formulations with further assumptions also exist in the literature. For instance, Simchi-Levi et al. (2019) study problems with a network flow structure in the second stage. The issue of calculating smaller big-M coefficients is additionally important and can be better achieved by exploiting problem structure. For instance, Gabrel et al. (2014a) study a facility location-transportation problem under strict feasibility constraints to calculate such bounds, which can have a significant effect on the numerical solution of subproblems.

C.2 Facility location transportation problem

For the FLT problem, for a given first-stage solution $\hat{z} \in \mathbb{R}_+^{|I|}$ such that $\hat{z}_i \leq c_i$ for each $i \in I$, depending on whether the second-stage problem is written as a feasibility problem (rendered feasible through artificial variables) or an optimality problem, the max-min formulation of the separation subproblem takes the

Case study			Applicable models
Relatively complete recourse	Formulation	Assumption 4	
no	(P)	no	$(SP\text{-}P\text{-}KKT(\hat{x}))$
		yes	$(SP\text{-}P\text{-}Dual(\hat{x}))$ or $(SP\text{-}P\text{-}KKT(\hat{x}))$
	(Q)	no	$(SP\text{-}Feas\text{-}KKT(\hat{x}))$ and $(SP\text{-}Opt\text{-}KKT(\hat{x}))$
		yes	$(SP\text{-}Feas\text{-}Dual(\hat{x}))$ and $(SP\text{-}Opt\text{-}Dual(\hat{x}))$ or $(SP\text{-}Feas\text{-}KKT(\hat{x}))$ and $(SP\text{-}Opt\text{-}KKT(\hat{x}))$
yes		no	$(SP\text{-}Opt\text{-}KKT(\hat{x}))$
		yes	$(SP\text{-}Opt\text{-}Dual(\hat{x}))$ or $(SP\text{-}Opt\text{-}KKT(\hat{x}))$

Table 4: Possible subproblem formulations under various hypotheses.

form:

$$\max_{\substack{\gamma\in B_I^\Gamma\\ \delta\in B_J^\Delta}} \min_{\substack{\alpha\in\mathbb{R}_+\\ y\in\mathbb{R}_+^{|I|\times|J|}\\ h\in\mathbb{R}_+^{|J|}}} \alpha \tag{30a}$$

$$\text{s.t.} \quad \sum_{(i,j)\in I\times J} f_{ij}y_{ij} + \sum_{j\in J}\hat{f}h_j - \alpha \le \hat{\theta} \tag{30b}$$

$$\sum_{j\in J} y_{ij} - \alpha \le \hat{z}_i(1-\gamma_i) \qquad \forall i\in I \tag{30c}$$

$$\sum_{i\in I} y_{ij} + h_j \ge \bar{d}_j + \hat{d}_j\delta_j \qquad \forall j\in J, \tag{30d}$$

and

$$\max_{\substack{\gamma\in B_I^\Gamma\\ \delta\in B_J^\Delta}} \min_{\substack{y\in\mathbb{R}_+^{|I|\times|J|}\\ h\in\mathbb{R}_+^{|J|}}} \sum_{(i,j)\in I\times J} f_{ij}y_{ij} + \sum_{j\in J}\hat{f}h_j \tag{31a}$$

$$\text{s.t.} \quad \sum_{j\in J} y_{ij} \le \hat{z}_i(1-\gamma_i) \qquad \forall i\in I \tag{31b}$$

$$\sum_{i\in I} y_{ij} + h_j \ge \bar{d}_j + \hat{d}_j\delta_j \qquad \forall j\in J, \tag{31c}$$

respectively. We remark that, in formulation (30), the artificial variable, α, is added only to those constraints involving the first-stage variables $\hat{\theta}$, $\hat{z}$ as mentioned in Appendix C.1. Additionally, the FLT problem satisfies the relatively complete recourse assumption formalized in Assumption 5. As a result, feasibility subproblems are not necessary when this problem is formulated with an optimality problem at the second stage as in (31).

We next derive the dual sets associated with the inner minimization problem of these models using linear programming duality. Let u and v denote the dual variables associated with the constraints (30c) and (31b) and constraints (30d) and (31c), respectively, in each model. Let, further, w denote the dual variable associated with the constraint (30b). We obtain the dual sets:

$$\Pi^{\text{FLT}}_{\text{norm}} = \left\{ w\in\mathbb{R}_+, u\in\mathbb{R}_+^{|I|}, v\in\mathbb{R}_+^{|J|} \;\middle|\; \sum_{i\in I} u_i + w \le 1, -u_i + v_j \le f_{ij}w \;\; \forall (i,j)\in I\times J, v_j \le \hat{f}w \;\; \forall j\in J \right\},$$

$$\bar{\Pi}^{\text{FLT}}_{\text{opt}} = \left\{ u \in \mathbb{R}^{|I|}_{+}, v \in \mathbb{R}^{|J|}_{+} \middle| -u_i + v_j \leq f_{ij} \ \ \forall (i,j) \in I \times J, v_j \leq \hat{f} \ \ \forall j \in J \right\},$$

for (30) and (31), respectively.

Depending on whether linear programming duality or KKT conditions are used to reformulate the inner minimization problem in (30) and (31), we will have different reformulations, which can then be linearized. In the following, we describe these linearized formulations.

We start with reformulations based on linear programming duality. We recall that, in order to linearize the bilinear subproblem formulation obtained in this case, we require the extreme points of the uncertainty set to be integer. To this end, for a general budgeted uncertainty set where we do not assume the budget parameter to be integer, we apply the transformation proposed in Ayoub and Poss (2016). More specifically, for the budgeted uncertainty set $B^b_S := \{\xi \in [0,1]^{|S|} | \sum_{s \in S} \xi_s \leq b\}$, we write:

$$\Omega^b_S := \ \left\{ \omega^1 \in [0,1]^{|S|}, \ \omega^2 \in [0,1]^{|S|} \ \middle| \ \omega^1 + \omega^2 \leq \mathbf{1}, \ \mathbf{1}^\top \omega^1 \leq \lfloor b \rfloor, \ \mathbf{1}^\top \omega^2 \leq 1, \ \right\}.$$

We then substitute $\xi_s = \omega^1_s + (b - \lfloor b \rfloor)\omega^2_s$ for $s \in S$ where variables ω^2 do not affect the value of ξ when the budget parameter b is integer.

As a result of this transformation of the uncertainty set, uncertain parameters γ and δ will be split into γ^1, γ^2 and δ^1, δ^2, respectively. Dualizing the inner minimization problem in (30) yields the following bilinear problem:

$$\max \quad -\sum_{i \in I} \hat{z}_i u_i (1 - (\gamma^1_i + (\Gamma - \lfloor \Gamma \rfloor)\gamma^2_i)) + \sum_{j \in J} \bar{d}_j v_j + \hat{d}_j v_j (\delta^1_j + (\Delta - \lfloor \Delta \rfloor)\delta^2_j) \ - \hat{\theta} w \tag{32a}$$

$$\text{s.t.} \quad (\gamma^1, \gamma^2) \in \Omega^\Gamma_I \cap \left(\{0,1\}^{|I|} \times \{0,1\}^{|I|} \right) \tag{32b}$$

$$(\delta^1, \delta^2) \in \Omega^\Delta_J \cap \left(\{0,1\}^{|J|} \times \{0,1\}^{|J|} \right) \tag{32c}$$

$$(u, v, w) \in \Pi^{\text{FLT}}_{\text{norm}} \tag{32d}$$

We may then, for $l \in \{1,2\}$, proceed to the linearization of bilinear terms $u \circ \gamma^l$ and $v \circ \delta^l$ that appear in the objective function of the bilinear maximization problem. Let μ and ν be the new variables representing the bilinear products, for a given $l \in \{1,2\}$, *i.e.*, $\mu^l = u \circ \gamma^l \in \mathbb{R}^{|I|}_{+}$, $\nu^l = v \circ \delta^l \in \mathbb{R}^{|J|}_{+}$. We obtain the linearized $(SP\text{-}P\text{-}Dual(\hat{x}))$ model for the FLT problem as:

$$\max \quad -\sum_{i \in I} \hat{z}_i (u_i - (\mu^1_i + (\Gamma - \lfloor \Gamma \rfloor)\mu^2_i)) + \sum_{j \in J} \bar{d}_j v_j + \hat{d}_j (\nu^1_j + (\Delta - \lfloor \Delta \rfloor)\nu^2_j) \ - \hat{\theta} w \tag{33a}$$

$$\text{s.t.} \quad \mu^l_i \leq u_i \qquad \forall l \in \{1,2\}, i \in I \tag{33b}$$

$$\mu^l_i \leq \gamma^l_i \qquad \forall l \in \{1,2\}, i \in I \tag{33c}$$

$$\nu^l_j \leq v_j \qquad \forall l \in \{1,2\}, j \in J \tag{33d}$$

$$\nu^l_j \leq \max\{1, \min_{i \in I}(f_{ij})\}\delta^l_j \qquad \forall l \in \{1,2\}, j \in J \tag{33e}$$

$$(\gamma^1, \gamma^2) \in \Omega^\Gamma_I \cap \left(\{0,1\}^{|I|} \times \{0,1\}^{|I|} \right) \tag{33f}$$

$$(\delta^1, \delta^2) \in \Omega_J^{\Delta} \cap \left(\{0,1\}^{|J|} \times \{0,1\}^{|J|}\right) \tag{33g}$$

$$(u, v, w) \in \Pi_{\text{norm}}^{\text{FLT}} \tag{33h}$$

$$\mu^1, \mu^2 \in \mathbb{R}_+^{|I|} \tag{33i}$$

$$\nu^1, \nu^2 \in \mathbb{R}_+^{|J|}, \tag{33j}$$

where we use $u_i \leq 1$ (through $\sum_{i \in I} u_i + w \leq 1$) for $i \in I$ and $v_j \leq \max\{1, \min_{i \in I} f_{ij}\}$ (through $v_j \leq u_i + f_{ij} w \leq f_{ij}(u_i + w) \leq f_{ij}$ when $f_{ij} \geq 1$ and $v_j \leq u_i + f_{ij} w \leq 1$ otherwise) for $j \in J$ to calculate the necessary big-M coefficients.

Similarly, applying linear programming duality to the inner minimization problem of (31) leads to the bilinear and the linearized (*SP-Opt-Dual*($\hat{x}$)) models for the FLT problem:

$$\max \quad -\sum_{i \in I} \hat{z}_i u_i (1 - (\gamma_i^1 + (\Gamma - \lfloor \Gamma \rfloor)\gamma_i^2)) + \sum_{j \in J} \bar{d}_j v_j + \hat{d}_j v_j (\delta_j^1 + (\Delta - \lfloor \Delta \rfloor)\delta_j^2) \tag{34a}$$

$$\text{s.t.} \quad (\gamma^1, \gamma^2) \in \Omega_I^{\Gamma} \cap \left(\{0,1\}^{|I|} \times \{0,1\}^{|I|}\right) \tag{34b}$$

$$(\delta^1, \delta^2) \in \Omega_J^{\Delta} \cap \left(\{0,1\}^{|J|} \times \{0,1\}^{|J|}\right) \tag{34c}$$

$$(u, v) \in \bar{\Pi}_{\text{opt}}^{\text{FLT}} \tag{34d}$$

and:

$$\max \quad -\sum_{i \in I} \hat{z}_i (u_i - (\mu_i^1 + (\Gamma - \lfloor \Gamma \rfloor)\mu_i^2) + \sum_{j \in J} \bar{d}_j v_j + \hat{d}_j (\nu_j^1 + (\Delta - \lfloor \Delta \rfloor)\nu_j^2) \tag{35a}$$

$$\text{s.t.} \quad \mu_i^l \leq u_i \qquad \forall l \in \{1,2\}, i \in I \tag{35b}$$

$$\mu_i^l \leq \hat{f}\gamma_i^l \qquad \forall l \in \{1,2\}, i \in I \tag{35c}$$

$$\nu_j^l \leq v_j \qquad \forall l \in \{1,2\}, j \in J \tag{35d}$$

$$\nu_j^l \leq \hat{f}\delta_j^l \qquad \forall l \in \{1,2\}, j \in J \tag{35e}$$

$$(\gamma^1, \gamma^2) \in \Omega_I^{\Gamma} \cap \left(\{0,1\}^{|I|} \times \{0,1\}^{|I|}\right) \tag{35f}$$

$$(\delta^1, \delta^2) \in \Omega_J^{\Delta} \cap \left(\{0,1\}^{|J|} \times \{0,1\}^{|J|}\right) \tag{35g}$$

$$(u, v) \in \bar{\Pi}_{\text{opt}}^{\text{FLT}} \tag{35h}$$

$$\mu^1, \mu^2 \in \mathbb{R}_+^{|I|} \tag{35i}$$

$$\nu^1, \nu^2 \in \mathbb{R}_+^{|J|}, \tag{35j}$$

where we use $u_i \leq \hat{f}$ for $i \in I$ and $v_j \leq \hat{f}$ for $j \in J$ to calculate the necessary big-M coefficients. We remark that variables u are not naturally bounded in the set $\bar{\Pi}_{\text{opt}}^{\text{FLT}}$. However, since the inner minimization problem in (31) is always feasible (hence its dual is always bounded), they can artificially be bounded by the value $\hat{f}$ without cutting off any optimal solution.

We next focus on reformulations based on KKT optimality conditions for which we do not need to

transform the uncertainty set. In these formulations, we will need to linearize the complementary slackness conditions.

For the $(SP\text{-}P\text{-}KKT(\hat{x}))$ model based on applying the KKT conditions to the inner minimization problem of (30) complementary slackness conditions take the form $\alpha\left(\sum_{i\in I} u_i + w - 1\right) = 0$, $w\left(\sum_{(i,j)\in I\times J} f_{ij}y_{ij} + \sum_{j\in J}\hat{f}h_j - \alpha - \hat{\theta}\right) = 0$, $y_{ij}\left(-u_i + v_j - f_{ij}w\right) = 0$ for $(i,j) \in I \times J$, $h_j\left(v_j - \hat{f}w\right) = 0$ for $j \in J$, $u_i\left(\sum_{j\in J} y_{ij} - \alpha - \hat{z}_i(1-\gamma_i)\right) = 0$ for $i \in I$, and $v_j\left(\sum_{i\in I} y_{ij} + h_j - (\bar{d}_j + \hat{d}_j\delta_j)\right) = 0$ for $j \in J$. To linearize these conditions, we introduce the binary variables $\sigma^\alpha \in \{0,1\}$, $\sigma^w \in \{0,1\}$, $\sigma^y \in \{0,1\}^{|I|\times|J|}$, $\sigma^h \in \{0,1\}^{|J|}$, $\sigma^u \in \{0,1\}^{|I|}$, and $\sigma^v \in \{0,1\}^{|J|}$. We then obtain the linearized $(SP\text{-}P\text{-}KKT(\hat{x}))$ model for the FLT problem as:

$$\max \quad \alpha \tag{36a}$$

$$\text{s.t.} \quad \text{(30b)} - \text{(30d)} \tag{36b}$$

$$(w,u,v) \in \Pi^{\text{FLT}}_{\text{norm}} \tag{36c}$$

$$\sum_{i\in I} u_i + w \geq 1 + (\sigma^\alpha - 1) \tag{36d}$$

$$\alpha \leq M_\alpha \sigma^\alpha \tag{36e}$$

$$-u_i + v_j \geq f_{ij}w + \max\{1, f_{ij}\}(\sigma^y_{ij} - 1) \qquad \forall (i,j) \in I \times J \tag{36f}$$

$$y_{ij} \leq (\bar{d}_j + \hat{d}_j)\sigma^y_{ij} \qquad \forall (i,j) \in I \times J \tag{36g}$$

$$v_j \geq \hat{f}w + \hat{f}(\sigma^h_j - 1) \qquad \forall j \in J \tag{36h}$$

$$h_j \leq (\bar{d}_j + \hat{d}_j)\sigma^h_j \qquad \forall j \in J \tag{36i}$$

$$\sum_{(i,j)\in I\times J} f_{ij}y_{ij} + \sum_{j\in J}\hat{f}h_j - \alpha \geq \hat{\theta} + M_\alpha(\sigma^w - 1) \tag{36j}$$

$$w \leq \sigma^w \tag{36k}$$

$$\sum_{j\in J} y_{ij} - \alpha \geq \hat{z}_i(1-\gamma_i) + (M_\alpha + \hat{z}_i)(\sigma^u_i - 1) \qquad \forall i \in I \tag{36l}$$

$$u_i \leq \sigma^u_i \qquad \forall i \in I \tag{36m}$$

$$\sum_{i\in I} y_{ij} + h_j \leq \bar{d}_j + \hat{d}_j\delta_j + (\bar{d}_j + \hat{d}_j)(1 - \sigma^v_j) \qquad \forall j \in J \tag{36n}$$

$$v_j \leq \max\{1, \min_{i\in I}(f_{ij})\}\sigma^v_j \qquad \forall j \in J \tag{36o}$$

$$\gamma \in B^\Gamma_I \tag{36p}$$

$$\delta \in B^\Delta_J \tag{36q}$$

$$\alpha \in \mathbb{R}_+, y \in \mathbb{R}^{|I|\times|J|}_+, h \in \mathbb{R}^{|J|}_+ \tag{36r}$$

$$\sigma^\alpha \in \{0,1\} \tag{36s}$$

$$\sigma^y_{ij} \in \{0,1\} \qquad \forall (i,j) \in I \times J \tag{36t}$$

$$\sigma^h_j \in \{0,1\} \qquad \forall j \in J \tag{36u}$$

$$\sigma^w \in \{0,1\} \tag{36v}$$

$$\sigma^u_i \in \{0,1\} \qquad \forall i \in I \tag{36w}$$

$$\sigma^v_j \in \{0,1\} \qquad \forall j \in J, \tag{36x}$$

where we use $\alpha \leq M_\alpha := \max\{1,\hat{f}\} \max_{\delta \in B^\Delta_J} \left\{\sum_{j\in J} \bar{d}_j + \hat{d}_j\delta_j\right\}$, $\sum_{i\in I} u_i + w - 1 \geq -1$ (since $u, w \geq 0$), $-u_i + v_j - f_{ij}w \geq -\max\{1, f_{ij}\}$, $y_{ij} \leq (\bar{d}_j + \hat{d}_j)$ for $(i,j) \in I \times J$ (since $f \geq 0$), $v_j - \hat{f}w \geq -\hat{f}$ (since $v \geq 0$ and $w \leq 1$), $h_j \leq (\bar{d}_j + \hat{d}_j)$ (since $\hat{f} \geq 0$) for $j \in J$, $\sum_{(i,j)\in I\times J} f_{ij}y_{ij} + \sum_{j\in J} \hat{f}h_j - \hat{\theta} - \alpha \geq -M_\alpha$ (since algorithmically $\sum_{(i,j)\in I\times J} f_{ij}y_{ij} + \sum_{j\in J} \hat{f}h_j - \hat{\theta} \geq 0$), $w \leq 1$, $\sum_{j\in J} y_{ij} - \alpha - \hat{z}_i(1-\gamma_i) \geq -M_\alpha - \hat{z}_i$ (since $y \geq 0$), $u_i \leq 1$ for $i \in I$, $\sum_{i\in I} y_{ij} + h_j - (\bar{d}_j + \hat{d}_j\delta_j) \leq 0$ (since $f, \hat{f} \geq 0$), $v_j \leq \max\{1, \min_{i\in I}(f_{ij})\}$ for $j \in J$ to calculate the necessary big-M coefficients.

Similarly, for the (SP-Opt-$KKT(\hat{x})$) model based on applying the KKT conditions to the inner minimization problem of (31), the complementary slackness conditions take the form $y_{ij}\left(-u_i + v_j - f_{ij}\right) = 0$ for $(i,j) \in I\times J$, $h_j\left(v_j - \hat{f}\right) = 0$ for $j \in J$, $u_i\left(\sum_{j\in J} y_{ij} - \hat{z}_i(1-\gamma_i)\right) = 0$ for $i \in I$, $v_j\Bigg(\sum_{i\in I} y_{ij} + h_j - (\bar{d}_j + \hat{d}_j\delta_j)\Bigg) = 0$ for $j \in J$. To linearize these conditions, we introduce the binary variables $\sigma^y \in \{0,1\}^{|I|\times|J|}$, $\sigma^h \in \{0,1\}^{|J|}$, $\sigma^u \in \{0,1\}^{|I|}$, and $\sigma^v \in \{0,1\}^{|J|}$. We then obtain the linearized (SP-Opt-$KKT(\hat{x})$) model for the FLT problem as:

$$\max \quad \sum_{(i,j)\in I\times J} f_{ij}y_{ij} + \sum_{j\in J} \hat{f}h_j \tag{37a}$$

$$\text{s.t.} \quad (31b) - (31c) \tag{37b}$$

$$(u,v) \in \bar{\Pi}^{\text{FLT}}_{\text{opt}} \tag{37c}$$

$$-u_i + v_j \geq f_{ij} + (f_{ij} + \hat{f})(\sigma^y_{ij} - 1) \qquad \forall (i,j) \in I\times J \tag{37d}$$

$$y_{ij} \leq \min(c_i, \bar{d}_j + \hat{d}_j)\sigma^y_{ij} \qquad \forall (i,j) \in I\times J \tag{37e}$$

$$v_j \geq \hat{f} + \hat{f}(\sigma^h_j - 1) \qquad \forall j \in J \tag{37f}$$

$$h_j \leq (\bar{d}_j + \hat{d}_j)\sigma^h_j \qquad \forall j \in J \tag{37g}$$

$$\sum_{j\in J} y_{ij} \geq \hat{z}_i(1-\gamma_i) + \hat{z}_i(\sigma^u_i - 1) \qquad \forall i \in I \tag{37h}$$

$$u_i \leq \hat{f}\sigma^u_i \qquad \forall i \in I \tag{37i}$$

$$\sum_{i\in I} y_{ij} + h_j \leq \bar{d}_j + \hat{d}_j\delta_j + 0(1-\sigma^v_j) \qquad \forall j \in J \tag{37j}$$

$$v_j \leq \hat{f}\sigma^v_j \qquad \forall j \in J \tag{37k}$$

$$\gamma \in B^\Gamma_I \tag{37l}$$

$$\delta \in B^\Delta_J \tag{37m}$$

$$y \in \mathbb{R}^{|I|\times|J|}_+, h \in \mathbb{R}^{|J|}_+ \tag{37n}$$

$$\sigma^y_{ij} \in \{0,1\} \qquad \forall (i,j) \in I\times J \tag{37o}$$

$$\sigma^h_j \in \{0,1\} \qquad \forall j \in J \tag{37p}$$

$$\sigma^u_i \in \{0,1\} \qquad \forall i \in I \tag{37q}$$

$$\sigma^v_j \in \{0,1\} \qquad \forall j \in J. \tag{37r}$$

Here, we use $y_{ij} \leq \min(c_i, \bar{d}_j + \hat{d}_j)$ (since $f \geq 0$), $-u_i + v_j - f_{ij} \geq -f_{ij} - \hat{f}$ (since $u_i \leq \hat{f}$) for $(i,j) \in I\times J$,

$h_j \leq (\bar{d}_j + \hat{d}_j)$ (since $\hat{f} \geq 0$), $v_j - \hat{f} \geq -\hat{f}$ (since $v \geq 0$) for $j \in J$, $u_i \leq \hat{f}$, $\sum_{j \in J} y_{ij} - \hat{z}_i(1 - \gamma_i) \geq -\hat{z}_i$ (since $y \geq 0$) for $i \in I$, $v_j \leq \hat{f}$ and $\sum_{i \in I} y_{ij} + h_j - (\bar{d}_j + \hat{d}_j \delta_j) \leq 0$ (since $f, \hat{f} \geq 0$) to calculate the necessary big-M coefficients. We, once again, remark that variables u are artificially bounded by the value $\hat{f}$ without cutting off any optimal solution.

C.3 Network design problem

For the ND problem, the second-stage problem is a feasibility problem by nature. For a given first-stage solution $\hat{x} \in \mathbb{R}_+^{|A|}$, the max-min formulation of the subproblem, which is rendered feasible with the use of artificial variables, is given as:

$$\max_{\substack{\gamma \in B_A^\Gamma \\ \delta \in B_K^\Delta}} \min_{\substack{\alpha \in \mathbb{R}_+ \\ y \in \mathbb{R}_+^{|K| \times |A|}}} \alpha \tag{38a}$$

$$\text{s.t.} \quad \sum_{a \in \sigma_v^-} y_a^k - \sum_{a \in \sigma_v^+} y_a^k = \begin{cases} -\bar{d}_k - \hat{d}_k \delta_k & if\ v = s_k \\ +\bar{d}_k + \hat{d}_k \delta_k & if\ v = t_k \\ 0 & otherwise \end{cases} \qquad \forall (v,k) \in V \times K \tag{38b}$$

$$\sum_{k \in K} y_a^k - \alpha \leq \hat{x}_a (1 - \gamma_a) \qquad \forall a \in A. \tag{38c}$$

We remark that, similar to the FLT problem, we add the artificial variable only to the capacity constraints (38c) which involve the first-stage variable $\hat{x}$.

We next derive the dual set associated with the inner minimization problem of (38) through linear programming duality. Let u and w denote the dual variables associated with constraints (38b) and (38c), respectively. We obtain:

$$\Pi_{\text{norm}}^{\text{ND}} = \left\{ u \in \mathbb{R}^{|K| \times |V|}, w \in \mathbb{R}_+^{|A|} \middle| \sum_{a \in A} w_a \leq 1, u_j^k - u_i^k - w_a \leq 0 \ \ \forall a = (i,j) \in A, k \in K, u_{s_k}^k = 0 \ \ \forall k \in K \right\}.$$

The first and second constraints are the constraints associated to the primal variables α and y, respectively. The third constraint is a valid constraint that eliminates some feasible solutions without cutting off all optimal solutions. Indeed, for a given first-stage solution, an optimal solution can be obtained for any value of $u_{s^k}^k$ for k in K.

Reformulations ($SP\text{-}P\text{-}Dual(\hat{x})$) and ($SP\text{-}P\text{-}KKT(\hat{x})$) of (38) are respectively obtained by applying LP duality and KKT optimality conditions to the inner minimization problem. We next describe the linearized ($SP\text{-}P\text{-}Dual(\hat{x})$) and ($SP\text{-}P\text{-}KKT(\hat{x})$) models.

For the ($SP\text{-}P\text{-}Dual(\hat{x})$) model, we need the extreme points of the uncertainty set to be integer. This is achieved by transforming the budgeted uncertainty set as described in Section C.2. As a result of this transformation of the uncertainty set, uncertain parameters γ and δ will be split into γ^1, γ^2 and δ^1, δ^2, respectively. Dualizing the inner minimization problem in (38) yields the following bilinear problem:

$$\max \quad -\sum_{a \in A} \hat{x}_a w_a (1 - (\gamma_a^1 + (\Gamma - \lfloor \Gamma \rfloor)\gamma_a^2)) + \sum_{k \in K} \bar{d}_l u_{t_k}^k + \hat{d}_k u_{t_k}^k (\delta_k^1 + (\Delta - \lfloor \Delta \rfloor)\delta_k^2) \tag{39a}$$

$$\text{s.t.} \quad (\gamma^1, \gamma^2) \in \Omega_A^\Gamma \cap \left(\{0,1\}^{|A|} \times \{0,1\}^{|A|} \right) \tag{39b}$$

$$(\delta^1, \delta^2) \in \Omega_K^\Delta \cap \left(\{0,1\}^{|K|} \times \{0,1\}^{|K|} \right) \tag{39c}$$

$$(u, w) \in \Pi^{\text{ND}}_{\text{norm}} \tag{39d}$$

We may then, for $l \in \{1, 2\}$, proceed to the linearization of bilinear terms $w \circ \gamma^l$ and $u \circ \delta^l$ that appear in the objective function of the bilinear maximization problem. Let μ and ν be the new variables representing the bilinear products, for a given l in $\{1, 2\}$, *i.e.*, $\mu^l = w \circ \gamma^l \in \mathbb{R}^{|A|}_+$ and $\nu^l_k = u^k_{t_k} \delta^l_k \in \mathbb{R}_+$ for $k \in K$. We remark that only the variables $u^k_{t_k}$ for $k \in K$ are involved in the latter expression since the variables $u^k_{s_k}$ for $k \in K$ are fixed to zero and the right-hand side of constraints (38b) are zero for $k \in K$ and $v \in V \setminus \{s_k, t_k\}$. Further, since the cost coefficients of variables u in the objective function (40a) are non-negative, these variables can be restricted to be non-negative. We obtain the linearized $(SP\text{-}P\text{-}Dual(\hat{x}))$ formulation as:

$$\max \quad -\sum_{a \in A} \hat{x}_a (w_a - \mu^1_a - (\Gamma - \lfloor \Gamma \rfloor)\mu^2_a) + \sum_{k \in K} \bar{d}_l u^k_{t_k} + \hat{d}_k (\nu^1_k + (\Delta - \lfloor \Delta \rfloor)\nu^2_k) \tag{40a}$$

$$\text{s.t.} \quad \mu^l_a \leq w_a \qquad \forall l \in \{1, 2\}, a \in A \tag{40b}$$

$$\mu^l_a \leq \gamma^l_a \qquad \forall l \in \{1, 2\}, a \in A \tag{40c}$$

$$\nu^l_k \leq u^k_{t_k} \qquad \forall l \in \{1, 2\}, k \in K \tag{40d}$$

$$\nu^l_k \leq \delta^l_k \qquad \forall l \in \{1, 2\}, k \in K \tag{40e}$$

$$(\gamma^1, \gamma^2) \in \Omega^\Gamma_A \cap \left(\{0, 1\}^{|A|} \times \{0, 1\}^{|A|} \right) \tag{40f}$$

$$(\delta^1, \delta^2) \in \Omega^\Delta_K \cap \left(\{0, 1\}^{|K|} \times \{0, 1\}^{|K|} \right) \tag{40g}$$

$$(u, w) \in \Pi^{\text{ND}}_{\text{norm}} \tag{40h}$$

$$\mu^1, \mu^2 \in \mathbb{R}^{|A|}_+ \tag{40i}$$

$$\nu^1, \nu^2 \in \mathbb{R}^{|K|}_+, \tag{40j}$$

where we use $w_a \leq 1$ for $a \in A$ and $u^k_{t_k} \leq \sum_{a \in A} w_a \leq 1$ to calculate the necessary big-M coefficients.

For the $(SP\text{-}P\text{-}KKT(\hat{x}))$ model, obtained by applying the KKT conditions to the inner minimization problem of (38), the complementary slackness conditions take the form $\alpha \left(\sum_{a \in A} w_a - 1 \right) = 0$, $y^k_a \left(u^k_i - u^k_j - w_a \right) = 0$ for $a \in A$, $k \in K$, and $w_a \left(\sum_{k \in K} y^k_a - \alpha - \hat{x}_a (1 - \gamma_a) \right) = 0$. To linearize these conditions, we introduce the binary variables $\sigma^\alpha \in \{0, 1\}$, $\sigma^y_{ak} \in \{0, 1\}^{|A| \times |K|}$, and $\sigma^w_a \in \{0, 1\}^{|A|}$. We then obtain the linearized $(SP\text{-}P\text{-}KKT(\hat{x}))$ formulation as:

$$\max \quad \alpha \tag{41a}$$

$$\text{s.t.} \quad (38\text{b}) - (38\text{c}) \tag{41b}$$

$$(u, w) \in \Pi^{\text{ND}}_{\text{norm}} \tag{41c}$$

$$\sum_{a \in A} w_a \geq 1 + (\sigma^\alpha - 1) \tag{41d}$$

$$\alpha \leq M_\alpha \sigma^\alpha \tag{41e}$$

$$u^k_i - u^k_j - w_a \geq 0 + 2(\sigma^y_{ak} - 1) \qquad \forall a = (i, j) \in A, k \in K \tag{41f}$$

$$y^k_a \leq (\bar{d}_k + \hat{d}_k)\sigma^y_{ak} \qquad \forall a = (i, j) \in A, k \in K \tag{41g}$$

$$\sum_{k \in K} y^k_a - \alpha \geq \hat{x}_a (1 - \gamma_a) + (M_\alpha + \hat{x}_a)(\sigma^w_a - 1) \qquad \forall a \in A \tag{41h}$$

$$w_a \leq \sigma_a^w \qquad \forall a \in A \tag{41i}$$
$$\gamma \in B_A^\Gamma \tag{41j}$$
$$\delta \in B_K^\Delta \tag{41k}$$
$$\sigma^\alpha \in \{0,1\} \tag{41l}$$
$$\sigma_{ak}^y \in \{0,1\}^{|A|\times|K|} \tag{41m}$$
$$\sigma_a^w \in \{0,1\}^{|A|}, \tag{41n}$$

where we used $\alpha \leq M_\alpha := \max_{\delta \in B_K^\delta} \left\{ \sum_{k\in K} \bar{d}_k + \hat{d}_k \delta_k \right\}$, $\sum_{a\in A} w_a - 1 \geq -1$ (since $w \geq 0$), $y_a^k \leq \bar{d}_k + \hat{d}_k$, $u_i^k - u_j^k - w_a \geq -2$ (since $u_j \leq \sum_{a\in A} w_a \leq 1$) for $a = (i,j) \in A$ and $k \in K$, $w_a \leq 1$, and $\sum_{k\in K} y_a^k - \alpha - \hat{x}_a(1-\gamma_a) \geq -M_\alpha - \hat{x}_a$ (since $y \geq 0$) for $a \in A$ to calculate the necessary big-M coefficients. We remark that variables y_a^k are originally bounded by $\hat{x}_a(1-\gamma_a)$ in formulation (38). However, they can further be bounded by $\bar{d}_k + \hat{d}_k$ without changing the optimal value of the inner minimization problem in (38).

C.4 Handling numerical instabilities

To address the numerical instabilities caused by big-M coefficients in the subproblem, we implement a two-phase procedure. In the first phase of this procedure, we solve the subproblem, $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$, and recover its optimal solution $\hat{\xi} \in \Xi$. We then check whether the first-stage solution $\hat{x} \in \mathcal{X}$ is feasible for $\hat{\xi}$ by verifying whether the set $\{y \in \mathbb{R}_+^{n_y} | Wy \geq h(\hat{\xi}) - T(\hat{\xi})\hat{x}\}$ is empty. If this is the case, then the realization $\hat{\xi}$ is identified as a violated realization and corresponding constraints are added to the master problem. Otherwise, there might exist another realization, $\xi' \in \Xi$, that is violated but whose value in $(SP\text{-}P(\hat{x}, \Xi, \Pi_{\text{norm}}))$ is smaller than that of $\hat{\xi}$. In this case, the algorithm stops with a conclusion of numerical instability. A similar procedure was also applied in the case of optimality problems in the second stage. As a result, among the 12960 runs, 51 ended with a conclusion of numerical instability. All of the corresponding runs were from the network design application and are considered to be not solved to optimality in the reported numerical results.

C.5 Comparing subproblem reformulations

In this subsection, we numerically compare the subproblem reformulations for the FLT and ND problems. To this end, using the same experimental environment as described in Section 3.2, we test the C&CG algorithm with each subproblem formulation presented in Sections C.2 and C.3 as well as the Gurobi MINLP solver as its separation oracle.

Instance generation For the FLT problem, to test the formulations presented in Section C.2, we randomly generate 80 instances with n facilities and n customers as described in (Zeng and Zhao, 2013; Gabrel et al., 2014a). We vary n in $\{10, 20\}$, and denote these instances as FLT-10 and FLT-20. We remark that the FLT-20 instances are the same as those used in Section 3. Similar to Section 3, we vary Δ and Γ in $\{\lfloor \frac{n}{3} \rfloor, \lfloor \frac{n}{3} \rfloor + 0.5\}$ and $\{0, \lfloor \frac{n}{5} \rfloor, \lfloor \frac{n}{5} \rfloor + 0.5\}$, respectively, to obtain a total of 480 instances per instance size (n).

For the ND problem, to test the formulations presented in Section C.3, we create a smaller graph by restricting the giul_39 instance (Orlowski et al., 2010) to its first five nodes, denoted giul_5 in the following. We consider $|K| \in \{1, 2, 3, 4, 5\}$ commodities with the largest nominal demand, and set

the deviation $\hat{d}$ to $0.4\bar{d}$. We let $\Delta = 1$ and vary $\Gamma \in \{0, 1, 1.5\}$ to obtain a total of 15 instances per formulation.

Results The results are provided in Tables 5, 6, and 7 as an average over all instances of the same size. In each table, #Opt and #Err refer to the number of instances solved to optimality and the number of instances subject to numerical instabilities, respectively. Time refers to the solution time in seconds. %SP and #SP refer to the percentage of time spent solving the separation problems and the number of times they are solved, respectively. In these tables, (SP-P-$Bil(\hat{x})$) refers to subproblem formulations (32) and (39) for problems FLT and ND, respectively, and (SP-Opt-$Bil(\hat{x})$) stands for formulation (34). These formulations are solved using the MINLP solver of Gurobi. To further study the performance of the dualization-based formulations versus Gurobi's MINLP solver, we extended the comparison to the larger instances. For FLT-30 instances, (SP-Opt-$Bil(\hat{x})$) (254/480 instances solved) performs slightly better than (SP-Opt-$Dual(\hat{x})$) (245/480 instances solved). Over the whole set of ND instances, (SP-P-$Dual(\hat{x})$) (346/480 instances solved) significantly outperforms (SP-P-$Bil(\hat{x})$) (280/480 instances solved). It is interesting to note that in the absence of fine-tuned big-M parameters the MINLP solver might be the best option for those problems in which the uncertain parameters can be assumed to be binary/integer.

Formulation	#Opt	#Err	Time	%SP	#SP
(SP-P-$Dual(\hat{x})$)	480	0	4.0	64.1	7.4
(SP-P-$Bil(\hat{x})$)	480	0	6.3	66.2	7.4
(SP-P-$KKT(\hat{x})$)	52	107	7507.1	99.9	3.6
(SP-Opt-$Dual(\hat{x})$)	480	0	1.5	48.6	7.4
(SP-Opt-$Bil(\hat{x})$)	480	0	1.3	41.7	7.4
(SP-Opt-$KKT(\hat{x})$)	478	2	12.0	88.1	7.4

Table 5: Comparison of the C&CG algorithm with different subproblem formulations on FLT-10 instances.

Formulation	#Opt	#Err	Time	%SP	#SP
(SP-P-$Dual(\hat{x})$)	377	0	3738.5	97.3	12.7
(SP-P-$Bil(\hat{x})$)	298	0	5158.0	97.3	10.8
(SP-P-$KKT(\hat{x})$)	0	0	10800.1	100.0	2.0
(SP-Opt-$Dual(\hat{x})$)	480	0	323.3	89.7	14.4
(SP-Opt-$Bil(\hat{x})$)	480	0	258.1	87.4	14.4
(SP-Opt-$KKT(\hat{x})$)	190	10	6731.5	99.8	7.0

Table 6: Comparison of the C&CG algorithm with different subproblem formulations on FLT-20 instances.

According to Tables 5 and 6, the C&CG algorithm integrating the formulation (SP-P-$KKT(\hat{x})$) as its separation oracle performs the worst for the FLT problem. Indeed, this algorithm was not able to solve some of the FLT-10 instances. Further, subproblems could not be solved for the FLT-20 instances with this formulation. On the other hand, among all other versions of the algorithm, the one integrating the formulation (SP-Opt-$Dual(\hat{x})$) as its separation oracle performs the best. We further remark that, for each method of reformulating the inner minimization problem (LP duality and KKT optimality conditions), the subproblem formulations resulting from optimality problems at the second stage are solved faster than those resulting from feasibility problems at the second stage. Indeed, by writing:

$$\Pi_N := \left\{ \pi \in \Pi \;\middle|\; N^\top \pi \leq \mathbf{1} \right\}$$

$$
\begin{aligned}
&= \left\{ \pi \in \mathbb{R}_+^{m_y} \;\middle|\; W^\top \pi \leq \mathbf{0}, N^\top \pi \leq \mathbf{1} \right\} \\
&= \left\{ \pi \in \mathbb{R}_+^{m_y} \;\middle|\; \begin{pmatrix} -f^\top \\ \bar{W} \end{pmatrix}^\top \pi \leq \mathbf{0}, N^\top \pi \leq \mathbf{1} \right\} \\
&= \left\{ \bar{\pi} \in \mathbb{R}_+^{m_y - 1},\ \kappa \in \mathbb{R}_+ \;\middle|\; \bar{W}^\top \bar{\pi} \leq f\kappa, N^\top \begin{pmatrix} \kappa \\ \bar{\pi} \end{pmatrix} \leq \mathbf{1} \right\},
\end{aligned}
$$

we can show that $\bar{\Pi}_{\text{opt}} := \{\bar{\pi} \in \mathbb{R}_+^{m_y - 1} \mid \bar{W}^\top \bar{\pi} \leq f\}$, the dual set resulting from optimality problems at the second stage, corresponds to the dual set Π_N, resulting from feasibility problems at the second stage, with the variable κ fixed to 1 and the constraint $N^\top \begin{pmatrix} \kappa \\ \bar{\pi} \end{pmatrix} \leq \mathbf{1}$ removed. Tables 5 and 6 also show that reformulating the bilinear subproblem using LP duality and the ensuing linearization is more efficient than the reformulation derived from KKT optimality conditions.

For the numerical results presented in Section 3 for the FLT problem, we thus use formulation $(SP\text{-}Opt\text{-}Dual(\hat{x}))$ as the separation oracle in each algorithm.

Formulation	#Opt	#Err	Time	%SP	#SP
$(SP\text{-}P\text{-}Dual(\hat{x}))$	15	0	0.7	16.2	8.7
$(SP\text{-}P\text{-}Bil(\hat{x}))$	15	0	0.8	21.5	8.7
$(SP\text{-}P\text{-}KKT(\hat{x}))$	7	0	5990.2	98.4	3.9

Table 7: Comparison of the C&CG algorithm with different subproblem formulations on instance giul_5.

For the ND problem, Table 7 shows that the $(SP\text{-}P\text{-}Dual(\hat{x}))$ formulation performs significantly better than the $(SP\text{-}P\text{-}KKT(\hat{x}))$ formulation. Indeed, the C&CG algorithm solved all instances in less than one second on average using the $(SP\text{-}P\text{-}Dual(\hat{x}))$ formulation as its separation oracle, while it could not solve some of these instances using the $(SP\text{-}P\text{-}KKT(\hat{x}))$ formulation.

For the numerical results presented in Section 3 for the ND problem, we thus use the $(SP\text{-}P\text{-}Dual(\hat{x}))$ formulation as the separation oracle in each algorithm.

D Optimality problems at the second stage

Throughout this paper, we have primarily focused on formulation $(\bar{P})$ of adjustable robust optimization problems, which writes the second-stage problem as a feasibility problem. This formulation allowed us to treat the questions of optimality and feasibility of a given first-stage solution $\hat{x} \in \mathcal{X}$ in a unified manner. On the other hand, under this framework, the decomposition algorithms presented in Section 2.1 do not produce primal bounds since their subproblems evaluate only the worst-case feasibility of $\hat{x} \in \mathcal{X}$ and not its worst-case objective value. This might be a drawback for adjustable robust optimization problems that are naturally formulated with an optimality problem at the second stage. Indeed, the ability to calculate primal bounds within these algorithms leads to optimality gaps that can be used as an early termination criterion, which can be very useful in practice. Further, if the algorithms need to be stopped due to solution time limits, then they can provide a primal solution with an optimality gap guarantee, provided that a feasible solution has been found within the imposed time limitations. We, therefore, consider the following variant:

$$
\min_{\bar{x} \in \bar{\mathcal{X}}} \bar{c}^\top \bar{x} + \max_{\xi \in \Xi} \min_{y \in \mathbb{R}_+^{n_y}} f^\top y \tag{$\bar{Q}$}
$$

$$\text{s.t. } \bar{W}y \geq \bar{h}(\xi) - \bar{T}(\xi)\bar{x}, \tag{42}$$

where the second-stage problem is written as an optimization problem. We next detail the extension of algorithms presented in Section 2.1 to this case.

Similar to Section 2.1, we start with a reformulation of $(\bar{Q})$ from which the decomposition algorithms will be derived. Using $\theta \in \mathbb{R}$ to represent the second-stage cost, we obtain the following nonlinear monolithic reformulation:

$$\min_{\bar{x}\in\bar{\mathcal{X}},\theta\in\mathbb{R}} \ \bar{c}^\top \bar{x} + \theta \tag{Q}$$

$$\text{s.t.} \quad \max_{\xi\in\Xi} \ \min_{\substack{y\in\mathbb{R}^{n_y}_+ \\ \bar{W}y\geq\bar{h}(\xi)-\bar{T}(\xi)\bar{x}}} \ 0 \leq 0 \tag{43}$$

$$\max_{\xi\in\Xi} \ \min_{\substack{y\in\mathbb{R}^{n_y}_+ \\ \bar{W}y\geq\bar{h}(\xi)-\bar{T}(\xi)\bar{x}}} \ f^\top y \leq \theta. \tag{44}$$

Nonlinear constraints (43) and (44) can be reformulated using an exponential number of linear constraints (CG), an exponential number of linear constraints and variables (C&CG), or an exponential number of semi-infinite linear constraints (SICG). Decomposition algorithms based on these reformulations will then solve and reinforce the corresponding master relaxations in which not all constraints (and variables) are present. We next present how a first-stage solution, $(\hat{x}, \hat{\theta}) \in \bar{\mathcal{X}} \times \mathbb{R}$, is separated before introducing the linear reformulation of, and the information added to, the master problem for each algorithm.

To certify the feasibility and optimality of a first-stage solution obtained from a relaxation, decomposition algorithms will follow two successive phases, referred to as feasibility and optimality phases, corresponding to separation of constraints (43) and (44). Formally, for a given first-stage solution $(\hat{x}, \hat{\theta}) \in \bar{\mathcal{X}}\times\mathbb{R}$, the feasibility phase checks whether the second-stage problem is feasible for each realization in the uncertainty set by solving:

$$z_{\text{SP}-\text{Feas}}(\hat{x}) := \max_{\xi\in\Xi} \ \max_{\bar{\pi}\in\bar{\Pi}_{\text{feas}}} \ \bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\hat{x}\right), \tag{$SP\text{-}Feas(\hat{x}, \Xi, \bar{\Pi}_{\text{feas}})$}$$

where $\bar{\Pi}_{\text{feas}} := \{\bar{\pi} \in \mathbb{R}^{m_y-1}_+ \mid \bar{W}^\top\bar{\pi} \leq \mathbf{0}, \mathbf{1}^\top\bar{\pi} \leq 1\}$ through linear programming duality. We remark that, similar to Section 2.1.1, the ℓ_1-norm of the dual variables $\bar{\pi}$ in $\bar{\Pi}_{\text{feas}}$ is bounded to obtain a bounded maximization problem. If $z_{\text{SP}-\text{Feas}}(\hat{x}) > 0$, then the solution $\hat{x}$ is not feasible, and the master problem should be reinforced to cut off the solution $(\hat{x}, \hat{\theta})$. Otherwise ($z_{\text{SP}-\text{Feas}}(\hat{x}) = 0$), the feasibility of the first-stage solution $\hat{x}$ is established, and we can proceed to the optimality phase to calculate its worst-case objective value. To do so, we need to compute the value of the left-hand side of constraint (44). We begin by verifying that the inner minimization problem is bounded for $x \in \mathcal{X}$ and $\xi \in \Xi$. This can be achieved by checking that the dual polyhedron $\bar{\Pi}_{\text{opt}} := \{\bar{\pi} \in \mathbb{R}^{m_y-1}_+ \mid \bar{W}^\top\bar{\pi} \leq f\}$ is non-empty. We remark that, since $\bar{\Pi}_{\text{opt}}$ does not depend on x and ξ, this operation can be done in a preprocessing step (in practice, one can optimize any linear function over this set with an LP solver if a trivial feasible solution is not available). If the set $\bar{\Pi}_{\text{opt}}$ is empty, then problem $(\bar{Q})$ is either unbounded or infeasible and, as such, does not have an optimal solution. We, therefore, assume that $\bar{\Pi}_{\text{opt}}$ is not empty and use linear programming

duality to obtain the following bilinear formulation:

$$z_{\text{SP}-\text{Opt}}(\hat{x}) := \max_{\xi \in \Xi} \max_{\bar{\pi} \in \bar{\Pi}_{\text{opt}}} \bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\hat{x}\right). \qquad (SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$$

This problem is bounded since it is solved only when $z_{\text{SP}-\text{Feas}}(\hat{x}) = 0$. As such, its value $\bar{c}^\top \hat{x} + z_{\text{SP}-\text{Opt}}(\hat{x})$ can be established as a primal bound on the optimal value of problem (Q) since it corresponds to the worst-case value of the feasible solution $\hat{x}$. If $z_{\text{SP}-\text{Opt}}(\hat{x}) > \hat{\theta}$, then the worst-case value of solution $\hat{x}$ is not correctly estimated, so the master problem should be reinforced to cut off the solution $(\hat{x}, \hat{\theta})$. As with formulation (P), a first-stage solution $(\hat{x}, \hat{\theta})$ is proven to be feasible and optimal when there are no more violated constraints, *i.e.*, $z_{\text{SP}-\text{Feas}}(\hat{x}) = 0$ and $z_{\text{SP}-\text{Opt}}(\hat{x}) = \hat{\theta}$ at which point the algorithms stop. Thanks to the calculated primal bound, it is also possible to stop each algorithm, albeit with a potentially suboptimal solution, before this convergence criterion is reached if an acceptable relative optimality gap is achieved.

D.1 Algorithmic development

We next detail how the high-level description above can be adapted to the CG, C&CG and SICG algorithms to handle optimization problems at the second stage. To do so, for each algorithm, we provide a reformulation of problem (Q) on which the algorithm will be based. We then specify how the master relaxations based on these reformulations should be reinforced to cut off first-stage solutions $(\hat{x}, \hat{\theta})$. Further, in the context of the SICG algorithm, we detail how the augmented constraint generation algorithm (Algorithm 2) and the mountain climbing algorithm (Algorithm 7) can be adapted to formulation (Q).

For the constraint generation (CG) algorithm, using the definition of $z_{\text{SP}-\text{Feas}}$ and z_{opt} restricted to the extreme points of $\bar{\Pi}_{\text{feas}}$ and $\bar{\Pi}_{\text{opt}}$ inside the formulation (Q), we obtain the linear reformulation:

$$\min_{\bar{x} \in \bar{\mathcal{X}}, \theta \in \mathbb{R}} c^\top \bar{x} + \theta \qquad (Q\text{-}CG)$$

$$\text{s.t.} \quad \bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\bar{x}\right) \leq 0 \qquad \forall (\xi, \bar{\pi}) \in \text{ext}(\Xi) \times \text{ext}(\bar{\Pi}_{\text{feas}}) \qquad (45)$$

$$\bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\bar{x}\right) \leq \theta \qquad \forall (\xi, \bar{\pi}) \in \text{ext}(\Xi) \times \text{ext}(\bar{\Pi}_{\text{opt}}). \qquad (46)$$

The CG algorithm (Algorithm 8) is based on a relaxation of $(Q\text{-}CG)$ defined by restricting the set of constraints (45) and (46) to $\tilde{\Psi}_{\text{feas}} \subseteq \text{ext}(\Xi) \times \text{ext}(\bar{\Pi}_{\text{feas}})$ and $\tilde{\Psi}_{\text{opt}} \subseteq \text{ext}(\Xi) \times \text{ext}(\bar{\Pi}_{\text{opt}})$, respectively:

$$\min_{\substack{\bar{x} \in \bar{\mathcal{X}} \\ \theta \in \mathbb{R}}} c^\top \bar{x} + \theta \qquad (MQ\text{-}CG(\tilde{\Psi}_{\text{feas}}, \tilde{\Psi}_{\text{opt}}))$$

$$\text{s.t.} \quad \bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\bar{x}\right) \leq 0 \qquad \forall (\xi, \bar{\pi}) \in \tilde{\Psi}_{\text{feas}} \qquad (47)$$

$$\bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\bar{x}\right) \leq \theta \qquad \forall (\xi, \bar{\pi}) \in \tilde{\Psi}_{\text{opt}}. \qquad (48)$$

For a given first-stage solution, $(\hat{x}, \hat{\theta})$, of this relaxation, if $z_{\text{SP}-\text{Feas}}(\hat{x}) > 0$, then a constraint of the form (45) should be added to the master problem with $(\xi, \bar{\pi})$ an optimal solution of $(SP\text{-}Feas(\hat{x}, \Xi, \bar{\Pi}_{\text{feas}}))$. Otherwise, if $z_{\text{SP}-\text{Opt}}(\hat{x}) > \hat{\theta}$ then a constraint of the form (46) is added with $(\xi, \bar{\pi})$ an optimal solution of $(SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$. These are analogous, respectively, to the classical feasibility and optimality cuts from the Benders' decomposition literature.

In Algorithm 8, we use the notation lb and ub to store the lower and upper bound values on the objective value of problem (Q), respectively. We also define the stopping criteria based on the absolute

Algorithm 8: Constraint Generation (CG) algorithm for (Q)

lb $\leftarrow -\infty$, ub $\leftarrow \infty$, $x^* \leftarrow$ null, $\theta^* \leftarrow$ null, $\tilde{\Psi}_{\text{feas}} \leftarrow \emptyset$, $\tilde{\Psi}_{\text{opt}} \leftarrow \emptyset$
if $\bar{\Pi}_{\text{opt}} = \emptyset$ **then**
  return (Q) *is infeasible or unbounded*
end
while true **do**
  Solve $(MQ\text{-}CG(\tilde{\Psi}_{\text{feas}}, \tilde{\Psi}_{\text{opt}}))$
  if $(MQ\text{-}CG(\tilde{\Psi}_{\text{feas}}, \tilde{\Psi}_{\text{opt}}))$ *is feasible* **then**
    Let $(\hat{x}, \hat{\theta})$ be an optimal solution of $(MQ\text{-}CG(\tilde{\Psi}_{\text{feas}}, \tilde{\Psi}_{\text{opt}}))$
    lb $\leftarrow c^\top \hat{x} + \hat{\theta}$
    Let $(\xi^*, \bar{\pi}^*)$ be an optimal solution of $(SP\text{-}Feas(\hat{x}, \Xi, \bar{\Pi}_{\text{feas}}))$
    if $\bar{\pi}^{*\top} \left(\bar{h}(\xi^*) - \bar{T}(\xi^*)\hat{x}\right) > 0$ **then**
      $\tilde{\Psi}_{\text{feas}} \leftarrow \tilde{\Psi}_{\text{feas}} \cup \{(\xi^*, \bar{\pi}^*)\}$
    else
      Let $(\xi^*, \bar{\pi}^*)$ be an optimal solution of $(SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$
      if $c^\top \hat{x} + \bar{\pi}^{*\top} \left(\bar{h}(\xi^*) - \bar{T}(\xi^*)\hat{x}\right) <$ ub **then**
        ub $\leftarrow c^\top \hat{x} + \bar{\pi}^{*\top} \left(\bar{h}(\xi^*) - \bar{T}(\xi^*)\hat{x}\right)$
        $x^* \leftarrow \hat{x}$, $\theta^* \leftarrow \hat{\theta}$
      end
      if $|\text{ub} - \text{lb}| > \jmath$ *and* $\frac{|\text{ub}-\text{lb}|}{|\text{ub}|+\text{fff}} > \zeta$ **then**
        $\tilde{\Psi}_{\text{opt}} \leftarrow \tilde{\Psi}_{\text{opt}} \cup \{(\xi^*, \bar{\pi}^*)\}$
      else
        return *An optimal solution of* (Q) *:* (x^*, θ^*)
      end
    end
  else
    return (Q) *is infeasible*
  end
end

and relative optimality gap tolerances denoted by $\eta \in \mathbb{R}_+$ and $\zeta \in \mathbb{R}_+$, respectively. In calculating relative optimality gaps, we use the parameter $\epsilon > 0$, which is a very small value, to avoid divisions by zero.

For the constraint-and-column generation (C&CG) algorithm, the reformulation for problem (Q) is the same as that for problem (P):

$$\min_{\substack{\bar{x}\in\bar{\mathcal{X}},\theta\in\mathbb{R} \\ y\in\mathbb{R}_+^{|\text{ext}(\Xi)|\times n_y}}} \quad c^\top \bar{x} + \theta \qquad (Q\text{-}C\&CG)$$

$$\text{s.t.} \quad \bar{W} y^\xi \geq \bar{h}(\xi) - \bar{T}(\xi)\bar{x} \qquad \forall \xi \in \text{ext}(\Xi) \qquad (49)$$

$$-f^\top y^\xi \geq -\theta \qquad \forall \xi \in \text{ext}(\Xi). \qquad (50)$$

The C&CG algorithm (Algorithm 9) is based on a relaxation of $(Q\text{-}C\&CG)$ defined by restricting the set of constraints (49) and (50) to $\tilde{\Xi} \subseteq \text{ext}(\Xi)$:

$$\min_{\substack{\bar{x}\in\bar{\mathcal{X}} \\ \theta\in\mathbb{R} \\ y\in\mathbb{R}_+^{|\tilde{\Xi}|\times n_y}}} \quad c^\top x \qquad (MQ\text{-}C\&CG(\tilde{\Xi}))$$

$$\text{s.t.} \quad \bar{W} y^{\xi} \geq \bar{h}(\xi) - \bar{T}(\xi)\bar{x} \qquad \forall \xi \in \tilde{\Xi} \tag{51}$$

$$-f^{\top} y^{\xi} \geq -\theta \qquad \forall \xi \in \tilde{\Xi}. \tag{52}$$

Algorithm 9: Constraint-and-Column Generation (C&CG) algorithm for (Q)

```
lb ← −∞, ub ← ∞, x* ← null, θ* ← null, Ξ̃ ← ∅
if Π̄_opt = ∅ then
    return (Q) is infeasible or unbounded
end
while true do
    Solve (MQ-C&CG(Ξ̃))
    if (MQ-C&CG(Ξ̃)) is feasible then
        Let (x̂, θ̂) be an optimal solution of (MQ-C&CG(Ξ̃))
        lb ← c^⊤ x̂ + θ̂
        Let (ξ*, π̄*) be an optimal solution of (SP-Feas(x̂, Ξ, Π̄_feas))
        if π̄*^⊤ (h̄(ξ*) − T̄(ξ*)x̂) > 0 then
            Ξ̃ ← Ξ̃ ∪ {ξ*}
        else
            Let (ξ*, π̄*) be an optimal solution of (SP-Opt(x̂, Ξ, Π̄_opt))
            if c^⊤ x̂ + π̄*^⊤ (h̄(ξ*) − T̄(ξ*)x̂) < ub then
                ub ← c^⊤ x̂ + π̄*^⊤ (h̄(ξ*) − T̄(ξ*)x̂)
                x* ← x̂, θ* ← θ̂
            end
            if |ub − lb|> ȷ and |ub−lb| / (|ub|+ffl) > ζ then
                Ξ̃ ← Ξ̃ ∪ {ξ*}
            else
                return An optimal solution of (Q) : (x*, θ*)
            end
        end
    else
        return (Q) is infeasible
    end
end
```

For a given first-stage solution $(\hat{x}, \hat{\theta})$, the relaxation is augmented with the same constraints and variables regardless of whether $z_{\text{SP}-\text{Feas}}(\hat{x}) > 0$ or $z_{\text{SP}-\text{Opt}}(\hat{x}) > \hat{\theta}$. Let $\xi \in \Xi$ such that $(\xi, \bar{\pi})$ is an optimal solution to $(SP\text{-}Feas(\hat{x}, \Xi, \bar{\Pi}_{\text{feas}}))$ or $(SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$ in each case. Then, a new second-stage variable $y^{\xi} \in \mathbb{R}^{n_y}_{+}$ is created and the constraints (49) and (50) are added to the master relaxation for this ξ. In the first case, although adding only (49) would yield a valid algorithm, it is standard to add (50) as well, since it is a single constraint and prevents identifying ξ as a violated realization in a later iteration (Zeng and Zhao, 2013).

For the semi-infinite constraint generation (SICG) algorithm, a linear reformulation can be obtained from $(Q\text{-}CG)$ by considering $\bar{\Pi}^{\xi}_{\text{feas}}$ and $\bar{\Pi}^{\xi}_{\text{opt}}$ instead of their extreme points:

$$\min_{\bar{x} \in \bar{\mathcal{X}}, \theta \in \mathbb{R}} \; c^{\top}\bar{x} + \theta \tag{Q-SICG}$$

$$\text{s.t.} \;\; \bar{\pi}^{\top}\left(\bar{h}(\xi) - \bar{T}(\xi)\bar{x}\right) \leq 0 \qquad \forall \xi \in \text{ext}(\Xi), \bar{\pi} \in \bar{\Pi}^{\xi}_{\text{feas}} \tag{53}$$

$$\bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\bar{x}\right) \leq \theta \qquad \forall \xi \in \text{ext}(\Xi), \bar{\pi} \in \bar{\Pi}^\xi_{\text{opt}}. \tag{54}$$

Formulation (*Q-SICG*) is comprised of two sets of exponentially many semi-infinite constraints, which can be interpreted as semi-infinite feasibility and optimality cuts. The SICG algorithm (Algorithm 10) is then based on a relaxation of this reformulation where we consider only $\tilde{\Xi} \subseteq \text{ext}(\Xi)$. We further treat the resulting semi-infinite constraints through a cut generation approach by letting $\tilde{\Pi}_{\text{feas}} := (\tilde{\Pi}^\xi_{\text{feas}})_{\xi \in \tilde{\Xi}}$ (resp. $\tilde{\Pi}_{\text{opt}} := (\tilde{\Pi}^\xi_{\text{opt}})_{\xi \in \tilde{\Xi}}$) with $\tilde{\Pi}^\xi_{\text{feas}} \subseteq \bar{\Pi}^\xi_{\text{feas}}$ (resp. $\tilde{\Pi}^\xi_{\text{opt}} \subseteq \bar{\Pi}^\xi_{\text{opt}}$) for all $\xi \in \tilde{\Xi}$, and write the master relaxation of the SICG algorithm:

$$\begin{aligned}
\min_{\substack{\bar{x} \in \bar{\mathcal{X}} \\ \theta \in \mathbb{R}}} \quad & c^\top \bar{x} + \theta && (MQ\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}})) \\
\text{s.t.} \quad & \bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\bar{x}\right) \leq 0 && \forall \xi \in \tilde{\Xi}, \bar{\pi} \in \tilde{\Pi}^\xi_{\text{feas}} && (55) \\
& \bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\bar{x}\right) \leq \theta && \forall \xi \in \tilde{\Xi}, \bar{\pi} \in \tilde{\Pi}^\xi_{\text{opt}}. && (56)
\end{aligned}$$

Let $(\hat{x}, \hat{\theta})$ be an optimal solution of this relaxation. Then both in the cases where $z_{\text{SP}-\text{Feas}}(\hat{x}) > 0$ and $z_{\text{SP}-\text{Opt}}(\hat{x}) > \hat{\theta}$, we augment the master relaxation with the semi-infinite constraints (55)-(56) corresponding to $\xi \in \Xi$ such that $(\xi, \bar{\pi})$ is an optimal solution of either (*SP-Feas*$(\hat{x}, \Xi, \bar{\Pi}_{\text{feas}})$) or (*SP-Opt*$(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}})$). In the first case, we further initialize the set $\tilde{\Pi}^\xi_{\text{feas}}$ with $\bar{\pi} \in \bar{\Pi}_{\text{feas}}$ while in the second case we do so for the set $\tilde{\Pi}^\xi_{\text{opt}}$ with $\bar{\pi} \in \bar{\Pi}_{\text{opt}}$ (while $\tilde{\Pi}^\xi_{\text{opt}} = \emptyset$ and $\tilde{\Pi}^\xi_{\text{feas}} = \emptyset$, respectively). We remark that adding a constraint of the form (55) or (56) with either $\tilde{\Pi}^\xi_{\text{feas}} = \emptyset$ or $\tilde{\Pi}^\xi_{\text{opt}} = \emptyset$ does not have an immediate effect on the master relaxation. However, this allows us to generate new violated constraints using the augmented constraint generation algorithm and to avoid identifying ξ as a violated realization in a later iteration, similar to what was done for the C&CG algorithm.

We next detail how the augmented constraint generation algorithm presented in Algorithm 2 can be adapted to formulation (*Q*). To this end, consider a first-stage solution $\hat{x} \in \bar{\mathcal{X}}$ and a generated realization $\hat{\xi} \in \tilde{\Xi}$. Since both constraints (55) and (56) are present for each generated realization, the cut generation phase of Algorithm 2 would naturally proceed to solving (*SP-Feas*$(\hat{x}, \{\hat{\xi}\}, \bar{\Pi}_{\text{feas}})$) as well as (*SP-Opt*$(\hat{x}, \{\hat{\xi}\}, \bar{\Pi}_{\text{opt}})$). However, if $\left\{y \in \mathbb{R}^{n_y}_+ \mid \bar{W}y \geq \bar{h}(\hat{\xi}) - \bar{T}(\hat{\xi})\hat{x}\right\} = \emptyset$, *i.e.*, there is no feasible recourse solution corresponding to $\hat{x}$ and $\hat{\xi}$, then the latter optimization problem is unbounded. Therefore, to avoid such a situation, before separating an optimality cut for a given solution $\hat{x}$ and generated realization $\hat{\xi}$, we first test the feasibility of the second-stage problem using (*SP-Feas*$(\hat{x}, \{\hat{\xi}\}, \bar{\Pi}_{\text{feas}})$), similarly to the classical Benders approach. If its value is strictly positive, then a feasibility cut should be added after a cut selection phase. Otherwise, we proceed to solving (*SP-Opt*$(\hat{x}, \{\hat{\xi}\}, \bar{\Pi}_{\text{opt}})$) which is now guaranteed to be bounded. If its value is greater than $\hat{\theta}$, then an optimality cut should be added after a cut selection phase. In both cases, for each $\bar{\pi} \in \bar{\Pi}_{\text{feas}} \cup \bar{\Pi}_{\text{opt}}$, the cut selection phase solves the problem $\max_{\xi \in \Xi} \bar{\pi}^\top \left(\bar{h}(\xi) - \bar{T}(\xi)\hat{x}\right)$ which is a bounded linear program since Ξ is assumed to be a polytope (Assumption 1 (b)). We remark that this understanding can also be extended to the mountain climbing algorithm presented in Algorithm 7. In each iteration of this algorithm, whenever a new uncertainty realization ξ^* is encountered, one should first verify whether it leads to a violated feasibility cut for $\hat{x}$ before proceeding to separate an optimality cut. The mountain climbing algorithm, adapted in this manner, can then be incorporated into the augmented constraint generation algorithm (as shown in Algorithm 11). Finally, we address the management of the pool expansion phase of our algorithm. Indeed, regardless of whether a feasibility or an optimality cut is added as a result of the cut generation phase, we add the associated uncertainty realization to $\tilde{\Xi}_{new}$, the effect of which is creating

Algorithm 10: Semi-Infinite Constraint Generation (SICG) algorithm for (Q)

lb $\leftarrow -\infty$, ub $\leftarrow \infty$, $x^* \leftarrow$ null, $\theta^* \leftarrow$ null, $\tilde{\Xi} \leftarrow \emptyset$, $\tilde{\Pi}_{\text{feas}} := (\tilde{\Pi}^{\xi}_{\text{feas}})_{\xi \in \tilde{\Xi}}$, $\tilde{\Pi}_{\text{opt}} := (\tilde{\Pi}^{\xi}_{\text{opt}})_{\xi \in \tilde{\Xi}}$
if $\bar{\Pi}_{\text{opt}} = \emptyset$ **then**
  return (Q) *is infeasible or unbounded*
end
while true **do**
  Let $(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}}, \hat{x}, \hat{\theta})$ be the output of ACGQ $(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}})$
  if $(MQ\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}}))$ *is feasible (i.e.,* $\hat{x} \neq$ null *and* $\hat{\theta} \neq$ null*)* **then**
    lb $\leftarrow c^\top \hat{x} + \hat{\theta}$
    Let $(\xi^*, \bar{\pi}^*)$ be an optimal solution of $(SP\text{-}Feas(\hat{x}, \Xi, \bar{\Pi}_{\text{feas}}))$
    if $\bar{\pi}^{*\top} \left(\bar{h}(\xi^*) - \bar{T}(\xi^*)\hat{x}\right) > 0$ **then**
      $\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \{\xi^*\}$
      $\tilde{\Pi}^{\xi^*}_{\text{feas}} \leftarrow \{\bar{\pi}^*\}$
      $\tilde{\Pi}^{\xi^*}_{\text{opt}} \leftarrow \emptyset$
    else
      Let $(\xi^*, \bar{\pi}^*)$ be an optimal solution of $(SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$
      if $c^\top \hat{x} + \bar{\pi}^{*\top} \left(\bar{h}(\xi^*) - \bar{T}(\xi^*)\hat{x}\right) <$ ub **then**
        ub $\leftarrow c^\top \hat{x} + \bar{\pi}^{*\top} \left(\bar{h}(\xi^*) - \bar{T}(\xi^*)\hat{x}\right)$
        $x^* \leftarrow \hat{x}$, $\theta^* \leftarrow \hat{\theta}$
      end
      if $|\text{ub} - \text{lb}| > \jmath$ *and* $\frac{|\text{ub}-\text{lb}|}{|\text{ub}|+\text{fff}} > \zeta$ **then**
        $\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \{\xi^*\}$
        $\tilde{\Pi}^{\xi^*}_{\text{feas}} \leftarrow \emptyset$
        $\tilde{\Pi}^{\xi^*}_{\text{opt}} \leftarrow \{\bar{\pi}^*\}$
      else
        return *An optimal solution of* (Q) *:* (x^*, θ^*)
      end
    end
  else
    return (Q) *is infeasible*
  end
end

two semi-infinite constraints of the form (55)-(56). In the first case, we further initialize the set $\tilde{\Pi}^{\xi}_{\text{feas}}$ with $\bar{\pi} \in \bar{\Pi}_{\text{feas}}$ found in the cut generation phase. In the second case we do so for the set $\tilde{\Pi}^{\xi}_{\text{opt}}$ with $\bar{\pi} \in \bar{\Pi}_{\text{opt}}$ (while $\tilde{\Pi}^{\xi}_{\text{opt}} = \emptyset$ and $\tilde{\Pi}^{\xi}_{\text{feas}} = \emptyset$, respectively). The complete augmented constraint generation algorithm adapted to formulation (Q) is provided in Algorithm 11.

We next consider formulation (Q) under the relatively complete recourse assumption, which is formalized in the following.

Assumption 5. *For every feasible first-stage solution, and for every uncertainty realization, there is a feasible second-stage solution,* i.e.*,* $\forall \bar{x} \in \bar{\mathcal{X}}, \forall \xi \in \Xi, \exists\, y \in \mathbb{R}^{n_y}_+$ *such that* $\bar{W}y \geq \bar{h}(\xi) - \bar{T}(\xi)x$.

Under Assumption 5, the feasibility phase is no longer necessary since every first-stage solution $\bar{x} \in \bar{\mathcal{X}}$ is feasible for the second-stage problem for all $\xi \in \Xi$. As such, one can directly proceed to the optimality phase in this case. We further remark that all separation problems are guaranteed to be bounded in this case.

Algorithm 11: Augmented constraint generation algorithm for (Q) (ACGQ) $(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}})$

while true **do**
  Solve $(MQ\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}}))$
  if $(MQ\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}}))$ *is feasible* **then**
    $ctr\text{-}added \leftarrow false$, $\tilde{\Xi}_{new} \leftarrow \emptyset$
    Let $(\hat{x}, \hat{\theta})$ be an optimal solution of $(MQ\text{-}SICG(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}}))$ `// master problem phase`
    for $\xi^* \in \tilde{\Xi}$ **do**
      $ctr\text{-}feas \leftarrow false$, $ctr\text{-}opt \leftarrow false$
      do
        $\hat{\xi} \leftarrow \xi^*$
        Let $\bar{\pi}^*$ be an optimal solution of $(SP\text{-}Feas(\hat{x}, \{\hat{\xi}\}, \bar{\Pi}_{\text{feas}}))$ `// feasibility cut generation phase`
        if $\bar{\pi}^{*\top}\left(\bar{h}(\hat{\xi}) - \bar{T}(\hat{\xi})\hat{x}\right) > 0$ **then**
          $ctr\text{-}added \leftarrow true$, $ctr\text{-}feas \leftarrow true$, $ctr\text{-}opt \leftarrow false$
          Let ξ^* be an optimal solution of $(SP\text{-}Feas(\hat{x}, \Xi, \{\bar{\pi}^*\}))$ `// cut selection phase`
        else
          Let $\bar{\pi}^*$ be an optimal solution of $(SP\text{-}Opt(\hat{x}, \{\hat{\xi}\}, \bar{\Pi}_{\text{opt}}))$ `// optimality cut generation phase`
          if $\bar{\pi}^{*\top}\left(\bar{h}(\hat{\xi}) - \bar{T}(\hat{\xi})\hat{x}\right) > \hat{\theta}$ **then**
            $ctr\text{-}added \leftarrow true$, $ctr\text{-}opt \leftarrow true$
            Let ξ^* be an optimal solution of $(SP\text{-}Opt(\hat{x}, \Xi, \{\bar{\pi}^*\}))$ `// cut selection phase`
          end
        end
      while $\bar{\pi}^{*\top}(h(\xi^*) - T(\xi^*)\hat{x}) > \bar{\pi}^{*\top}\left(h(\hat{\xi}) - T(\hat{\xi})\hat{x}\right)$ `// mountain climbing loop`
      if $ctr\text{-}feas$ **then**
        if $\xi^* \in \tilde{\Xi} \cup \tilde{\Xi}_{new}$ **then**
          $\tilde{\Pi}_{\text{feas}}^{\xi^*} \leftarrow \tilde{\Pi}_{\text{feas}}^{\xi^*} \cup \{\bar{\pi}^*\}$
        else
          $\tilde{\Xi}_{new} \leftarrow \tilde{\Xi}_{new} \cup \{\xi^*\}$, $\tilde{\Pi}_{\text{feas}}^{\xi^*} \leftarrow \{\bar{\pi}^*\}$, $\tilde{\Pi}_{\text{opt}}^{\xi^*} \leftarrow \emptyset$
        end
      end
      if $ctr\text{-}opt$ **then**
        if $\xi^* \in \tilde{\Xi} \cup \tilde{\Xi}_{new}$ **then**
          $\tilde{\Pi}_{\text{opt}}^{\xi^*} \leftarrow \tilde{\Pi}_{\text{opt}}^{\xi^*} \cup \{\bar{\pi}^*\}$
        else
          $\tilde{\Xi}_{new} \leftarrow \tilde{\Xi}_{new} \cup \{\xi^*\}$, $\tilde{\Pi}_{\text{feas}}^{\xi^*} \leftarrow \emptyset$, $\tilde{\Pi}_{\text{opt}}^{\xi^*} \leftarrow \{\bar{\pi}^*\}$
        end
      end
    end
    $\tilde{\Xi} \leftarrow \tilde{\Xi} \cup \tilde{\Xi}_{new}$ `// pool expansion phase`
    if $\neg ctr\text{-}added$ **then**
      return $(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}}, \hat{x}, \hat{\theta})$
    end
  else
    return $(\tilde{\Xi}, \tilde{\Pi}_{\text{feas}}, \tilde{\Pi}_{\text{opt}}, \text{null}, \text{null})$
  end
end

Finally, for adjustable robust optimization problems with an optimality problem at the second stage, decomposition algorithms can be based on either formulation (P) or (Q). We remark that, even for problems with the relatively complete recourse property, the subproblems cannot be guaranteed to be feasible under formulation (P) due to the presence of the epigraph constraint. On the other hand, under formulation (Q), two separation phases are necessary to certify the feasibility and optimality of a given solution (unless the problem has the relatively complete recourse property).

D.2 Subproblem reformulations

We next develop the reformulations of the bilinear subproblems $(SP\text{-}Feas(\hat{x}, \Xi, \bar{\Pi}_{\text{feas}}))$ and $(SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$ using LP duality or KKT optimality conditions. Similarly to Section 2.2, to estimate how far the second stage problem is from being feasible in the worst case for a given first-stage solution $\hat{x} \in \bar{\mathcal{X}}$, we use the artificial variables $\alpha \in \mathbb{R}^{n_\alpha}_+$ and any matrix $\bar{N} \in \mathbb{R}^{(m_y-1)\times n_\alpha}_{>0}$, to obtain:

$$\max_{\xi\in\Xi} \min_{\alpha\in\mathbb{R}^{n_\alpha}_+, y\in\mathbb{R}^{n_y}_+} \mathbf{1}^\top\alpha \qquad (SP\text{-}Feas\text{-}ArtMaxMin(\hat{x}))$$

$$\text{s.t.} \quad Wy + N\alpha \geq h(\xi) - T(\xi)\hat{x}. \qquad (57)$$

We then apply LP duality or KKT optimality conditions on the inner minimization problem of $(SP\text{-}Feas\text{-}ArtMaxMin(\hat{x}))$ and obtain the dual set $\bar{\Pi}_F := \{\bar{\pi} \in \mathbb{R}^{m_y-1}_+ \mid \bar{W}^\top\bar{\pi} \leq \mathbf{0}, \bar{N}^\top\bar{\pi} \leq \mathbf{1}\}$. Let, as in Section 2.2, $\Omega \subseteq \mathbb{R}^{n_\omega}$ be a polyhedron with binary extreme points such that the uncertainty set Ξ can be obtained as an affine transformation of Ω, *i.e.*, there exists a matrix $A \in \mathbb{R}^{n_\xi\times n_\omega}$ such that $\Xi := \{A\omega \mid \omega \in \Omega\}$. We thus obtain:

$$\max_{\omega\in\Omega} \max_{\bar{\pi}\in\bar{\Pi}_F} \bar{\pi}^\top\left(\bar{h}(A\omega) - \bar{T}(A\omega)\hat{x}\right), \qquad (SP\text{-}Feas\text{-}Dual(\hat{x}))$$

$$\max_{\xi\in\Xi} \max_{\substack{\alpha\in\mathbb{R}^{n_\alpha}_+, y\in\mathbb{R}^{n_y}_+ \\ \bar{\pi}\in\bar{\Pi}_F}} \mathbf{1}^\top\alpha \qquad (SP\text{-}Feas\text{-}KKT(\hat{x}))$$

$$\begin{aligned} \text{s.t.} \quad & \bar{W}y + \bar{N}\alpha \geq \bar{h}(\xi) - \bar{T}(\xi)\hat{x} \\ & \left(\bar{W}y + \bar{N}\alpha - \bar{h}(\xi) + \bar{T}(\xi)\hat{x}\right) \circ \bar{\pi} = \mathbf{0} \\ & \left(-\bar{W}^\top\bar{\pi}\right) \circ y = \mathbf{0} \\ & \left(\mathbf{1} - \bar{N}^\top\bar{\pi}\right) \circ \alpha = \mathbf{0}. \end{aligned}$$

In a similar manner, for a given feasible first-stage solution $\hat{x} \in \bar{\mathcal{X}}$, we obtain two reformulations of $(SP\text{-}Opt(\hat{x}, \Xi, \bar{\Pi}_{\text{opt}}))$:

$$\max_{\omega\in\Omega} \max_{\bar{\pi}\in\bar{\Pi}_{\text{opt}}} \bar{\pi}^\top\left(\bar{h}(A\omega) - \bar{T}(A\omega)\hat{x}\right), \qquad (SP\text{-}Opt\text{-}Dual(\hat{x}))$$

$$\max_{\xi\in\Xi} \max_{\substack{y\in\mathbb{R}^{n_y}_+ \\ \bar{\pi}\in\bar{\Pi}_{\text{opt}}}} f^\top y \qquad (SP\text{-}Opt\text{-}KKT(\hat{x}))$$

$$\text{s.t.} \quad \bar{W}y \geq \bar{h}(\xi) - \bar{T}(\xi)\hat{x}$$

$$\left(\bar{W}y - \bar{h}(\xi) + \bar{T}(\xi)\hat{x}\right) \circ \bar{\pi} = \mathbf{0}$$
$$\left(f - \bar{W}^\top \bar{\pi}\right) \circ y = \mathbf{0},$$

where $\bar{\Pi}_{\text{opt}}$ is as defined in Section D.

(*SP-Feas-Dual*($\hat{x}$)) and (*SP-Opt-Dual*($\hat{x}$)) can then be linearized by restricting the uncertain parameter ω to be binary and using the McCormick envelope as described in Section 2.2.1. Similarly, (*SP-Feas-KKT*($\hat{x}$)) and (*SP-Opt-KKT*($\hat{x}$)) can be linearized using the *big-M* method described in Section 2.2.2.

E Complementary numerical results

In this section, we present further numerical results. Performance plots are reported for all instances, whereas the data in each table is reported as an average over the instances solved by all methods presented in the table. The number of such instances is provided in the legend of each table. In each table, Time refers to the solution time in seconds, whereas %SP and #SP refer to the percentage of time spent solving the bilinear separation problems and the number of times they are solved, respectively. Finally, #Ctr and $|\tilde{\Xi}|$ refer to the number of constraints and uncertainty realizations added to the master problem. We remark that the value of $|\tilde{\Xi}|$ is not reported for the CG algorithm since, unlike C&CG and SICG algorithms, this algorithm does not generate systems of constraints or semi-infinite constraints associated with an uncertainty realization.

Detailed results for FLT-20 instances: 480/480 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	319.5	89.3	14.4	537.4	13.4
SICG-noPECS	324.6	80.0	14.5	195.3	13.5
SICG-noPE	295.6	83.6	12.1	99.0	11.1
SICG	107.7	70.8	3.9	336.4	108.0
SICG-MC	129.2	72.8	4.4	165.7	58.5
SICG-MCS	94.1	68.9	3.9	269.4	167.2
SICG-MCC	154.9	74.2	5.4	360.2	38.0
SICG-MCSC	122.2	71.1	4.7	632.5	91.7

Detailed results for FLT-30 instances: 234/480 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	1364.9	98.6	12.8	709.7	11.8
SICG-noPECS	1405.9	87.7	12.8	284.9	11.8
SICG-noPE	1363.4	90.9	11.0	167.8	10.0
SICG	473.3	74.5	3.5	1006.4	109.4
SICG-MC	578.1	77.8	4.1	350.7	74.3
SICG-MCS	478.1	75.5	3.7	413.7	136.1
SICG-MCC	741.3	75.2	4.8	1594.4	55.9
SICG-MCSC	716.7	72.9	4.6	2135.1	89.3

Detailed results for ND instances: 309/480 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	1237.8	83.2	43.4	40590.5	42.4
SICG-noPECS	1420.8	70.1	43.9	696.7	42.9
SICG-noPE	866.2	74.9	26.3	291.9	25.3
SICG	383.9	41.3	8.7	1613.8	275.5
SICG-MC	432.8	37.5	8.9	1720.5	244.6
SICG-MCS	440.3	34.1	8.5	2019.4	298.8
SICG-MCC	447.5	43.2	10.2	3241.3	192.8
SICG-MCSC	431.8	39.8	9.7	3959.3	240.3

Figure 5: Tables of detailed results for all algorithms except the CG algorithm on FLT-20, FLT-30, and ND instances. The data in each table is presented as an average over all instances in each category that were solved to optimality by all methods (except CG).

Detailed results for FLT-20 instances, $\Gamma = 0$: 160/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	9.3	75.9	6.1	204.8	5.1
SICG-noPECS	13.3	52.3	6.1	130.6	5.1
SICG-noPE	10.7	57.0	5.7	88.7	4.7
SICG	4.9	36.4	2.2	228.9	21.8
SICG-MC	4.6	36.4	2.3	120.7	19.4
SICG-MCS	4.9	33.9	2.2	125.7	25.9
SICG-MCC	5.0	35.1	2.3	437.1	17.3
SICG-MCSC	5.5	32.9	2.2	523.4	22.4

Detailed results for FLT-20 instances, $\Gamma = 4$: 160/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	97.5	94.1	17.2	647.0	16.2
SICG-noPECS	102.0	89.9	17.2	238.8	16.2
SICG-noPE	98.0	95.0	14.7	115.9	13.7
SICG	37.1	83.3	4.7	358.2	120.6
SICG-MC	38.9	86.4	5.2	197.5	73.5
SICG-MCS	38.7	81.7	4.7	293.4	160.6
SICG-MCC	48.6	90.3	6.5	321.7	46.7
SICG-MCSC	43.1	85.6	5.6	540.0	94.6

Detailed results for FLT-20 instances, $\Gamma = 4.5$: 160/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	851.7	97.9	20.0	760.5	19.0
SICG-noPECS	858.6	97.7	20.0	216.5	19.0
SICG-noPE	778.1	98.7	16.1	92.4	15.1
SICG	281.1	92.7	4.8	422.0	181.6
SICG-MC	344.1	95.5	5.9	178.7	82.6
SICG-MCS	238.9	91.2	4.7	389.0	315.0
SICG-MCC	411.2	97.2	7.4	321.6	49.9
SICG-MCSC	318.0	94.8	6.2	834.3	158.0

Figure 6: Tables of detailed results for all algorithms except the CG algorithm on FLT-20 instances. The data in each table is presented as an average over different Δ values for instances that were solved to optimality by all methods (except CG).

Detailed results for FLT-30 instances, $\Gamma = 0$: 156/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	693.9	98.3	8.9	476.9	7.9
SICG-noPECS	745.7	81.9	8.9	319.9	7.9
SICG-noPE	643.5	86.4	8.3	209.1	7.3
SICG	145.6	62.4	2.3	1362.5	101.6
SICG-MC	130.1	67.1	2.3	458.5	80.5
SICG-MCS	130.1	63.9	2.3	502.2	125.8
SICG-MCC	162.8	63.1	2.4	2278.6	62.8
SICG-MCSC	170.0	59.7	2.4	3030.7	97.4

Detailed results for FLT-30 instances, $\Gamma = 6$: 52/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	3126.3	99.3	20.5	1168.8	19.5
SICG-noPECS	3157.3	99.1	20.5	268.3	19.5
SICG-noPE	3303.0	99.8	17.2	105.2	16.2
SICG	1404.3	98.7	6.2	370.0	146.9
SICG-MC	1647.4	99.3	7.3	178.1	74.0
SICG-MCS	1360.9	98.8	6.5	320.3	193.8
SICG-MCC	2092.5	99.5	9.2	294.8	50.1
SICG-MCSC	2137.9	99.3	8.9	454.9	88.2

Detailed results for FLT-30 instances, $\Gamma = 6.5$: 26/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	1867.6	99.3	20.8	1188.5	19.8
SICG-noPECS	1864.1	99.5	20.8	107.9	19.8
SICG-noPE	1803.8	99.8	15.2	45.0	14.2
SICG	577.7	98.7	4.9	142.9	81.5
SICG-MC	1126.9	99.6	8.1	49.3	37.6
SICG-MCS	800.6	98.9	6.7	70.0	82.3
SICG-MCC	1510.1	99.8	10.8	88.4	25.8
SICG-MCSC	1154.4	99.6	9.6	122.0	42.7

Figure 7: Tables of detailed results for all algorithms except the CG algorithm on FLT-30 instances. The data in each table is presented as an average over different Δ values for instances that were solved to optimality by all methods (except CG).

Detailed results for ND instances, $\Gamma = 0$: 131/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	615.8	76.3	21.2	22856.0	20.2
SICG-noPECS	840.7	57.1	21.7	485.0	20.7
SICG-noPE	523.9	63.3	11.2	166.6	10.2
SICG	187.1	32.8	3.1	724.0	79.7
SICG-MC	209.1	29.8	3.1	786.2	75.9
SICG-MCS	211.8	26.6	2.9	901.5	87.7
SICG-MCC	199.9	31.2	3.2	1703.8	70.7
SICG-MCSC	181.2	28.7	3.0	1987.0	82.5

Detailed results for ND instances, $\Gamma = 1$: 116/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	1534.4	85.1	49.5	48032.2	48.5
SICG-noPECS	1682.8	75.3	50.0	824.4	49.0
SICG-noPE	877.7	79.9	29.8	364.9	28.8
SICG	398.6	40.3	8.3	2061.7	321.9
SICG-MC	457.6	35.4	8.4	2175.3	285.3
SICG-MCS	476.6	31.8	7.8	2617.8	348.5
SICG-MCC	465.5	43.9	9.8	4055.8	228.8
SICG-MCSC	450.2	40.1	9.3	4975.5	288.5

Detailed results for ND instances, $\Gamma = 1.5$: 62/160 instances

Method	Time	%SP	#SP	#Ctr	$\lvert\tilde{\Xi}\rvert$
C&CG	1996.9	94.1	78.9	64138.7	77.9
SICG-noPECS	2156.3	88.0	79.4	904.8	78.4
SICG-noPE	1567.7	90.0	51.5	420.4	50.5
SICG	772.3	61.2	21.6	2655.7	602.4
SICG-MC	859.3	57.6	22.3	2843.7	524.8
SICG-MCS	855.3	54.5	21.5	3261.9	651.7
SICG-MCC	937.1	67.4	25.7	4966.1	383.4
SICG-MCSC	927.1	62.9	24.7	6225.6	483.4

Figure 8: Tables of detailed results for all algorithms except the CG algorithm on ND instances. The data in each table is presented as an average over different Δ values for instances that were solved to optimality by all methods (except CG).

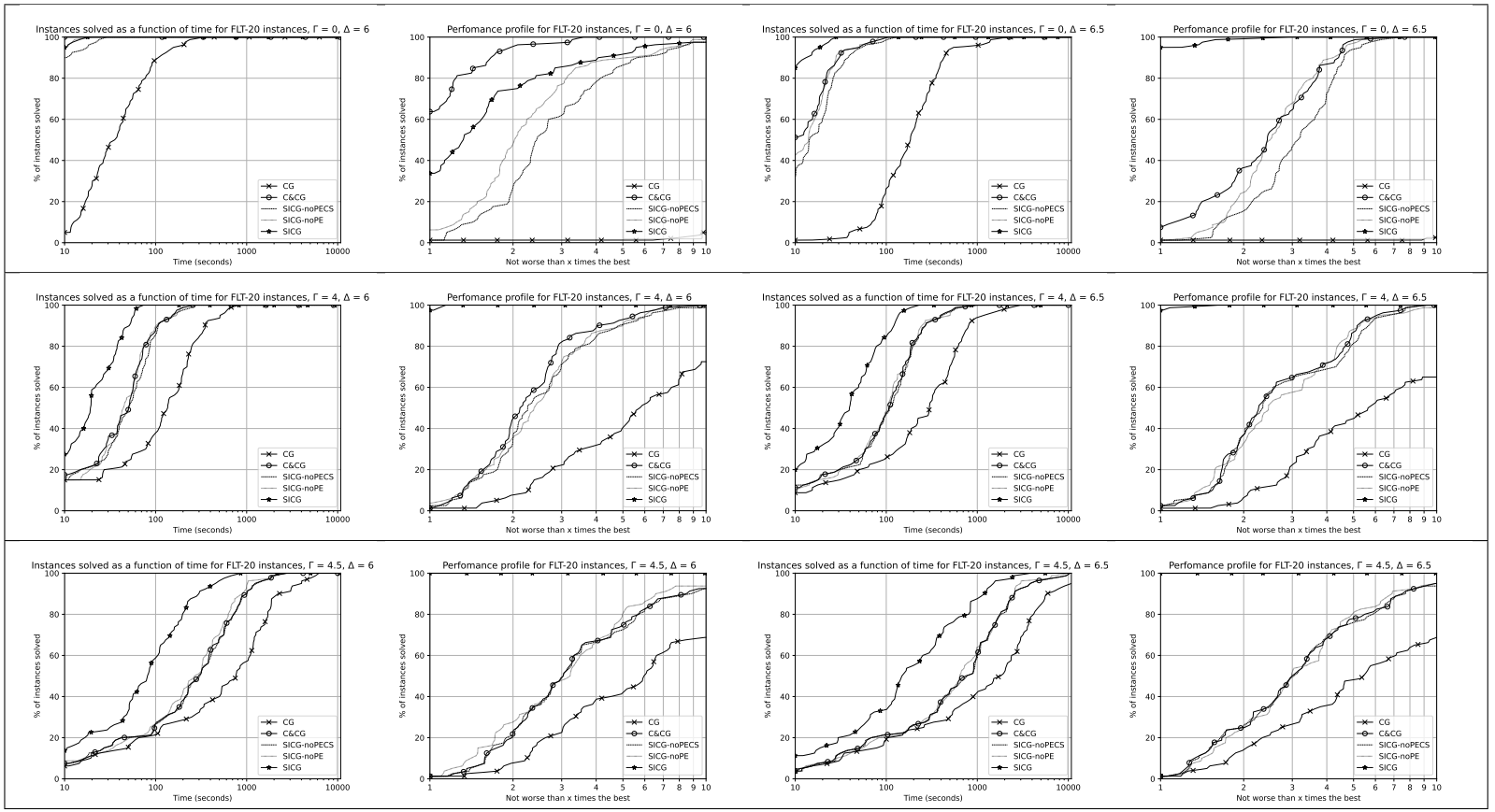


Figure 9: Performance plots of the CG, C&CG, SICG-noPECS, SICG-noPE and SICG algorithms on FLT-20 instances detailed for all Γ and Δ values.

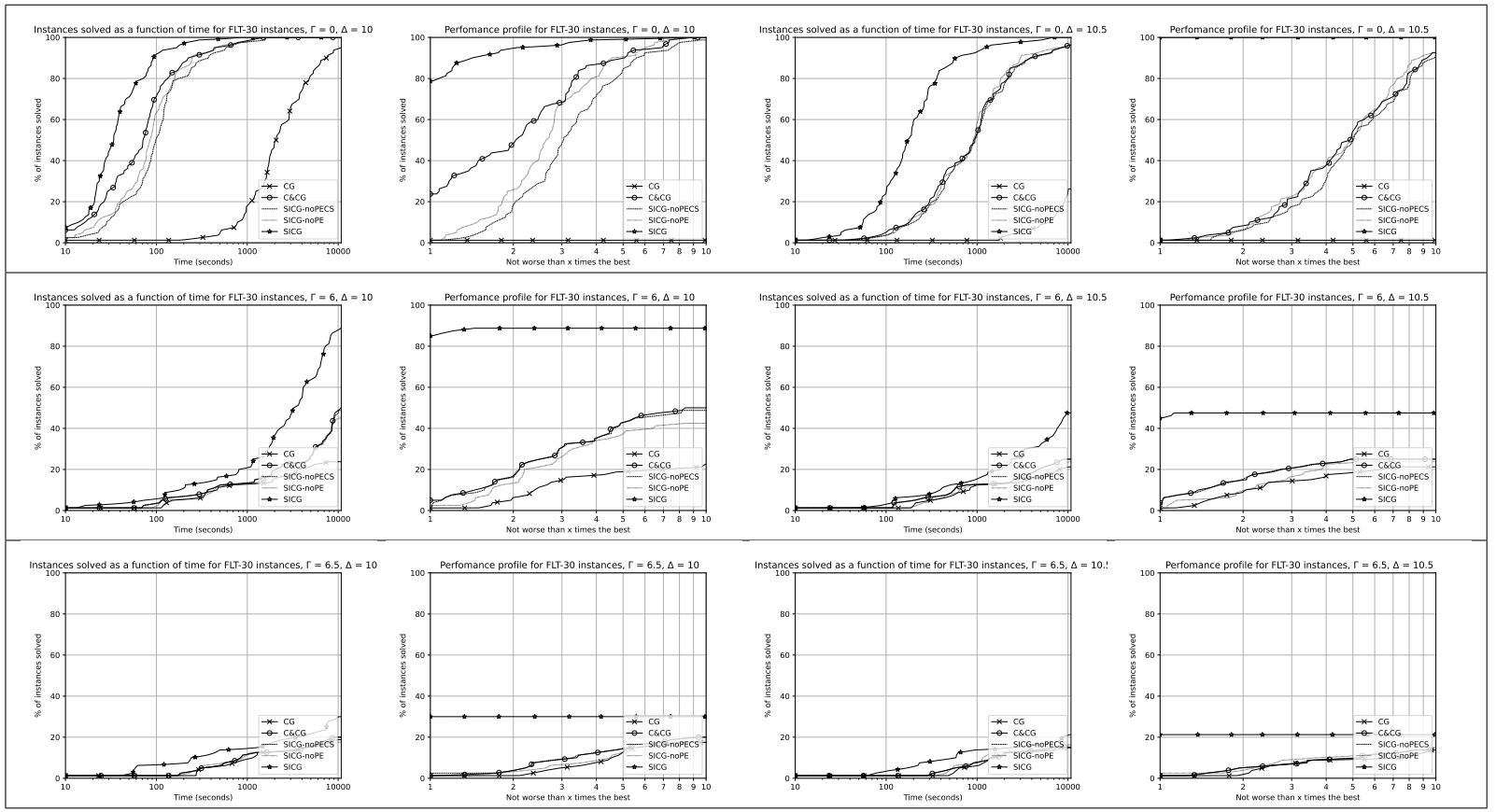


Figure 10: Performance plots of the CG, C&CG, SICG-noPECS, SICG-noPE and SICG algorithms on FLT-30 instances detailed for all Γ and Δ values.

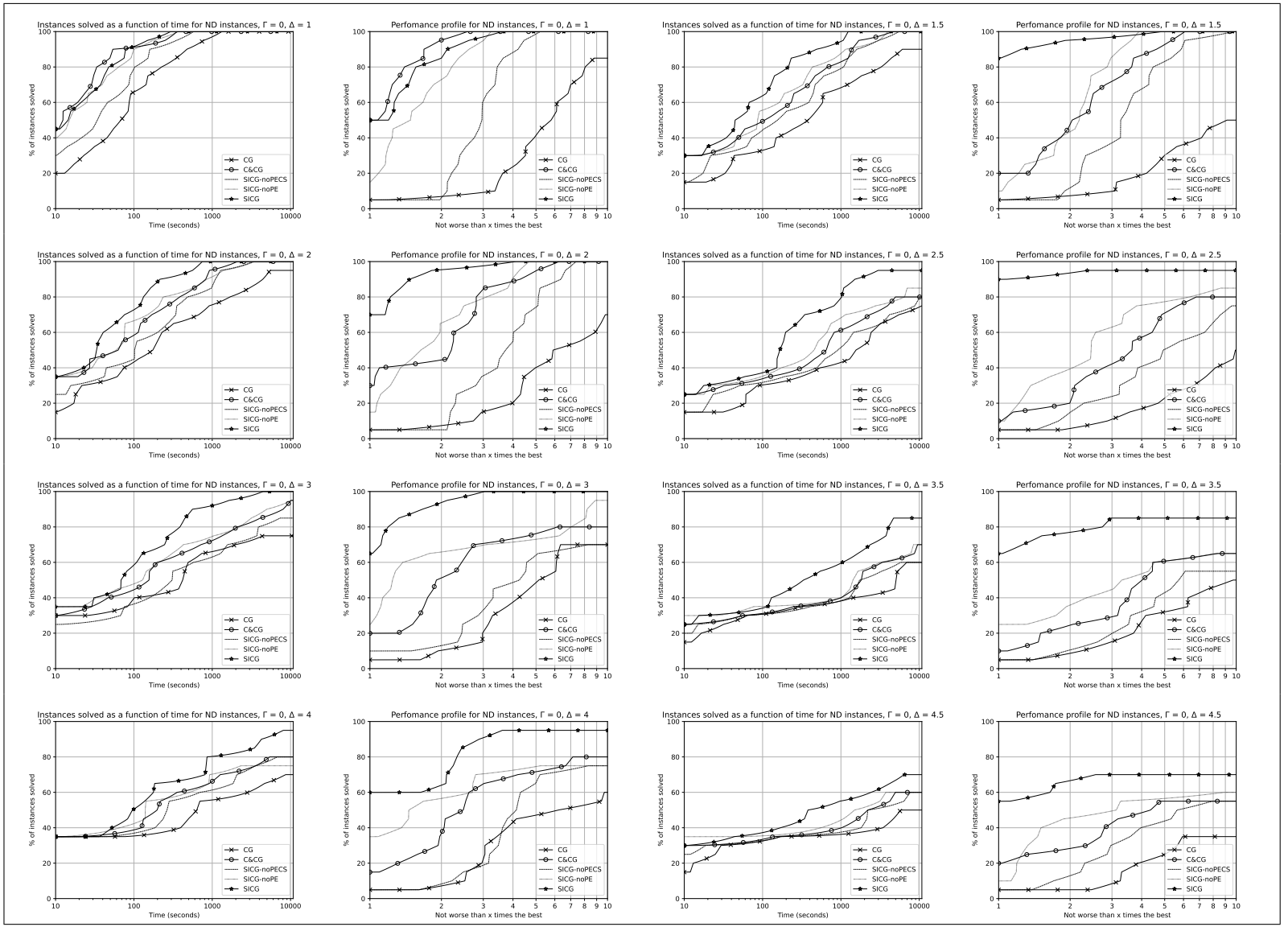


Figure 11: Performance plots of the CG, C&CG, SICG-noPECS, SICG-noPE and SICG algorithms on ND instances with $\Gamma = 0$.

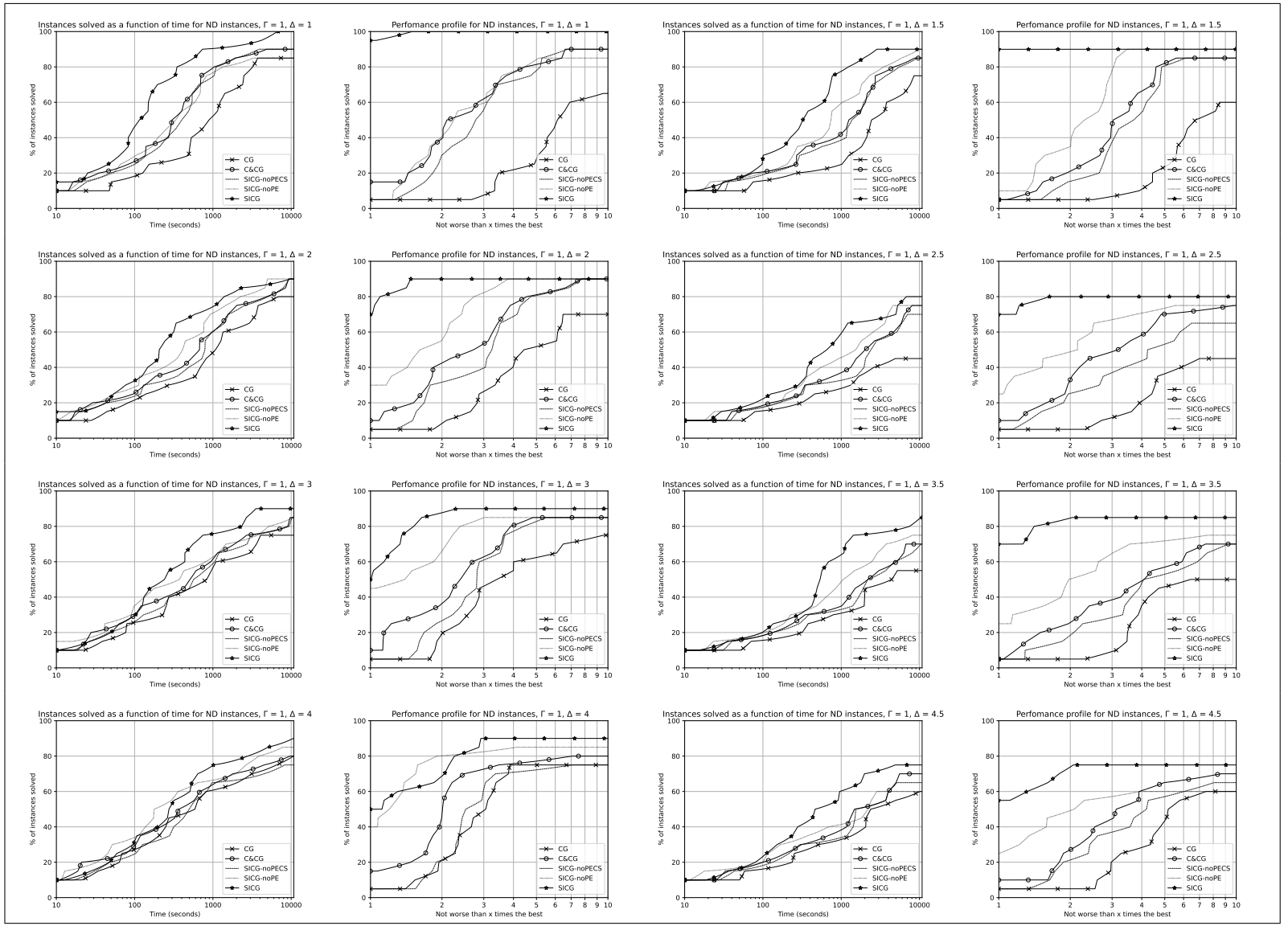


Figure 12: Performance plots of the CG, C&CG, SICG-noPECS, SICG-noPE and SICG algorithms on ND instances with $\Gamma = 1$.

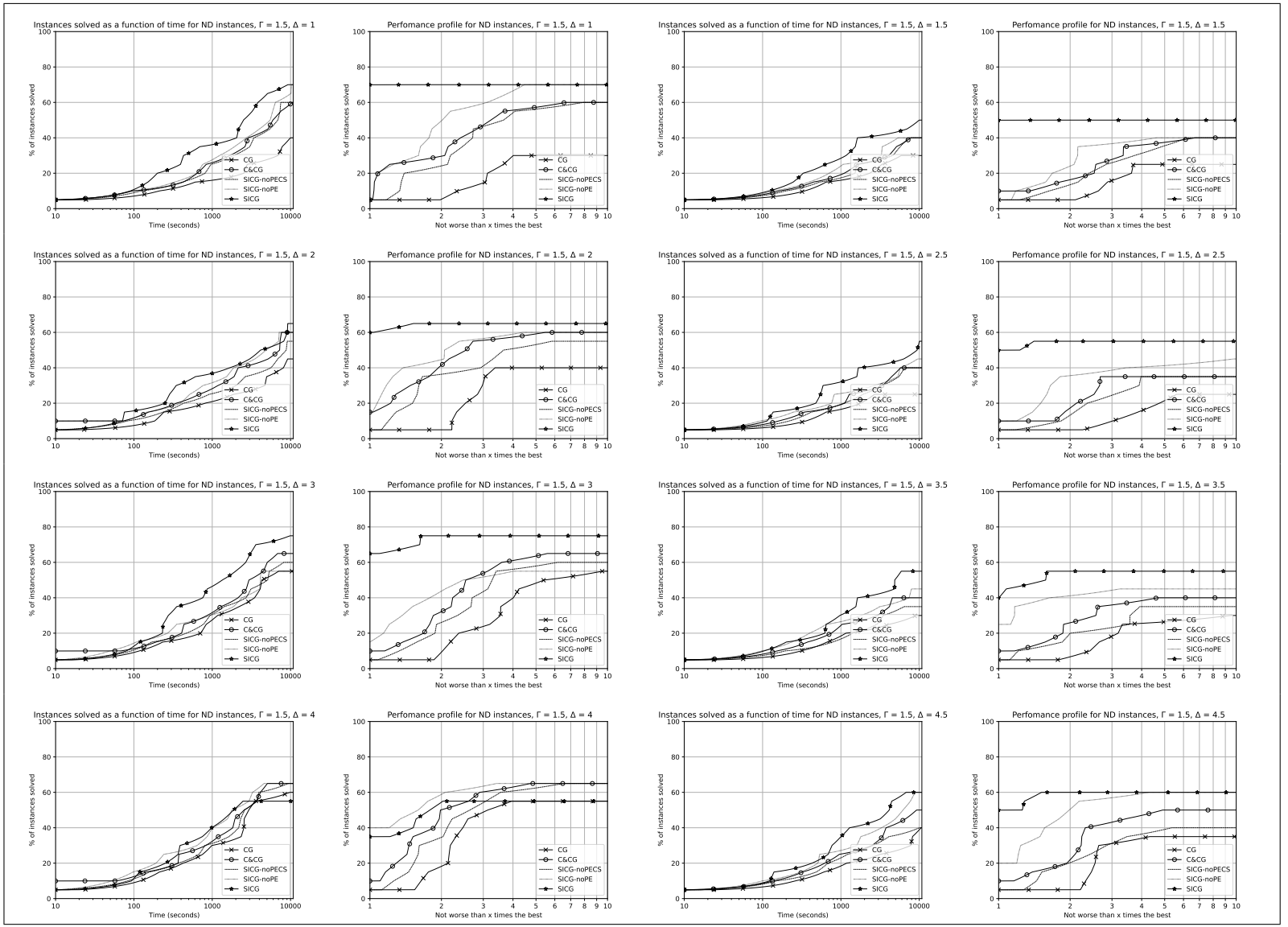


Figure 13: Performance plots of the CG, C&CG, SICG-noPECS, SICG-noPE and SICG algorithms on ND instances with $\Gamma = 1.5$.

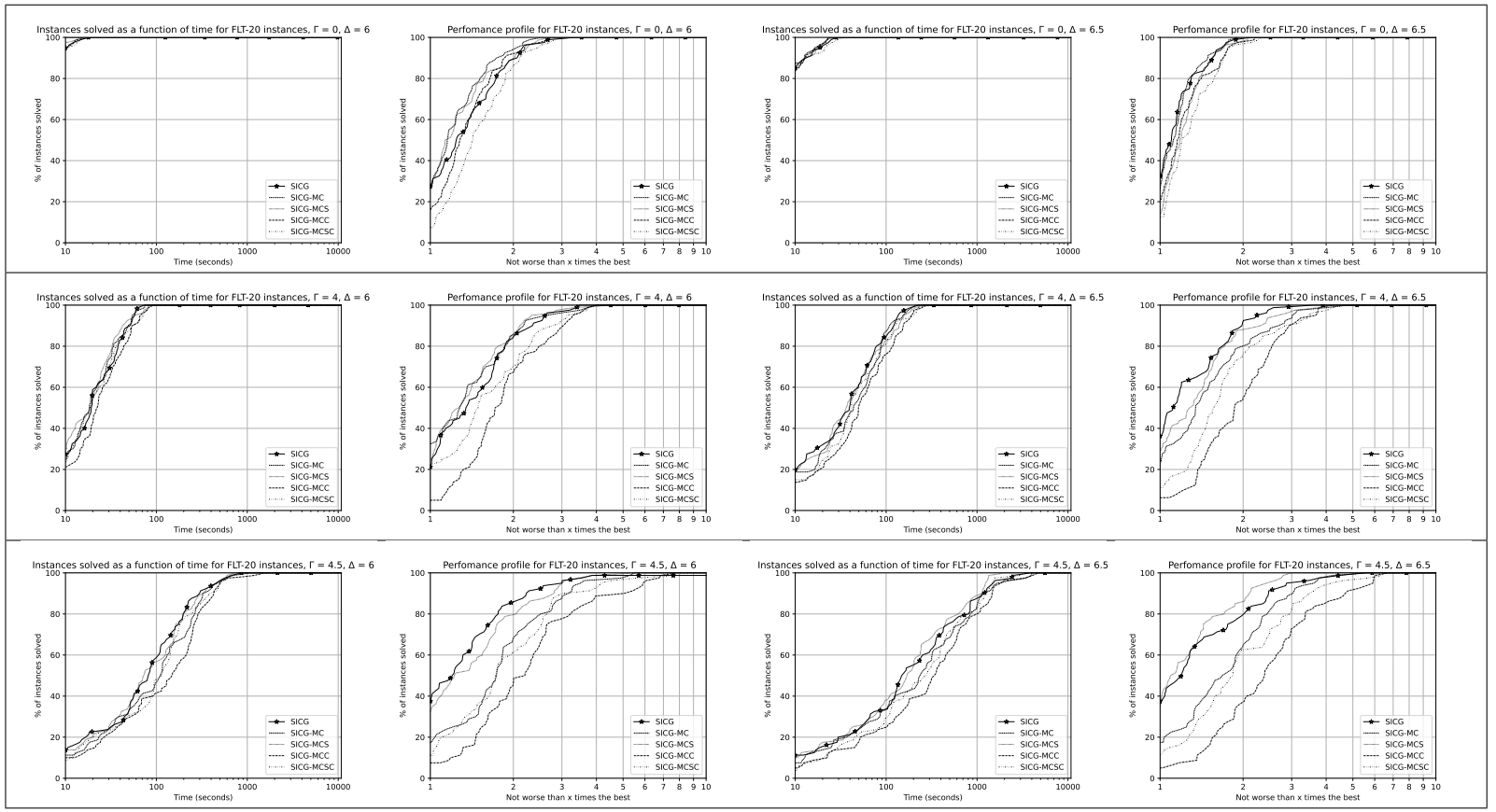


Figure 14: Performance plots of the SICG algorithm and its mountain climbing variants on FLT-20 instances detailed for all Γ and Δ values.

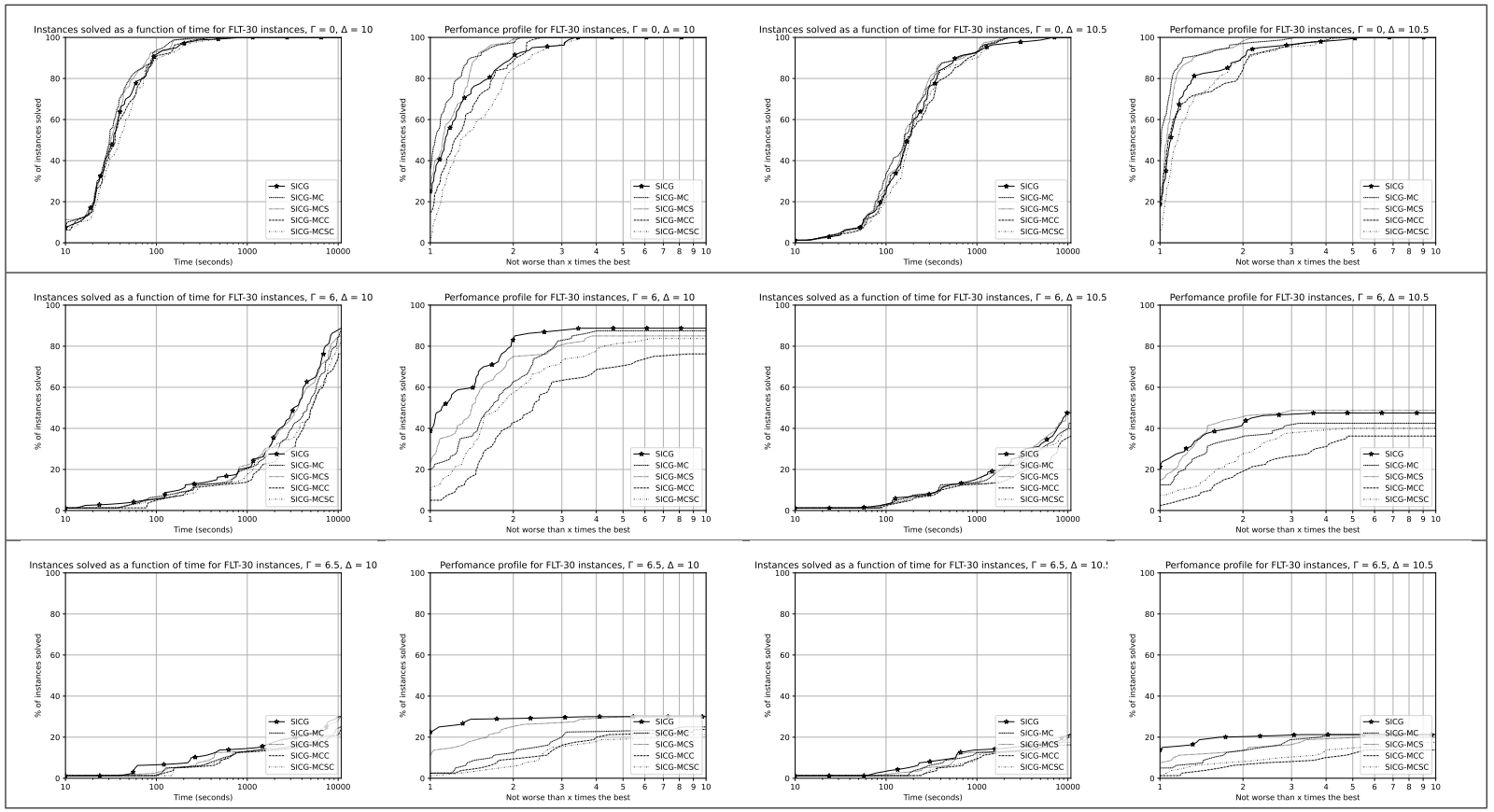


Figure 15: Performance plots of the SICG algorithm and its mountain climbing variants on FLT-30 instances detailed for all Γ and Δ values.

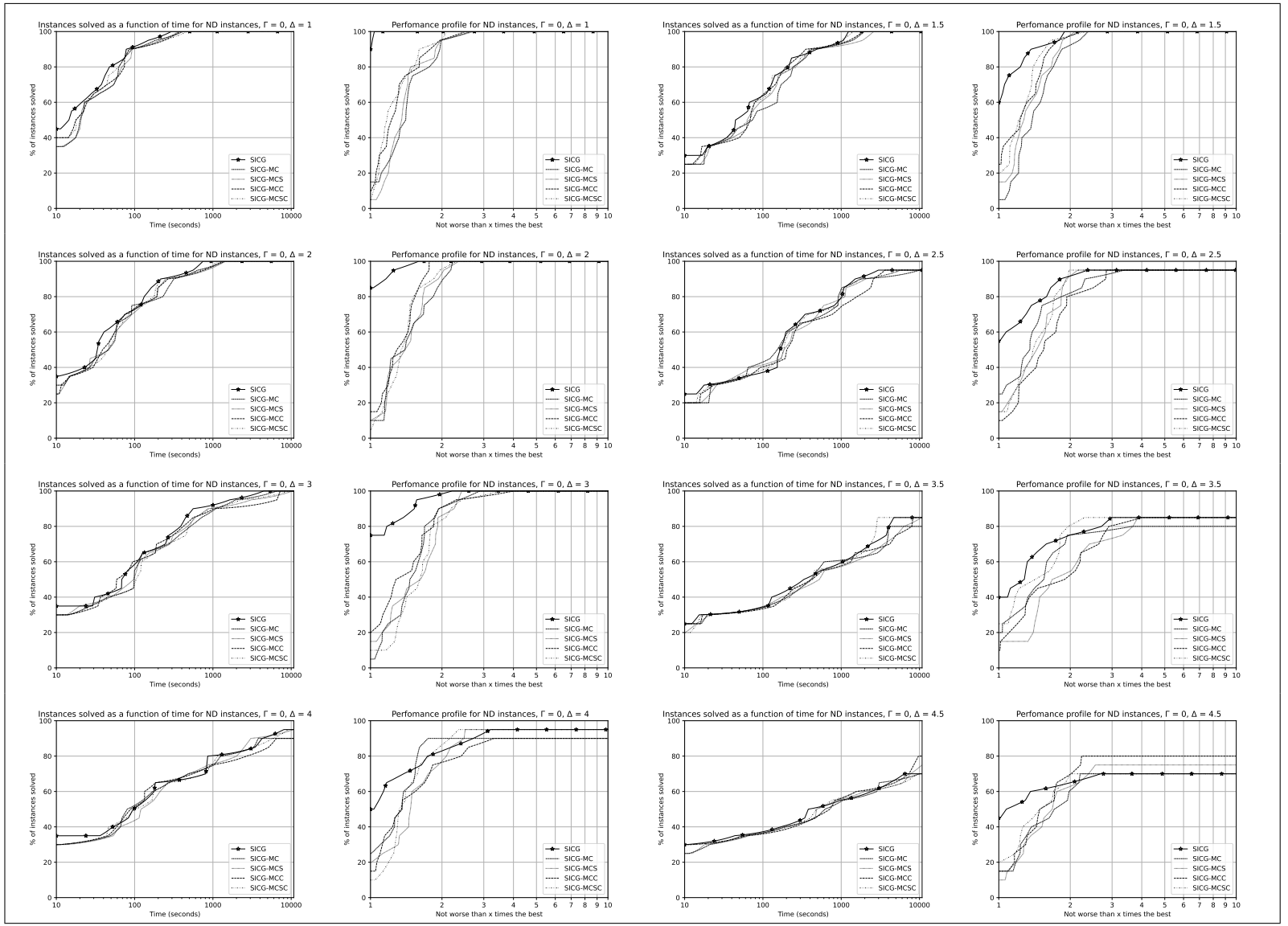


Figure 16: Performance plots of the SICG algorithm and its mountain climbing variants on ND instances with $\Gamma = 0$.

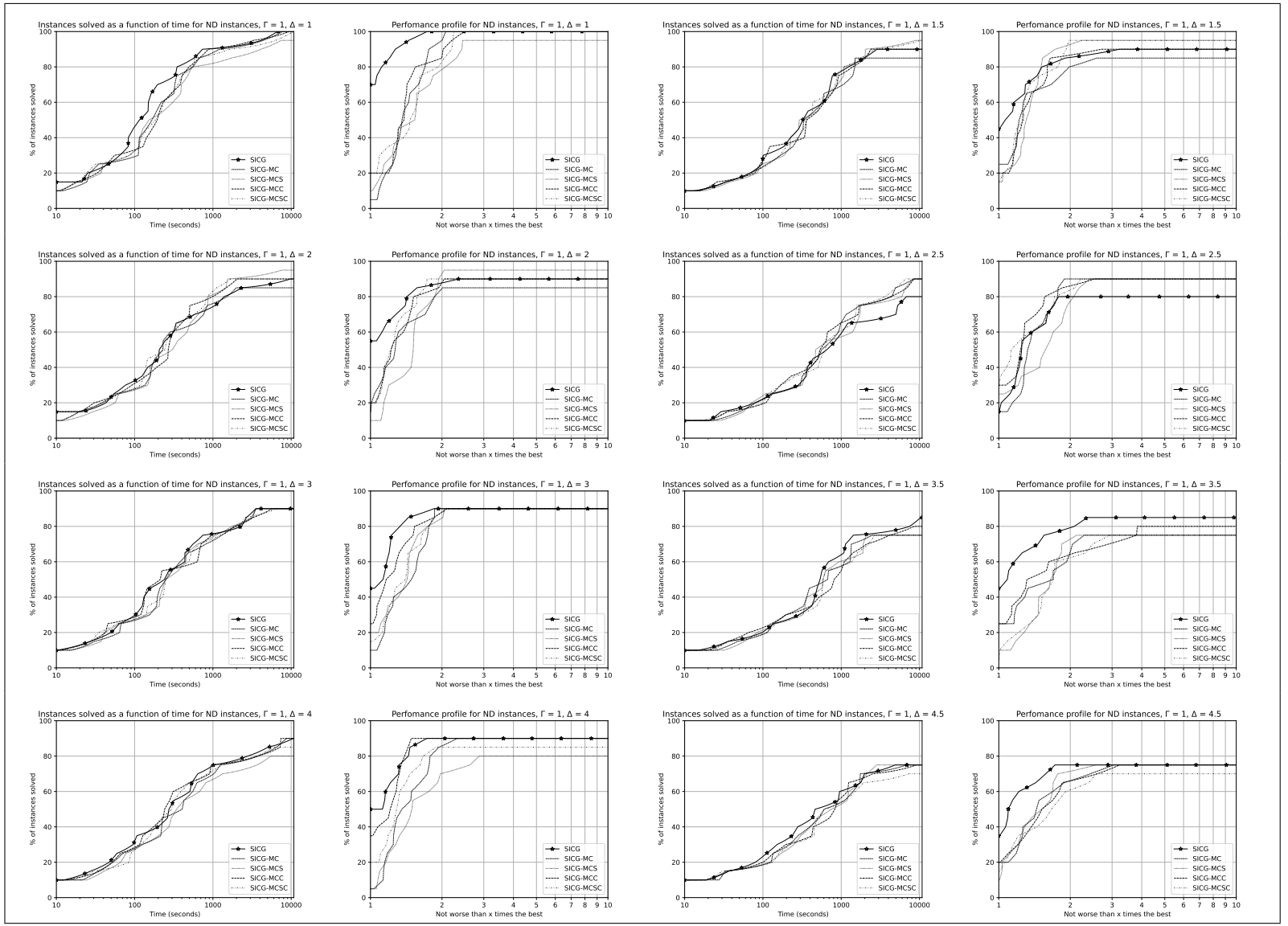


Figure 17: Performance plots of the SICG algorithm and its mountain climbing variants on ND instances with $\Gamma = 1$.

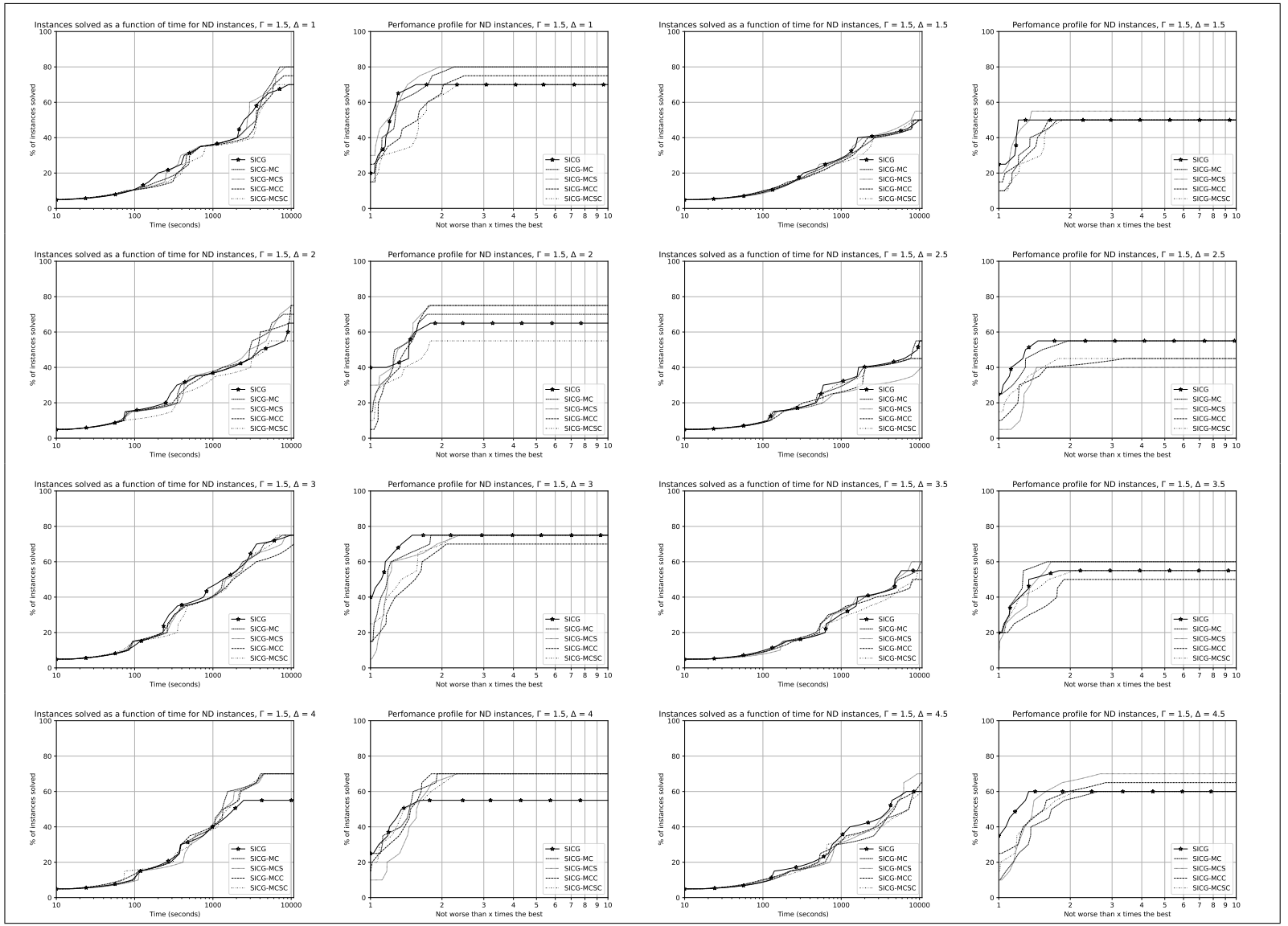


Figure 18: Performance plots of the SICG algorithm and its mountain climbing variants on ND instances with $\Gamma = 1.5$.